Foundations of the Synthetic Control Method

Jared Greathouse

Table of contents

1 Introduction **1**
- 1.1 Why I Wrote This Book? . . . 1
- 1.2 Who This Book Is For . . . 2
- 1.3 Personal Reflection . . . 2
- 1.4 Structure of the Book . . . 3

I Foundations **5**

2 Mathematical Preliminaries **7**
- 2.1 Objects . . . 7
- 2.2 Matrices and Vectors . . . 9
- 2.3 Calculus . . . 16
- 2.4 Optimization . . . 20
- 2.5 Convexity . . . 22
- 2.6 Inequalities . . . 23
- 2.7 Conclusion . . . 29

3 Averages **31**
- 3.1 Averaging in Statistics . . . 31
- 3.2 Kinds of Arithmetic Averages . . . 34
- 3.3 What Are We Weighting For? Introducing Weighted Averages . . . 45
- 3.4 Conclusion . . . 50

4 Getting Started with mlsynth **53**
- 4.1 Installing mlsynth . . . 53
- 4.2 One pattern, every estimator . . . 54
- 4.3 What you get back . . . 55
- 4.4 Preparing data with dataprep . . . 58
- 4.5 Looking before you estimate . . . 59
- 4.6 Conclusion . . . 60

II Estimation and Inference **61**

5 Potential Outcomes and Comparative Case Studies **63**
5.1 The Fundamental Problem of Causal Inference . . . 64
5.2 Counterfactuals and the Role of Controls . . . 66
5.3 What Is a Comparative Case Study? . . . 70
5.4 West Germany . . . 71
5.5 Mexican Drug War . . . 72
5.6 Marketing Measurement . . . 73
5.7 Beyond Simple Control Selection . . . 73
5.8 Looking Ahead . . . 74

6 Using The Control Group Mean **77**
6.1 Two Way Fixed Effects . . . 77
6.2 Demeaning and Difference-in-Differences . . . 78
6.3 Counterfactuals Under DID . . . 79
6.4 Parallel Trends in Small N Settings . . . 82

7 Using Least Squares **89**
7.1 The Model . . . 89
7.2 Counterfactuals Generated by OLS . . . 90
7.3 The Problem with OLS . . . 94
7.4 Where OLS Leaves Off . . . 94

8 Convex Synthetic Controls **97**
8.1 Convex Least Squares . . . 97
8.2 Returning to the Basque Example . . . 103
8.3 Finite Sample Error Under The Linear Factor Model . . . 104
8.4 Conclusion . . . 110
8.5 Asymptotic Bias Bound . . . 110

III Further Topics **117**

9 Penalized Synthetic Controls **119**
9.1 Cross Validation . . . 124
9.2 Implementing Penalized OLS Models . . . 127
9.3 Principal Components Regression . . . 128
9.4 Downsides of Penalized Least Squares . . . 134
9.5 Onward to Convex SC . . . 134

Chapter 1

Introduction

Hey, I'm Jared, a PhD candidate studying public policy at Georgia State University. I work in econometrics for policy analysis. Welcome to *Foundations of the Synthetic Control Method* (SCM). This book is a practical guide to mastering SCM, directed at graduate students and industry scientists seeking an intuitive yet rigorous understanding of this transformative method for policy and geolift analysis (the study of location-based impacts in marketing, using holdout studies).

One idea runs through every chapter: a synthetic control is just a carefully chosen weighted average. We build it up slowly — from a plain mean, to regression, to the convex synthetic control — so that nothing ever arrives as a formula you are asked to accept on faith.

1.1 Why I Wrote This Book?

While excellent articles on SCM exist, they often skip critical details that cannot fit in a single paper. This book bridges that gap, offering a clear, hands-on introduction to SCM's mechanics and applications without overwhelming readers with advanced theory.

I remember reading in masters school articles/book chapters (Nguyen, 2025; Stöffelbauer, 2023) that said things like

> In many real-world cases, however, we actually find ourselves more interested in understanding a causal effect rather than correlations. Part of the reason is that causal effects are more robust because they do not suffer as much from model drift, and they are a great tool for supporting data-driven decisions. This article is about one such method, called the synthetic control method (SCM). In short, its main idea is to use a synthetic control group instead of a randomized control group, which is what is typically done in A/B testing. As a result, SCM is significantly easier to apply in various use cases.

or

> The Synthetic Control Method (SCM) is a powerful causal inference tool used

> when only one treated unit exists, and researchers must construct a synthetic comparison group from a donor pool of untreated units.

These definitions only confused me. Both of them used the word "synthetic" in the definition. Other articles would simply present what I now know to be the dot product or the convex optimization formula, say a few words about it, and move on. Almost no resource, that I could find, bothered to start from the barebones first principles and walk through what was meant as a concept by a "fake unit" or a "weighted average" or how it differed from a regular average. So, this book has a plain quest: explain the synthetic control method to students and working professionals who may use it for their own tasks.

1.2 Who This Book Is For

This book is for graduate students and practitioners in economics, social sciences, or related fields who want to apply SCM in practice. No prior SCM experience is needed, but readers should be comfortable with algebra, causal reasoning, and basic econometrics such as regression analysis. You do not need linear algebra or calculus going in: the book builds the specific math it uses — vectors, dot products, matrices, a derivative here and there — from scratch, at the moment each idea is first needed, rather than front-loading it in a chapter you have to survive before the payoff. If you want a full course in those subjects, the standard references are excellent (e.g., Gilbert Strang's Linear Algebra for the linear algebra, the numerous open source texts on causal inference, or the list of free Python books).

This is the master's volume. It deliberately stops short of the advanced extensions — proximal inference, matrix completion, staggered adoption, and the heavier asymptotic theory — which are the subject of a planned second, more technical volume. Where those methods matter, the closing chapter points to them; it does not develop them here.

1.3 Personal Reflection

1.3.1 Historical & Methodological Context

Given the advent of the computing advances of the 20th century, economics, public policy/political science, and related fields developed an increasing reliance on computing methods in academic and industrial research. Part of this transition involved a more careful concern with causal impact estimation methods. Sometimes called the credibility revolution, social scientists of all varieties began to ask questions such as "What was the effect of Policy X on Outcome Y?"

In the early 1990s and 2000s, methods such as propensity score matching, difference-in-differences, instrumental variable regression models, and others became important weapons in the toolkit of empirical scientists. Across the 2000s and 2010s, the SCM — developed by Alberto Abadie and his advisor Javier Gardeazabal — was formalized in articles published in *Journal of the American Statistical Association* and *American Journal of Political Science*.

1.3.2 My Journey with SCM

My journey with the SCM began in 2021. I had just become a PhD student of public policy at Georgia State University, and then the Georgia Institute of Technology. The COVID-19 pandemic had begun only a year and a few months earlier. Young and aspiring economists, computer scientists, and policy scholars rushed to use SCMs to evaluate the effect of public policy interventions on COVID-19 cases, vaccination outcomes, and other outcomes of interest.

Throughout my undergraduate years, I had heard about how SCM was a kind of generalization of the difference-in-differences method (another design one could write several books on). However, the reasoning for this to me was very arcane. Furthermore, the math behind SCM intimidated me; I was a starting-level PhD student with little background in calculus or real analysis. Either way, these questions burnt at me from within, and I sought to develop a better understanding of SCM for my own work. This is the book I wish I had learning the method years ago, coming from a very non-technical background.

1.4 Structure of the Book

The book has one organizing idea: we build the counterfactual — the synthetic control — one rung at a time, each method motivated by the limits of the one before it. It is organized in three parts.

1. *Foundations.* The mathematical vocabulary you will actually use, introduced as it comes up, and the central insight that every estimator in the book is a structured weighted average.
2. *Estimation and inference.* The ladder itself: from the potential-outcomes framing and the plain control-group mean, through least squares and penalized regression, up to the convex synthetic control — and then how to validate an estimate and attach uncertainty to it.
3. *Applications.* How to choose a donor pool, a full end-to-end study, and a short closing survey of what lies beyond this volume — proximal methods, matrix completion, staggered designs — as signposts to the second book rather than a treatment of them.

Each section ends with problems to work, and every method is demonstrated in `mlsynth`, the companion Python library, so you can run everything as you read.

Part I

Foundations

Chapter 2

Mathematical Preliminaries

> "If you stop at general math, you're only going to make general math money."
> — Snoop Dogg

This chapter collects the minimal, concrete mathematical vocabulary used throughout the book. Worked examples illustrate each concept. These tools will appear frequently in synthetic control methods: sets, vectors, matrices, norms, derivatives, objective functions, and convexity.

2.1 Objects

Definition 2.1 (Topological Space)**.** A *topological space* is a set equipped with a notion of "closeness" or limits between elements. For our purposes, the most common topological space is the set of real numbers:

$$\mathbb{R} = \text{the set of real numbers.}$$

Worked example: $y_1 = 2.5, y_2 = 4.0, y_3 = 3.1 \implies (y_1, y_2, y_3) \in \mathbb{R}^3$.

This is also called the real line. The study of the real numbers and the functions that live within it is called *Real Analysis*. However, you will not need any real analysis to read this book.

Definition 2.2 (Set)**.** A *set* is a collection of distinct and well-defined objects, called *elements* or *members* of the set. If an object x belongs to a set $\mathcal{A}$, we write $x \in \mathcal{A}$; otherwise, $x \notin \mathcal{A}$. Sets are unordered and contain no duplicates. Two sets are equal if and only if they contain exactly the same elements. Or more formally, by the axiom of extensionality:

$$\mathcal{A} = \mathcal{B} \iff (\forall x)(x \in \mathcal{A} \leftrightarrow x \in \mathcal{B}).$$

The set with no elements is called the empty set, denoted $\emptyset$. For example:

$$\mathcal{A} = \{1, 2, 3\}, \quad \mathcal{B} = \{x \in \mathbb{N} : x \text{ is even}\}, \quad \mathcal{C} = \emptyset.$$

The union of two sets $\mathcal{A}$ and $\mathcal{B}$ is the set of all elements that belong to either $\mathcal{A}$ or $\mathcal{B}$:

$$\mathcal{A} \cup \mathcal{B} = \{x : x \in \mathcal{A} \text{ or } x \in \mathcal{B}\}.$$

The intersection of two sets $\mathcal{A}$ and $\mathcal{B}$ is the set of all elements that belong to both $\mathcal{A}$ and $\mathcal{B}$:

$$\mathcal{A} \cap \mathcal{B} = \{x : x \in \mathcal{A} \text{ and } x \in \mathcal{B}\}.$$

The difference of two sets $\mathcal{A}$ and $\mathcal{B}$ is the set of elements in $\mathcal{A}$ but not in $\mathcal{B}$:

$$\mathcal{A} \setminus \mathcal{B} = \{x : x \in \mathcal{A} \text{ and } x \notin \mathcal{B}\}.$$

If $\mathcal{U}$ is the universal set, the complement of a set $\mathcal{A}$ is:

$$\mathcal{A}^c = \{x \in \mathcal{U} : x \notin \mathcal{A}\}.$$

The cardinality of a set $\mathcal{A}$ is the number of elements in the set, denoted $|\mathcal{A}|$.

In econometrics, sets appear frequently to organize units, time periods, and other objects of interest. For example, let $\mathcal{N}$ denote the set of all units, and $\mathcal{N}_0 \subseteq \mathcal{N}$ the donor set. Let $\mathcal{T}$ be the set of all time periods, with $\mathcal{T}_1 = \{t \in \mathbb{N} : t \leq T_0\} \subseteq \mathcal{T}$ representing the pre-intervention periods, and $\mathcal{T}_2 = \{t \in \mathbb{N} : t > T_0\} \subseteq \mathcal{T}$ representing the post-treatment periods. Sets in econometrics can contain numbers, individuals, time periods, or even other sets, depending on the application.

Definition 2.3 (Scalar)**.** A *scalar* is a single real number, $x \in \mathbb{R}$.

Worked example: $c = 2.5 \cdot q$; if $q = 4$, $c = 10$.
In SCM, $y_{jt} \in \mathbb{R}$ is an observed outcome, $d_j \in \{0, 1\}$ is treatment.

We work with these all the time.

Definition 2.4 (Summation)**.** For numbers $x_1, \ldots, x_n$:

$$\sum_{i=1}^{n} x_i = x_1 + x_2 + \cdots + x_n$$

Worked example: $x_1 = 2, x_2 = 4, x_3 = 5 \implies \sum_{i=1}^{3} x_i = 11$.

This offers a convenient way to express summations of scalars.

Definition 2.5 (Sequence)**.** A *sequence* is a function

$$a : \mathbb{N} \to S$$

where $\mathbb{N} = \{1, 2, 3, \dots\}$ is the set of natural numbers, and S is some set (often $\mathbb{R}$). We usually denote a sequence by $(a_n)_{n\in\mathbb{N}}$ or simply (a_n), where

$$a_n = a(n)$$

is the n-th term of the sequence. In words, a sequence is an *ordered list of elements* $(a_1, a_2, a_3, \dots)$ indexed by natural numbers. For example, the sequence of natural numbers: $(1, 2, 3, 4, \dots)$, where $a_n = n$ and the sequence of reciprocals: $\left(1, \frac{1}{2}, \frac{1}{3}, \dots\right)$, where $a_n = \frac{1}{n}$.

2.2 Matrices and Vectors

Scalars can be collected into rows and columns, which we call elements of vectors (single row/column sequences of scalars) or matrices (tables of vectors).

Definition 2.6 (Vector)**.** An *n-dimensional vector* is an ordered collection of scalars of *length* $n \in \mathbb{N}$:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^n.$$

Worked example:

$$\mathbf{v} = \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix} \in \mathbb{R}^3.$$

Definition 2.7 (Transpose)**.** The *transpose* of a vector $\mathbf{x}$, denoted $\mathbf{x}^\top$, is a row vector formed by flipping the vector:

$$\mathbf{x}^\top = \begin{bmatrix} x_1 & x_2 & \cdots & x_n \end{bmatrix}.$$

Worked example:
If

$$\mathbf{v} = \begin{bmatrix} 3 \\ 5 \\ 7 \end{bmatrix}, \quad \text{then } \mathbf{v}^\top = \begin{bmatrix} 3 & 5 & 7 \end{bmatrix}.$$

Definition 2.8 (Dot Product)**.** The *dot product* of two vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$ is

$$\mathbf{a}^\top \mathbf{b} = \sum_{i=1}^{n} a_i b_i.$$

Worked example:
Let

$$\mathbf{a} = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} 1 \\ 0 \\ 3 \end{bmatrix}.$$

Then

$$\mathbf{a}^\top \mathbf{b} = 2 \cdot 1 + 4 \cdot 0 + 6 \cdot 3 = 20.$$

Definition 2.9 (Matrix)**.** A *matrix* is a two-dimensional array (a table) of scalars arranged in rows and columns:

$$\mathbf{A} = \begin{bmatrix} a_{11} & \dots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \dots & a_{mn} \end{bmatrix} \in \mathbb{R}^{m \times n}.$$

Here, m is the number of rows and n the number of columns.

Worked example:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \in \mathbb{R}^{2 \times 3}.$$

As we can see, matrices are simple tables of vectors that we use to organize data into.

Definition 2.10 (Matrix Transpose)**.** The *transpose* of a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, written $\mathbf{A}^\top$, is obtained by interchanging its rows and columns:

$$(\mathbf{A}^\top)_{ij} = a_{ji}, \quad \mathbf{A}^\top \in \mathbb{R}^{n \times m}.$$

Worked example:
If

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix},$$

then

$$\mathbf{A}^\top = \begin{bmatrix} 1 & 4 \\ 2 & 5 \\ 3 & 6 \end{bmatrix}.$$

Definition 2.11 (Matrix–Vector and Vector–Matrix Products)**.** For $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{x} \in \mathbb{R}^n$,

$$\mathbf{A}\mathbf{x} \in \mathbb{R}^m, \quad (\mathbf{A}\mathbf{x})_i = \sum_{j=1}^{n} a_{ij} x_j.$$

Worked example:

$$\mathbf{A} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}, \quad \mathbf{x} = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}.$$

Then

$$\mathbf{A}\mathbf{x} = \begin{bmatrix} 1 \cdot 2 + 2 \cdot 1 + 3 \cdot 3 \\ 4 \cdot 2 + 5 \cdot 1 + 6 \cdot 3 \end{bmatrix} = \begin{bmatrix} 13 \\ 31 \end{bmatrix}.$$

For $\mathbf{x} \in \mathbb{R}^n$ and $\mathbf{A} \in \mathbb{R}^{n \times m}$,

$$\mathbf{x}^\top \mathbf{A} \in \mathbb{R}^{1 \times m}, \quad (\mathbf{x}^\top \mathbf{A})_j = \sum_{i=1}^{n} x_i a_{ij}.$$

Worked example:

$$\mathbf{x} = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}, \quad \mathbf{A} = \begin{bmatrix} 1 & 4 & 0 \\ 2 & 5 & 1 \\ 3 & 6 & 2 \end{bmatrix}.$$

Then

$$\mathbf{x}^\top \mathbf{A} = \begin{bmatrix} 13 & 31 & 7 \end{bmatrix}.$$

A *norm* measures the *magnitude* (or *length*) of a vector or matrix. It generalizes the concept of distance from the origin.

Definition 2.12 (Norm)**.** Formally, a norm $\|\cdot\|$ satisfies:

1. *Non-negativity:* $\|\mathbf{x}\| \geq 0$ and $\|\mathbf{x}\| = 0$ iff $\mathbf{x} = 0$.
2. *Homogeneity:* $\|\alpha\mathbf{x}\| = |\alpha|\|\mathbf{x}\|$.
3. *Triangle inequality:* $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$. We will formally define this below.

Norms are used to measure *size*, *distance*, and *error*, and to impose *regularization* in optimization problems (another thing we will define below).

2.2.1 Vector Norms

Definition 2.13 (ℓ_0 "Norm")**.** The ℓ_0 "norm" counts the number of nonzero entries in a vector:

$$\|\mathbf{x}\|_0 = \#\{i : x_i \neq 0\}.$$

It measures *sparsity* (how many components are active). It is *not* a true norm, since it violates the triangle inequality.

Worked example:
Let

$$\mathbf{x} = \begin{bmatrix} 3 \\ -4 \\ 0 \end{bmatrix}.$$

Then

$$\|\mathbf{x}\|_0 = 2.$$

Definition 2.14 (ℓ_1 Norm)**.** The ℓ_1 *norm* (also called the *Manhattan* or *taxicab* norm) is the sum of absolute values:

$$\|\mathbf{x}\|_1 = \sum_i |x_i|.$$

It measures *total magnitude*, encouraging *sparsity* when used in optimization.

Worked example:
Let

$$\mathbf{x} = \begin{bmatrix} 3 \\ -4 \\ 0 \end{bmatrix}.$$

Then

$$\|\mathbf{x}\|_1 = |3| + |-4| + |0| = 7.$$

I did not really understand norms until I came up with silly examples for them. For the ℓ_1 *norm*, imagine walking on a trail. You take 10 steps forward and 2 steps back — even though your net position is 8 steps ahead, the total *distance walked* is $|10| + |-2| = 12$. That total distance is your ℓ_1 norm.

Definition 2.15 (ℓ_2 Norm)**.** The ℓ_2 *norm* (Euclidean norm) measures the *straight-line distance* from the origin:

$$\|\mathbf{x}\|_2 = \sqrt{\sum_i x_i^2}.$$

It corresponds to the geometric length of $\mathbf{x}$.

Worked example:
Let

$$\mathbf{x} = \begin{bmatrix} 3 \\ -4 \\ 0 \end{bmatrix}.$$

Then

$$\|\mathbf{x}\|_2 = \sqrt{3^2 + (-4)^2 + 0^2} = \sqrt{25} = 5.$$

In this case, it measures how far you are from the origin had you walked in a straight line. We can use the same trail analogy above. Suppose you take ten steps forward, and 2 back: we've traveled a total of 12 steps, but we are 8 steps from the starting point: $\|\mathbf{x}\|_2 = \sqrt{(10 + (-2))^2} = \sqrt{8^2} = 8$.

If **?@fig-pythag** look familiar to you, this is simply the Pythagorean Theorem in action! This is also why people jaywalk/cross the street at places other than cross-walks, because they will get to their destination quicker if they take a straight line instead of walking to the corner and crossing.

We also have the infinity norm:

Definition 2.16 (ℓ_∞ Norm)**.** The ℓ_∞ *norm* (supremum or max norm) takes the largest absolute entry:

$$\|\mathbf{x}\|_\infty = \max_i |x_i|.$$

It measures the *maximum deviation* among components.

Worked example:
Let

$$\mathbf{x} = \begin{bmatrix} 3 \\ -4 \\ 0 \end{bmatrix}.$$

Then

$$\|\mathbf{x}\|_\infty = \max(|3|, |-4|, |0|) = 4.$$

Using the same steps $\mathbf{x} = [10, -2]$, the ℓ_∞ norm measures your *largest single stride*.

$$\|\mathbf{x}\|_\infty = \max(|10|, |-2|) = 10.$$

It answers the question: *"What was my biggest single effort along the path?"*

2.2.2 Matrix Norms

As it turns out, matrices can have norms too! The first is the Frobenius norm

Definition 2.17 (Frobenius Norm)**.** The *Frobenius norm* measures the magnitude of a matrix by aggregating the squares of all its entries:

$$\|\mathbf{Y}\|_F = \sqrt{\sum_{i,j} y_{ij}^2}.$$

It generalizes the ℓ_2 norm to matrices and corresponds to the Euclidean length of the vector formed by stacking all entries.

If each column of $\mathbf{Y}$ represents a separate person on a trail, each row entry is a step along that trail. The Frobenius norm sums up the squared steps of all people and takes the square root. It measures the *combined magnitude of movement* across all people.

Worked example:
Let

$$\mathbf{Y} = \begin{bmatrix} 1 & 2 & 0 \\ 2 & 1 & 3 \end{bmatrix}.$$

Then

$$\|\mathbf{Y}\|_F = \sqrt{1^2 + 2^2 + 0^2 + 2^2 + 1^2 + 3^2} = \sqrt{19}.$$

Definition 2.18 (Eigenvalues)**.** Let $A \in \mathbb{R}^{n \times n}$ be a square matrix. A *scalar* $\lambda \in \mathbb{R}$ (or $\mathbb{C}$) is called an *eigenvalue* of A if there exists a nonzero vector $\mathbf{v} \in \mathbb{R}^n$ such that

$$A\mathbf{v} = \lambda\mathbf{v}.$$

The corresponding *eigenvector* $\mathbf{v}$ is the vector that is only *scaled, not rotated*, by the transformation A.

Equivalently, λ is an eigenvalue if the following *characteristic equation* holds:

$$\det(A - \lambda I) = 0,$$

where I is the $n \times n$ identity matrix.

Think of each column of A as a separate person on a trail, and each entry as a step they take along that trail. The matrix A mixes the trail walkers, so normally any direction of steps gets both *stretched and rotated*. An *eigenvector* is a special "trail direction" that only gets *stretched*, not turned, when applying A. The *eigenvalue* tells you *how much* that direction is stretched. Large eigenvalues correspond to the "dominant trail" that grows the fastest. Small eigenvalues correspond to directions that barely change. This is why in many systems, the largest eigenvalue often *drives the main behavior*, similar to how the biggest steps dominate the Frobenius norm in the trail analogy.

Notice the person who does not move at all — their steps are all zero. Because they have *no variance*, they are completely uninformative about the matrix. In linear algebra terms, their direction corresponds to a *zero eigenvalue*. This is why zero eigenvalues arise: they indicate directions that the matrix *completely flattens*. Practically, we do not care about these people because they contribute *nothing* to the overall movement or dominant directions of the system.

Definition 2.19 (Matrix Operator (Spectral) Norm)**.** The *matrix operator norm*, also called the *spectral norm* or *induced 2-norm*, measures how much a matrix can *stretch* a vector when multiplied by it.

Formally, for $\mathbf{A} \in \mathbb{R}^{m\times n}$:

$$\|\mathbf{A}\|_2 = \max_{\mathbf{x}\neq 0} \frac{\|\mathbf{A}\mathbf{x}\|_2}{\|\mathbf{x}\|_2}.$$

Equivalently, it equals the *largest singular value* of $\mathbf{A}$:

$$\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A}) = \sqrt{\lambda_{\max}(\mathbf{A}^\top \mathbf{A})}.$$

Intuition:
- $\|\mathbf{A}\|_2$ captures the *maximum amplification factor* of $\mathbf{A}$.
- If you think of $\mathbf{A}$ as a transformation, it tells how much $\mathbf{A}$ can stretch any unit vector.
- It is widely used in numerical analysis, stability, and conditioning of linear systems.

Worked example (2×2 case):
Let

$$\mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}.$$

Compute $\mathbf{A}^\top \mathbf{A}$:

$$\mathbf{A}^\top \mathbf{A} = \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}^\top \begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix} = \begin{bmatrix} 5 & 8 \\ 8 & 13 \end{bmatrix}.$$

The eigenvalues of $\mathbf{A}^\top \mathbf{A}$ are solutions to

$$\det(\mathbf{A}^\top \mathbf{A} - \lambda I) = 0 \quad \Rightarrow \quad \lambda^2 - 18\lambda + 1 = 0.$$

Hence

$$\lambda_{\max} = 9 + 4\sqrt{5}, \quad \|\mathbf{A}\|_2 = \sqrt{\lambda_{\max}} = \sqrt{9 + 4\sqrt{5}} \approx 4.11.$$

So $\mathbf{A}$ stretches some unit vector by about a factor of 4.1.

In other words, imagine all possible combinations of walkers taking steps according to any unit vector $\mathbf{x}$ The operator norm tells you the combination that produces the largest overall movement. This is the dominant combined trail, the direction along which the walkers' weighted steps are amplified the most. This connects to previous concepts. Like the Frobenius norm, which sums the total effort across all walkers, the operator norm focuses on the single combination that experiences the maximum stretch. Zero eigenvalue directions correspond to walkers who do not move at all in that combination and therefore contribute nothing to the maximum stretch. The operator norm equals the largest singular value, which identifies the most dominant movement pattern among all walkers.

2.3 Calculus

Calculus is the study of change. It allows scientists to make precise statements about complex systems. In standard algebra we learn that the slope of a line is equal to the rise over run. **?@fig-linear-function** illustrates this idea.

This is the function $2x$, where we intuitively see that each movement along the x axis doubles whatever the input is for x. Suppose this is a price function where you are paying two dollars per pound of something. The rate of change for your pay is constant in the sense that you pay the same amount of money for every single pound that you buy. However, sometimes reality is a little more complex than nice linear functions. Consider the idea of kicking a football or soccerball.

Here, **?@fig-football-trajectory** shows us that the height of the ball changes at each point. More precisely, the speed at which the ball attains a given height changes at each point, as a function of time. At times point 0, the ball is with you, but by 2 seconds the ball will have attained a different height and be traveling a certain velocity at that height. Calculus allows us to study this phenomenon as well by using a derivative.

Definition 2.20 (Derivative)**.** For a scalar function $f : \mathbb{R} \to \mathbb{R}$, the *derivative* measures the instantaneous rate of change:

$$\frac{\mathrm{d}f(x)}{\mathrm{d}x} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}.$$

Intuition: How $f(x)$ changes for an infinitesimal change in x.

Derivatives have rules attached to them. For example, one is the power rule:

Definition 2.21 (Power Rule)**.** If $f(x) = x^n$ for $n \in \mathbb{R}$:

$$\frac{\mathrm{d}f(x)}{\mathrm{d}x} = nx^{n-1}.$$

Worked example: $\frac{\mathrm{d}}{\mathrm{d}x} x^3 = 3x^2$.

?@fig-football-cvxpy shows us the solution, but we will work it out by hand. Consider the football trajectory: $s(t) = -4t^2 + 16t + 3$. To find the time of maximum height, compute the derivative using the power rule: $\frac{\mathrm{d}s(t)}{\mathrm{d}t} = \frac{\mathrm{d}}{\mathrm{d}t}[-4t^2 + 16t + 3] = -8t + 16$. We set the derivative equal to zero to find the critical point: $-8t + 16 = 0 \Rightarrow t^* = \frac{16}{8} = 2$ seconds. Compute the maximum height by plugging t^* into the original function: $s(t^*) = -4(2)^2 + 16(2) + 3 = -16 + 32 + 3 = 19$ meters. Thus, the ball reaches its *highest point at* $t = 2$ *s*, with a height of 19 meters.

Definition 2.22 (Partial Derivative)**.** For a multivariate function $f : \mathbb{R}^n \to \mathbb{R}$, the *partial derivative* with respect to x_i is

$$\frac{\partial f(\mathbf{x})}{\partial x_i} = \lim_{h \to 0} \frac{f(x_1, \ldots, x_i + h, \ldots, x_n) - f(x_1, \ldots, x_i, \ldots, x_n)}{h}.$$

Intuition: Measures how f changes with x_i, holding other variables fixed.

Another important rule is the chain rule.

Definition 2.23 (Chain Rule)**.** For $y = f(g(x))$:

$$\frac{\mathrm{d}y}{\mathrm{d}x} = \frac{\mathrm{d}f}{\mathrm{d}g} \cdot \frac{\mathrm{d}g}{\mathrm{d}x}.$$

Consider a firm's profit given by revenue minus cost:

$$\Pi(x) = r(x) - c(x) = 75x - \Big[0.25(x+6)^2 + 191\Big].$$

The cost term can be viewed as a composition of functions. Let the outer function be

$$f(u) = 0.25u^2 + 191$$

and the inner function be

$$g(x) = x + 6,$$

so that

$$c(x) = f(g(x)).$$

Applying the chain rule, the derivative of cost with respect to x is

$$\frac{\mathrm{d}c}{\mathrm{d}x} = \frac{\mathrm{d}f}{\mathrm{d}u} \cdot \frac{\mathrm{d}g}{\mathrm{d}x} = 0.5(x+6) \cdot 1 = 0.5(x+6).$$

The derivative of profit with respect to x is then

$$\frac{\mathrm{d}\Pi}{\mathrm{d}x} = \frac{\mathrm{d}r}{\mathrm{d}x} - \frac{\mathrm{d}c}{\mathrm{d}x} = 75 - 0.5(x+6).$$

Setting this derivative equal to zero to find the maximum gives

$$75 - 0.5(x+6) = 0 \quad \Rightarrow \quad x^* = 144.$$

The maximum profit is

$$\Pi(144) = 75(144) - 0.25(150^2) - 191 = 10800 - 5625 - 191 = 4984.$$

The inner function is $g(x) = x + 6$, the outer function is $f(u) = 0.25u^2 + 191$, and the derivative of the cost term using the chain rule is $\frac{\mathrm{d}c}{\mathrm{d}x} = 0.5(x+6)$. The optimal quantity is

$x^* = 144$ and the maximum profit is $\Pi_{\max} = 4984$. To visualize this another way, consider the profit function as depending on revenue and costs, where costs depends on an inner function and outer function.

Realistically though, functions depend on many variables that we need to account for. To do this, we use the partial derivative.

Definition 2.24 (Partial Derivative)**.** For a multivariate function $f : \mathbb{R}^n \to \mathbb{R}$, the *partial derivative* with respect to x_i is

$$\frac{\partial f(\mathbf{x})}{\partial x_i} = \lim_{h \to 0} \frac{f(x_1, \ldots, x_i + h, \ldots, x_n) - f(x_1, \ldots, x_i, \ldots, x_n)}{h}.$$

Intuition: Measures how f changes with x_i, holding other variables fixed.

Consider a small company that sells two products, Product X and Product Y. The company can spend a certain amount of resources on marketing each product, denoted by x and y respectively.

The expected profit (in thousands of dollars) from marketing is modeled as

$$f(x, y) = 3x + 4y - x^2 - y^2$$

where $3x+4y$ represents the linear returns from marketing, and $-x^2-y^2$ represents diminishing returns.

To maximize profit, we can use partial derivatives with respect to each marketing variable:

$$\frac{\mathrm{d}f}{\mathrm{d}x} = 3 - 2x, \quad \frac{\mathrm{d}f}{\mathrm{d}y} = 4 - 2y$$

Setting the partial derivatives equal to zero gives the optimal allocation:

$$3 - 2x = 0 \quad \Rightarrow \quad x^* = 1.5$$

$$4 - 2y = 0 \quad \Rightarrow \quad y^* = 2$$

The maximum profit is

$$f(1.5, 2) = 3 \cdot 1.5 + 4 \cdot 2 - 1.5^2 - 2^2 = 6.25$$

This can also be solved in `cvxpy`

Definition 2.25 (Gradient)**.** The *gradient* of $f : \mathbb{R}^n \to \mathbb{R}$ is

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(\mathbf{x}) \\ \vdots \\ \frac{\partial f}{\partial x_n}(\mathbf{x}) \end{bmatrix}.$$

Consider the profit function of a company that markets two products, X and Y:

$$f(x, y) = 3x + 4y - x^2 - y^2$$

where x and y denote the marketing allocations for products X and Y. The gradient of $f(x, y)$ collects the partial derivatives with respect to x and y:

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} = \begin{bmatrix} 3 - 2x \\ 4 - 2y \end{bmatrix}.$$

At any point (x, y), the gradient points in the direction of steepest increase of profit.

For the optimal allocation found earlier:

$$x^* = 1.5, \quad y^* = 2,$$

the gradient evaluates to

$$\nabla f(x^*, y^*) = \begin{bmatrix} 3 - 2(1.5) \\ 4 - 2(2) \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix},$$

confirming that (x^*, y^*) is the critical point.

Definition 2.26 (Gramian Matrix)**.** Given vectors $x_1, \ldots, x_n \in \mathbb{R}^m$, the Gramian matrix is the $n \times n$ matrix

$$\mathbf{G} = \begin{bmatrix} \mathbf{x}_1^\top \mathbf{x}_1 & \mathbf{x}_1^\top \mathbf{x}_2 & \ldots & \mathbf{x}_1^\top \mathbf{x}_n \\ \mathbf{x}_2^\top \mathbf{x}_1 & \mathbf{x}_2^\top \mathbf{x}_2 & \ldots & \mathbf{x}_2^\top \mathbf{x}_n \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{x}_n^\top \mathbf{x}_1 & \mathbf{x}_n^\top \mathbf{x}_2 & \ldots & \mathbf{x}_n^\top \mathbf{x}_n \end{bmatrix}.$$

Consider two vectors in $\mathbb{R}^3$:

$$\mathbf{x}_1 = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}, \quad \mathbf{x}_2 = 2\mathbf{x}_1 = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}.$$

Form the matrix

$$\mathbf{X} = [\mathbf{x}_1 \ \mathbf{x}_2] = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \end{bmatrix}.$$

Compute the Gramian:

$$\mathbf{G} = \mathbf{X}^\top \mathbf{X} = \begin{bmatrix} \mathbf{x}_1^\top \mathbf{x}_1 & \mathbf{x}_1^\top \mathbf{x}_2 \\ \mathbf{x}_2^\top \mathbf{x}_1 & \mathbf{x}_2^\top \mathbf{x}_2 \end{bmatrix} = \begin{bmatrix} 14 & 28 \\ 28 & 56 \end{bmatrix}.$$

The correlation coefficient between the vectors is

$$\text{corr}(\mathbf{x}_1, \mathbf{x}_2) = \frac{\mathbf{x}_1^\top \mathbf{x}_2}{\|\mathbf{x}_1\|_2 \|\mathbf{x}_2\|_2} = \frac{28}{\sqrt{14}\sqrt{56}} = 1,$$

showing that $\mathbf{x}_1$ and $\mathbf{x}_2$ are perfectly aligned. To visualize what we mean by alignment, we can just plot these vectors:

We will oftentimes return to the Gramian in order to talk about the relationships between units (cities, states, or others for synthetic control methods)

2.4 Optimization

Much of this chapter has been building up to optimization. With the exception of some Bayesian SCMs, basic knowledge of optimization is crucial to being fluent in SCM research, instead of merely a user who uses preexisting code. Let's start with the idea of an objective function.

Definition 2.27 (Objective Function)**.** An *objective function* is a scalar-valued function we aim to minimize or maximize:

$$f : \mathbb{R}^n \to \mathbb{R}.$$

Optimization problem:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{or} \quad \max_{\mathbf{x}} f(\mathbf{x}).$$

An objective function may look like

$$f(\mathbf{w}) = \|\mathbf{y}_1 - \mathbf{Y}_0 \mathbf{w}\|_2^2$$

where the vector $\mathbf{y}_1$ is our target variable of interest, $\mathbf{Y}_0$ is some matrix, and $\mathbf{w}$ is what we call a decision variable, or, the vector of values that we seek to optimize the function with. In this case, we are using the Frobenius matrix to minimize the squared error between treated and the dot product of the matrix and some transformation of its values. The first step of minimizing an objective function is getting the first order conditions:

Definition 2.28 (First-Order Condition (FOC))**.** The *first-order condition* is necessary for a local extremum of a differentiable function $f(\mathbf{x})$:

$$\nabla f(\mathbf{x}^*) = \mathbf{0}.$$

This must hold at any local minimum or maximum.

This is essentially taking the first derivative. Consider the target vector $\mathbf{y}_1 = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$ and the matrix/vector $\mathbf{Y}_0 = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}$, with a scalar decision variable w. Our objective function is the squared 2-norm of the difference:

$$f(w) = \|\mathbf{y}_1 - \mathbf{Y}_0 w\|_2^2.$$

By definition, the squared 2-norm of a vector is the sum of the squares of its entries. Therefore, if we subtract $\mathbf{Y}_0 w$ from $\mathbf{y}_1$, we get the vector

$$\mathbf{y}_1 - \mathbf{Y}_0 w = \begin{bmatrix} 2 - w \\ 4 - 2w \\ 6 - 3w \end{bmatrix},$$

and taking the squared 2-norm of this vector gives

$$f(w) = (2 - w)^2 + (4 - 2w)^2 + (6 - 3w)^2.$$

Expanding each square, we find

$$f(w) = (4 - 4w + w^2) + (16 - 16w + 4w^2) + (36 - 36w + 9w^2),$$

and combining like terms results in

$$f(w) = 14w^2 - 56w + 56.$$

From here we can complete the square, use the power rule, or use the quadratic formula; I prefer the power rule. Differentiating with respect to w yields the first-order condition

$$f'(w) = 28w - 56 = 0,$$

To solve this, we add 56 to the right hand side and divide by 28.

$$w^* = 2.$$

Since the second derivative is positive, $f''(w) = 28 > 0$, we confirm that this is indeed a minimum. Now, in fairness, we could have just done this in our heads. But the reason I worked this out here is because we will be working a lot with objective functions. At least for me, I was very intimidated by them when I first saw them (and I still am all the time). So, I work through them to show you that the math behind this is not as arcane as it at first seems.

Frequently, we have optimization problems under constraints in real life. Often, there are limits to how much we can spend or how far we can go. These constraints change the feasibility set of our solutions"

Definition 2.29 (Feasible Region)**.** Let $f : \mathbb{R}^n \to \mathbb{R}$ be an objective function, and let the optimization problem be

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{subject to} \quad \begin{cases} g_i(x) \leq 0, & i = 1, \ldots, m, \\ h_j(x) = 0, & j = 1, \ldots, p, \end{cases}$$

where $g_i(x)$ are inequality constraints and $h_j(x)$ are equality constraints.

The *feasible region* (also called the feasibility set) is defined as

$$\mathcal{W} = \{x \in \mathbb{R}^n \mid g_i(x) \leq 0,\ i = 1, \ldots, m;\ h_j(x) = 0,\ j = 1, \ldots, p\}.$$

In words, $\mathcal{W}$ is the set of all points that satisfy *all the constraints* of the optimization problem. The solution of the optimization problem must lie in $\mathcal{W}$. For example, with the problem

$$\min_{x_1, x_2} x_1^2 + x_2^2 \quad \text{s.t.} \quad x_1 + x_2 = 1,$$

the feasible region is

$$\mathcal{W} = \{(x_1, x_2) \in \mathbb{R}^2 \mid x_1 + x_2 = 1\}.$$

2.5 Convexity

Convexity is an important idea that we will return to in later chapters. Here we define some basic terms, which we will build on later.

Definition 2.30 (Convex Function)**.** A function $f : \mathbb{R}^n \to \mathbb{R}$ is convex if for all $x, y \in \mathbb{R}^n$, $\lambda \in [0, 1]$:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda) f(y).$$

Differentiable functions satisfy $f(y) \geq f(x) + \nabla f(x)^\top (y - x)$.

The intuition is that a convex function is bowl-shaped and always lies below the line connecting any two points on its graph. An example is $f(x) = x^2$ on $\mathbb{R}$, which has a minimum at zero and curves upward on both sides. Another example is $f(x) = e^x$, which grows exponentially but also satisfies the convexity property.

Definition 2.31 (Convex Set)**.** A set $C \subset \mathbb{R}^n$ is convex if for any $x, y \in C$ and $\lambda \in [0, 1]$, the point $\lambda x + (1 - \lambda)y$ is also in C.

Intuitively, any line segment connecting points in the set stays entirely within the set. For example, the interval $[0, 1]$ in $\mathbb{R}$ is convex because any point between two numbers in the interval is also in the interval. Similarly, the unit disk $\{x \in \mathbb{R}^2 : \|x\| \leq 1\}$ is convex because any line segment between two points in the disk remains inside the disk.

Definition 2.32 (Closed Set)**.** A set $C \subset \mathbb{R}^n$ is closed if it contains all its limit points. That is, if a sequence (x_k) in C converges to x, then x must also be in C.

The intuition is that a closed set has no "missing boundary points." For example, the interval $[0, 1]$ is closed because its endpoints are included. Another example is $\{x \in \mathbb{R} : x \geq 0\}$, which contains all points that are limits of sequences within the set.

Definition 2.33 (Bounded Set)**.** A set $C \subset \mathbb{R}^n$ is bounded if all its points lie within some finite distance from the origin. Formally, there exists $M > 0$ such that $\|x\| \leq M$ for all $x \in C$.

Intuitively, a bounded set fits inside a ball of finite radius. The interval $[0, 1]$ is bounded, as is the unit circle $\{x \in \mathbb{R}^2 : \|x\| = 1\}$.

Definition 2.34 (Compact Set)**.** A set $C \subset \mathbb{R}^n$ is compact if it is both closed and bounded.

This means the set is finite in extent and contains all its boundary points. A compact set ensures that continuous functions defined on it achieve maximum and minimum values. Examples include the interval $[0, 1]$ and the closed unit disk $\{x \in \mathbb{R}^2 : \|x\| \leq 1\}$.

Definition 2.35 (Convex Hull)**.** The convex hull of points $\{x_1, \ldots, x_J\}$ is

$$\text{conv}\{x_1, \ldots, x_J\} = \left\{ \sum_{j=1}^{J} w_j x_j \;\middle|\; w_j \geq 0, \sum_{j=1}^{J} w_j = 1 \right\}.$$

Intuitively, the convex hull is the smallest convex set containing all the points, formed by all weighted averages of the points. For instance, the convex hull of points $(0, 0), (1, 0), (0, 1)$ is the triangle defined by these vertices. If the points lie on a line, such as $x_1 = 0, x_2 = 2, x_3 = 5$, the convex hull is the interval $[0, 5]$.

2.6 Inequalities

While this book is not a real analysis book, I will occasionally present some theoretical results if they help explain why certain models or ideas behave the way they do. Typically, these are simple algebraic manipulations. In an appendix, I will go through the asymptotic bias bound

of the standard SCM much later on. To do this, we need inequalities that allow us to give formal bounds.

The first of these is the triangle inequality.

Definition 2.36 (Triangle Inequality for Scalars)**.** For any real numbers a and b, the triangle inequality states:

$$|a + b| \leq |a| + |b|.$$

This inequality allows us to bound the absolute value of a sum of scalars by the sum of their absolute values. To build intuition, imagine walking along a trail. Suppose you first take 3 steps backward, then 5 steps forward. Your net position relative to the start is 2 steps forward, even though you actually moved a total of 8 steps. The triangle inequality formalizes this idea: the net change cannot exceed the total distance covered.

In symbols, if $a = -3$ and $b = 5$, then

$$|a + b| = |-3 + 5| = |2| = 2, \quad |a| + |b| = 3 + 5 = 8,$$

so indeed $|a + b| \leq |a| + |b|$.

We can also apply the triangle inequality to vectors. Intuitively, the idea is the same: the length of the sum of two vectors cannot exceed the sum of their lengths.

Definition 2.37 (Triangle Inequality for Vectors)**.** For any vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, the triangle inequality states:

$$\|\mathbf{x} + \mathbf{y}\|_2 \leq \|\mathbf{x}\|_2 + \|\mathbf{y}\|_2.$$

To build intuition, imagine walking along a trail in two dimensions. If you first walk along vector $\mathbf{x}$ and then along vector $\mathbf{y}$, your net displacement from the start is represented by $\mathbf{x} + \mathbf{y}$. The length of this net displacement (the straight-line distance from start to finish) is always less than or equal to the total distance you actually walked (the sum of the lengths of $\mathbf{x}$ and $\mathbf{y}$). For example, if $\mathbf{x} = (3, 0)$ and $\mathbf{y} = (0, 4)$, then

$$\|\mathbf{x} + \mathbf{y}\|_2 = \|(3, 4)\|_2 = 5, \quad \|\mathbf{x}\|_2 + \|\mathbf{y}\|_2 = 3 + 4 = 7,$$

so indeed $\|\mathbf{x} + \mathbf{y}\|_2 \leq \|\mathbf{x}\|_2 + \|\mathbf{y}\|_2$. This inequality really helped me understand the ℓ_1 norm.

Definition 2.38 (Jensen's Inequality)**.** Let X be a random variable and f a convex function. Then:

$$f(\mathbb{E}[X]) \leq \mathbb{E}[f(X)].$$

In our context, the convex function is $f(x) = |x|$, so

$$|\mathbb{E}[X]| \leq \mathbb{E}[|X|].$$

To build intuition, consider a trail example. Suppose you take two steps: $x = -2$ steps backward and $y = 4$ steps forward, with weights $\theta = 0.3$ and $1 - \theta = 0.7$. Then

$$|\theta x + (1-\theta)y| = |0.3(-2) + 0.7(4)| = 2.2,$$

while

$$\theta|x| + (1-\theta)|y| = 0.3 \cdot 2 + 0.7 \cdot 4 = 3.4.$$

Indeed, Jensen's inequality holds: $2.2 \leq 3.4$. The average magnitude is at least as big as the magnitude of the weighted average step.

Finally, we arrive at the great-granddaddy of inequalities: the Cauchy–Schwarz inequality. This inequality is fundamental in linear algebra and statistics, and it underpins many other results, including the triangle inequality for vectors.

Definition 2.39 (Cauchy–Schwarz Inequality)**.** For any vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, the Cauchy–Schwarz inequality states:

$$|\mathbf{x}^\top \mathbf{y}| \leq \|\mathbf{x}\|_2 \, \|\mathbf{y}\|_2.$$

Intuitively, Cauchy–Schwarz says that the absolute value of the dot product of two vectors cannot exceed the product of their lengths. Geometrically, it captures the idea that the "projection" of one vector onto another can never be longer than the product of their magnitudes. For example, consider vectors $\mathbf{x} = (1, 2, 2)$ and $\mathbf{y} = (2, 0, 1)$. Then

$$\mathbf{x}^\top \mathbf{y} = 1 \cdot 2 + 2 \cdot 0 + 2 \cdot 1 = 4, \quad \|\mathbf{x}\|_2 = \sqrt{1^2 + 2^2 + 2^2} = 3, \quad \|\mathbf{y}\|_2 = \sqrt{2^2 + 0^2 + 1^2} = \sqrt{5}.$$

The product of the norms is $3 \cdot \sqrt{5} \approx 6.71$, and indeed

$$|\mathbf{x}^\top \mathbf{y}| = 4 \leq 6.71 = \|\mathbf{x}\|_2 \, \|\mathbf{y}\|_2.$$

This inequality is extremely useful because it allows us to bound dot products in terms of vector lengths.

We can also see how Cauchy–Schwarz leads naturally to the triangle inequality for vectors, using our trail analogy. Suppose you first walk along vector $\mathbf{x}$, then along vector $\mathbf{y}$. Your net displacement is $\mathbf{x} + \mathbf{y}$, and the triangle inequality says

$$\|\mathbf{x} + \mathbf{y}\|_2 \leq \|\mathbf{x}\|_2 + \|\mathbf{y}\|_2.$$

Cauchy–Schwarz helps prove this because

$$\|\mathbf{x}+\mathbf{y}\|_2^2 = (\mathbf{x}+\mathbf{y})^\top(\mathbf{x}+\mathbf{y}) = \|\mathbf{x}\|_2^2+2\mathbf{x}^\top\mathbf{y}+\|\mathbf{y}\|_2^2 \le \|\mathbf{x}\|_2^2+2\|\mathbf{x}\|_2\|\mathbf{y}\|_2+\|\mathbf{y}\|_2^2 = (\|\mathbf{x}\|_2+\|\mathbf{y}\|_2)^2,$$

where we applied Cauchy–Schwarz to bound $|\mathbf{x}^\top\mathbf{y}| \le \|\mathbf{x}\|_2\,\|\mathbf{y}\|_2$.

For a concrete numeric example, consider a 2D trail: let $\mathbf{x} = (3, 0)$ represent walking 3 steps east, and $\mathbf{y} = (0, 4)$ represent walking 4 steps north. Then your net displacement is

$$\mathbf{x} + \mathbf{y} = (3, 4), \quad \|\mathbf{x} + \mathbf{y}\|_2 = \sqrt{3^2 + 4^2} = 5.$$

The sum of the lengths of your individual steps is

$$\|\mathbf{x}\|_2 + \|\mathbf{y}\|_2 = 3 + 4 = 7.$$

Cauchy–Schwarz guarantees that your net displacement cannot exceed the total distance you walked:

$$\|\mathbf{x} + \mathbf{y}\|_2 = 5 \le 7 = \|\mathbf{x}\|_2 + \|\mathbf{y}\|_2.$$

Thus, the triangle inequality for vectors is a direct consequence of Cauchy–Schwarz.

Minkowski's inequality generalizes the triangle inequality to sums of vectors (or sequences of vectors) and higher moments. It is particularly useful when we want to bound the "combined displacement" of multiple vectors at once.

Definition 2.40 (Minkowski Inequality)**.** For vectors $\mathbf{x}_1, \ldots, \mathbf{x}_n \in \mathbb{R}^F$ and $g \ge 1$:

$$\Big(\sum_{i=1}^{n} \|\mathbf{x}_i + \mathbf{y}_i\|_2^g\Big)^{1/g} \le \Big(\sum_{i=1}^{n} \|\mathbf{x}_i\|_2^g\Big)^{1/g} + \Big(\sum_{i=1}^{n} \|\mathbf{y}_i\|_2^g\Big)^{1/g}.$$

Intuitively, imagine a group of people walking on the same trail at once. Each person takes a step represented by $\mathbf{x}_i$ and another step represented by $\mathbf{y}_i$. Minkowski's inequality says that the combined "effort" of all people moving along both sets of steps, measured in the g-norm, cannot exceed the sum of the efforts of each set of steps separately.

For a numeric illustration, suppose three people take steps $\mathbf{x}_1 = (1, 0)$, $\mathbf{x}_2 = (0, 2)$, $\mathbf{x}_3 = (1, 1)$, and corresponding steps $\mathbf{y}_1 = (0, 1)$, $\mathbf{y}_2 = (1, 0)$, $\mathbf{y}_3 = (1, 0)$, with $g = 2$. Then the total "combined displacement" is

$$\Big(\sum_{i=1}^{3} \|\mathbf{x}_i + \mathbf{y}_i\|_2^2\Big)^{1/2} = \Big(\|(1,1)\|_2^2 + \|(1,2)\|_2^2 + \|(2,1)\|_2^2\Big)^{1/2} = \sqrt{2+5+5} = \sqrt{12} \approx 3.46.$$

The sum of the separate "efforts" is

$$\Big(\sum_{i=1}^{3}\|\mathbf{x}_i\|_2^2\Big)^{1/2}+\Big(\sum_{i=1}^{3}\|\mathbf{y}_i\|_2^2\Big)^{1/2}=\sqrt{1+4+2}+\sqrt{1+1+1}=\sqrt{7}+\sqrt{3}\approx 4.45.$$

Indeed, the combined displacement $3.46 \le 4.45$, illustrating Minkowski's inequality. This shows how the total movement of multiple people along a trail is controlled by the sum of their individual efforts.

Finally, we generalize Cauchy–Schwarz to Hölder's inequality. While Cauchy–Schwarz handles the case of $p = q = 2$, Hölder's inequality allows us to bound sums of products for any exponents $p, q \ge 1$ satisfying $1/p + 1/q = 1$. This is very useful when dealing with weighted sums or expectations.

Definition 2.41 (Hölder's Inequality)**.** For sequences of real numbers (a_i) and (b_i), and exponents $p, q \ge 1$ such that $1/p + 1/q = 1$, Hölder's inequality states:

$$\sum_i |a_i b_i| \le \left(\sum_i |a_i|^p\right)^{1/p}\left(\sum_i |b_i|^q\right)^{1/q}.$$

Intuitively, Hölder's inequality says that the "weighted overlap" between two sequences cannot exceed the product of their generalized lengths.

For a simple numeric example, consider two sequences $a = (1, 2)$ and $b = (3, 4)$ with $p = 3$ and $q = 3/2$ (so that $1/p + 1/q = 1$). Then

$$\sum_i |a_i b_i| = |1\cdot 3| + |2\cdot 4| = 3 + 8 = 11.$$

On the right-hand side, we compute the generalized p-norms:

$$\left(\sum_i |a_i|^p\right)^{1/p} = (1^3 + 2^3)^{1/3} = (1+8)^{1/3} = 9^{1/3} \approx 2.08,$$

$$\left(\sum_i |b_i|^q\right)^{1/q} = (3^{3/2} + 4^{3/2})^{2/3} \approx (5.196 + 8)^{2/3} = 13.196^{2/3} \approx 5.43.$$

Multiplying the norms gives $2.08 \cdot 5.43 \approx 11.3$, which indeed bounds the sum of products $11 \le 11.3$.

Rosenthal's inequality generalizes the idea of bounding the magnitude of a sum of independent random variables, similar to how the triangle, Cauchy–Schwarz, and Hölder inequalities help for deterministic sequences or vectors. In the random variable case, it controls the expected size of the sum, measured in terms of higher moments.

Definition 2.42 (Rosenthal Inequality (Scalar Version))**.** Let $X_1, \ldots, X_n$ be independent, mean-zero random variables with finite g-th moments ($g \geq 2$). Then there exists a constant $C(g)$ such that:

$$\mathbb{E}\Big[\Big|\sum_{i=1}^{n} X_i\Big|^g\Big] \leq C(g)\, \max\left(\sum_{i=1}^{n} \mathbb{E}[|X_i|^g], \Big(\sum_{i=1}^{n} \mathbb{E}[X_i^2]\Big)^{g/2}\right).$$

This inequality bounds the g-th moment of a sum of independent mean-zero random variables.

Intuitively, imagine walking along a trail where each step is random — sometimes forward, sometimes backward — and the steps are independent of each other. Your net displacement after n steps is random. Rosenthal's inequality tells us that the expected "size" of your total displacement (raised to the g-th power) can be bounded by either the sum of the g-th powers of your individual steps or by the $g/2$ power of the sum of the step variances, whichever is larger.

For a concrete numeric illustration, consider $n = 2$ independent random steps X_1 and X_2, where X_1 takes values 1 or -1 with equal probability, and X_2 takes values 2 or -2 with equal probability. Let $g = 4$. Then

$$\mathbb{E}[|X_1|^4] = 1, \quad \mathbb{E}[|X_2|^4] = 16, \quad \sum_i \mathbb{E}[|X_i|^4] = 17,$$

and

$$\mathbb{E}[X_1^2] = 1, \quad \mathbb{E}[X_2^2] = 4, \quad \left(\sum_i \mathbb{E}[X_i^2]\right)^{4/2} = (1+4)^2 = 25.$$

Hence, Rosenthal's inequality gives

$$\mathbb{E}[|X_1 + X_2|^4] \leq C(4) \cdot \max(17, 25) = C(4) \cdot 25,$$

showing that the fourth moment of the total displacement is controlled by either the sum of the fourth moments of the individual steps or by the squared sum of their variances. In other words, even though the steps are random and sometimes cancel each other out, Rosenthal gives a formal bound on how large the total displacement can be in expectation.

Definition 2.43 (Vector-Valued Rosenthal Inequality)**.** Let $X_1, \ldots, X_n \in \mathbb{R}^F$ be independent, mean-zero random vectors with finite g-th moments ($g \geq 2$). Then there exists a constant $C(g)$ such that:

$$\mathbb{E}\Big[\Big\|\sum_{i=1}^{n} X_i\Big\|_2^g\Big] \leq C(g)\, \max\left(\sum_{i=1}^{n} \mathbb{E}\|X_i\|_2^g, \Big(\sum_{i=1}^{n} \mathbb{E}\|X_i\|_2^2\Big)^{g/2}\right).$$

To see this intuitively, imagine walking along a trail where each step is a random vector in 2D space, e.g. $X_1 = (1, 0)$, $X_2 = (0, 2)$. The sum $X_1 + X_2 = (1, 2)$ represents your net displacement. Rosenthal's inequality bounds the expected size of the net displacement in the g-th moment norm:

$$\mathbb{E}\|X_1 + X_2\|_2^g \leq C(g) \max\left(\mathbb{E}\|X_1\|_2^g + \mathbb{E}\|X_2\|_2^g, (\mathbb{E}\|X_1\|_2^2 + \mathbb{E}\|X_2\|_2^2)^{g/2}\right).$$

Even if the steps sometimes cancel out in some directions, this inequality ensures the expected size cannot blow up arbitrarily.

2.7 Conclusion

These definitions and worked examples form the mathematical toolbox for synthetic control. Analysts can now reason rigorously about vectors, matrices, derivatives, convexity, and optimization, while connecting directly to SCM applications. I have simply defined these for use later on, so we will become more acquainted with these over time. A brief word of warning—and encouragement—is in order. While this book aims to make SCM accessible to a wide audience, mathematics will be, in a sense, non-negotiable.

One of my favorite econometricians once asked hypothetically in his lecture notes, "Do I really need the fancy mathematics to understand econometrics?" (Shi, n.d.). The answer, as he put it, is both yes and no: not all practitioners need measure theory to apply econometrics effectively. By extension, not all practitioners who use causal inference and SCM need measure theory or post-graduate understanding of optimization and real analysis. In fact, my first degrees were in political science where I had no exposure to linear algebra, calculus, set theory, or discrete mathematics. However, without some degree of mathematical precision, core ideas like weighted averages, convex hulls, interpolation and extrapolation biases, and other important concepts cannot be fully defined. In the same spirit, this text strives for balance. I will avoid unnecessary abstraction, but I will also insist on clarity and correctness where it matters most. Only by taking advantage of the language of probability and statistics can the full value of the SCM be made known.

Chapter 3

Averages

> "I'm too profound to go Back and forth, with no average dork"
> — *Solana Imani Rowe*

An average is one of the first summary statistics we learn—often in elementary or middle school, depending on where we grew up. Averages are everywhere in data analysis. We regularly compute metrics like average revenue, averages of income, average of prices, and for many other metrics. Yet, few people pause to consider what an average actually *is* as a concept or why we use them. In this chapter, we cover this in more detail, from simple arithmetic means to general weighted averages—as a foundation for understanding how computers construct synthetic controls in practice. There is an entire chapter for averaging because this is all a synthetic control is. People will say "fake unit" or "weighted mix" without being precise about what they mean, when at simplest we are just computing an average.

3.1 Averaging in Statistics

Averages arise naturally when we try to find a single value that best represents a collection of numbers. In the old days, Egyptians and Babylonians used averages to summarize construction and financial matters. In modern times, we use averages to summarize things like test grades, incomes, and other variables we care about. To formalize this, suppose we have a sequence of data points $(x_1, x_2, \ldots, x_n)$ of length n. We want to choose a single number μ that is as close as possible to all of them. Here, μ is a placeholder variable representing a potential candidate for the 'best' summary value; we will later find that the minimizing value, denoted $\bar{x}$, is the arithmetic mean. Because we seek a single number as close as possible to all values, this becomes our first example of an optimization problem—minimizing a function with respect to a variable. Recalling the definition of an *objective function*, let our observed values be x_i and the number that "is as close as possible to all the values" be the Greek letter μ (m-yoo). Because we are looking for the value that minimizes the distance, we can write $x_i - \mu$ as a shorthand for this deviation. To minimize the total deviation across all points, we sum over each element of the sequence i: $\sum_{i=1}^{n}(x_i - \mu)$. Finally, to penalize values of μ that are further away from the observed data, we square each term. This gives us the optimization problem:

Definition 3.1. Objective function

A *scalar-valued* function of a choice variable that we minimize (or maximize). We write the choice variable under the operator: $\min_\mu f(\mu)$ reads "search over μ for the value that makes f smallest."

$$\min_\mu \sum_{i=1}^{n} (x_i - \mu)^2.$$

This function is *convex* and differentiable. We can optimize this function by taking the derivative with respect to μ. We begin by applying the **chain rule** term by term.

Definition 3.2. The power and chain rules

The **power rule** differentiates a power: $\frac{\mathrm{d}}{\mathrm{d}u} u^k = k\, u^{k-1}$. The **chain rule** differentiates a function of a function: if g depends on u and u depends on μ, then $\frac{\mathrm{d}g}{\mathrm{d}\mu} = \frac{\mathrm{d}g}{\mathrm{d}u} \cdot \frac{\mathrm{d}u}{\mathrm{d}\mu}$ — outer derivative times inner.

For each term $(x_i - \mu)^2$, there are two parts to consider: the outer function and the inner function. The outer function is $f(u) = u^2$, where $u = x_i - \mu$. Its derivative (by the **power rule**) is

$$\frac{\mathrm{d}}{\mathrm{d}u} u^2 = 2u = 2(x_i - \mu)$$

The inner function is $u(\mu) = x_i - \mu$. Its derivative with respect to μ is

$$\frac{\mathrm{d}}{\mathrm{d}\mu}(x_i - \mu) = -1$$

Multiplying the outer and inner derivatives gives the derivative of a single term:

$$\frac{\mathrm{d}}{\mathrm{d}\mu}(x_i - \mu)^2 = 2(x_i - \mu) \cdot (-1) = -2(x_i - \mu)$$

Summing over all i gives the derivative of the entire objective function:

$$\frac{\mathrm{d}}{\mathrm{d}\mu} \sum_{i=1}^{n} (x_i - \mu)^2 = \sum_{i=1}^{n} [-2(x_i - \mu)] = -2 \sum_{i=1}^{n} (x_i - \mu)$$

Setting this derivative equal to zero gives the first-order condition for the minimum

$$-2 \sum_{i=1}^{n} (x_i - \mu) = 0.$$

We now solve for μ. First, just divide both sides by -2:

$$\sum_{i=1}^{n}(x_i - \mu) = 0.$$

Next, split the sum into two parts:

$$\sum_{i=1}^{n} x_i - \sum_{i=1}^{n} \mu = 0.$$

Since μ is constant with respect to the index i, the second sum simplifies to multiplication:

$$\sum_{i=1}^{n} x_i - n\mu = 0.$$

Now we add $n\mu$ to both sides:

$$\sum_{i=1}^{n} x_i = n\mu.$$

Finally, divide both sides by n to isolate μ:

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i.$$

Definition 3.3. Arithmetic average

For some $(x_1, x_2, \ldots, x_n) \in \mathbb{R}^n$, the arithmetic average is $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$, where n is the length of the sequence and $\frac{1}{n}$ is the weight.

Thus, the value of μ that minimizes the total squared deviation is the arithmetic mean of the data. The arithmetic mean is thus not an arbitrary formula: it is the unique value that minimizes the total squared distance between itself and the data.

But, recall the definition of an objective function: I mentioned how it is a **scalar valued** function, because it returns a scalar. That scalar is the value of the function where we set μ equal to the optimal value. Substituting this minimizing value back into the objective function, we get the total deviation around the mean:

$$f(\bar{x}) = \sum_{i=1}^{n}(x_i - \bar{x})^2.$$

Because this quantity grows with n, it is often more useful to consider the average squared deviation (also called the residuals):

$$\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2,$$

which we call the population variance. The variance measures how far the data tend to fall from their best-fitting constant value. We use squared deviations because they produce a smooth, differentiable, and convex objective that penalizes larger errors more heavily. Note that the absolute deviations, instead of the squared deviations, would return the median, not the mean. The cost of squaring the residuals is that variance is expressed in squared units; taking the square root restores the original units and gives the standard deviation. When variance is estimated from a sample rather than the entire population, the denominator is typically replaced by $n - 1$ to obtain an unbiased estimator.

3.2 Kinds of Arithmetic Averages

Okay, so we now have our definition. Now let's apply this to the other objects we already know about, scalars, vectors, and matrices.

3.2.1 Scalars

Definition 3.4. Scalar

A *scalar* is a single number — a lone magnitude with no direction or arrangement, an element of $\mathbb{R}$. The count of apples you own, one state's cigarette sales in a single year, or the weight $\frac{1}{n}$ are all scalars.

For the simplest case, we can simply apply the arithmetic mean to scalars. Suppose you have 12 apples and your friend has 18. Denote this sequence as $A = (12, 18) \in \mathbb{N}^2$. There are two elements in this sequence. Because the length of the sequence is 2, our weight is $\frac{1}{2}$. The way to compute the arithmetic average is

$$\bar{x} = \frac{1}{2}(12 + 18) = (0.5 \times 12) + (0.5 \times 18) = 6 + 9 = 15.$$

On average, you and your friend each have 15 apples. We can do this in `NumPy`

```
import numpy as np

A = np.array([12, 18])
mean_1 = np.mean(A)

weights = np.full(len(A), len(A)**-1)
mean_2 = np.average(A, weights=weights)

print(f"Mean of Apples: {mean_1}, {mean_2}")
```

```
Mean of Apples: 15.0, 15.0
```

See? Nothing special happened here, we just took the empirical mean of this sequence. Just to show that the above calculus derivation was not voodoo or an optical illusion, we can even just plot the objective function itself to see the scalar outputs at different values:

```
import numpy as np
import matplotlib.pyplot as plt

# Data points

A = np.array([12, 18])

# Objective function: sum of squared deviations

mu_values = np.linspace(10, 20, 400)
f_mu = np.sum((A[:, None] - mu_values)**2, axis=0)

# Plot

plt.figure(figsize=(6,4))
plt.plot(mu_values, f_mu, label=r'$f(\mu) = \sum (x_i - \mu)^2$')
plt.axvline(np.mean(A), color='red', linestyle='--', label=r'$\bar{x} = 15$')
plt.scatter(np.mean(A), np.sum((A - np.mean(A))**2), color='red')
plt.xlabel(r'$\mu$')
plt.ylabel(r'$f(\mu)$')
plt.title('Objective Function for Arithmetic Mean')
plt.legend()
plt.grid(True)
plt.show()
```

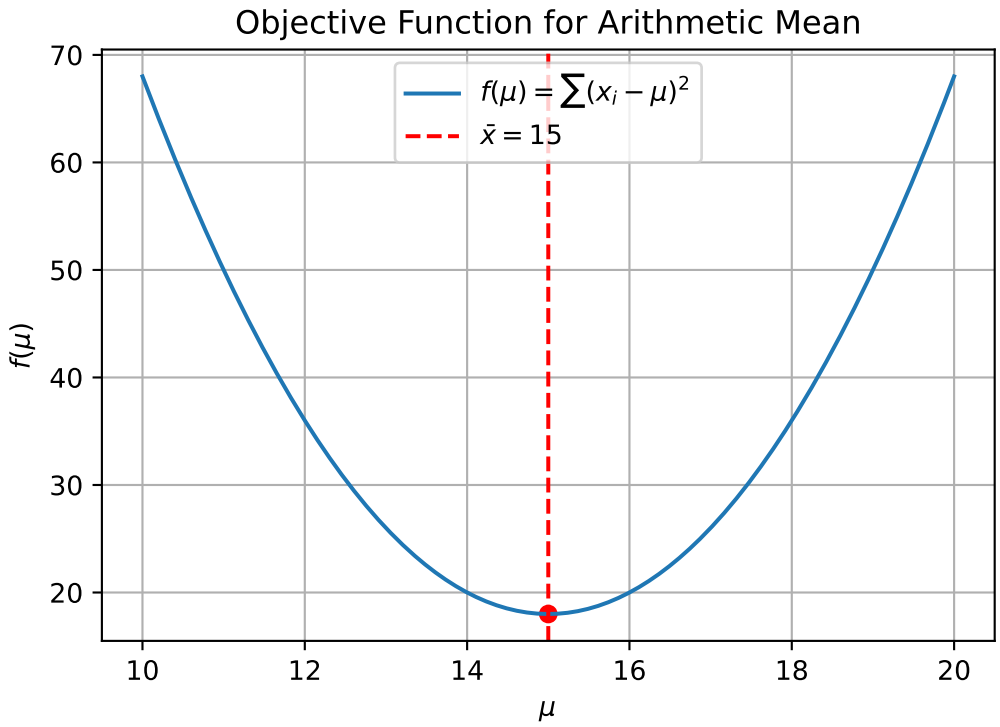


Figure 3.1: Objective Function: Arithmetic Mean

The next context in which we often encounter averages is when estimating treatment effects. Suppose we are interested in the Average Treatment effect on the Treated (ATT) for a single treated unit ($j = 1$) observed over two post-treatment periods ($|\mathcal{T}_2| = 2$). Let the observed (treated) outcome be denoted by $y_{1t}(1)$ and the predicted or counterfactual outcome by $y_{1t}(0)$. The treatment effect in each period is then $\tau_t = y_{1t}(1) - y_{1t}(0)$. Suppose the observed sequence is $y_1(1) = (533, 568)$ and the counterfactual sequence is $y_{1t}(0) = (400, 446)$. The resulting difference of these (the treatment effect sequence) is $\tau = y_1(1) - y_{1t}(0) = (133, 122)$. To compute the average treatment effect on the treated across these two post-treatment periods (formally defined later in the book), we apply the same arithmetic mean formula as before to this sequence of treatment effects. This looks like:

$$\bar{\tau}_{\text{ATT}} = \frac{1}{2}\sum_{t=1}^{2}\tau_t = \frac{1}{2}(133 + 122) = \tag{3.1}$$

$$(0.5 \times 133) + (0.5 \times 122) = 66.5 + 61 = 127.5. \tag{3.2}$$

Thus, on average, the treated unit's observed outcomes are about 127.5 units higher than what the counterfactual model predicted. Another thing we can do in Python:

```
import numpy as np

# Observed outcomes
```

```
y1_treated = np.array([533, 568])

# Counterfactual outcomes
y1_control = np.array([400, 446])

# Treatment effect sequence
tau = y1_treated - y1_control

# Equal weights
weights = np.full(len(tau), len(tau)**-1)

# Average Treatment Effect on the Treated (ATT)
ATT = np.average(tau, weights=weights)

print(f"Treatment effects by period: {tau}")
print(f"Average Treatment Effect on the Treated (ATT): {ATT}")
```

```
Treatment effects by period: [133 122]
Average Treatment Effect on the Treated (ATT): 127.5
```

Because each value in the sequence contributes equally to the final result, the arithmetic mean can be seen as a weighted sum where each weight for the i-th element is $\frac{1}{n}$. In general, for a sequence $(x_1, x_2, \ldots, x_n)$, the arithmetic mean can be written as

$$\bar{x} = \sum_{i=1}^{n} w_i x_i, \quad w_i = \frac{1}{n} \, \forall \, i$$

Problem 3.1. Mean of Small Numbers

Compute the arithmetic mean of the numbers: 4, 8, 12.

Problem 3.2. Mean of Single-Digit Values

Find the arithmetic mean of the sequence: 1, 3, 5, 7, 9.

Problem 3.3. Mean of Exam Scores

A student scored 70, 75, and 80 on three tests. Compute the arithmetic mean score.

Problem 3.4. Mean of Monthly Sales

A small shop made the following sales in dollars over 4 months: 200, 250, 300, 350. Find the average monthly sales.

Problem 3.5. Mean of Age Values

The ages of five friends are 20, 22, 24, 26, and 28. Compute the arithmetic mean age.

3.2.2 Vectors

Definition 3.5. Vector

A *vector* is an ordered list of scalars, written as a row $\begin{bmatrix} x_1 & \cdots & x_n \end{bmatrix}$ or a column, and living in $\mathbb{R}^n$, where n is its length. Order matters: the i-th entry has a fixed meaning. In this book a vector usually collects one unit's outcomes — a single state's cigarette sales across every year, say — so that each entry is a different time period.

The same idea generalizes easily to vectors. In higher dimensions, averaging is simply applying the same principle along rows or columns. We can represent the simple apple example as a row vector:

$$\mathbf{x} = \begin{bmatrix} 12 & 18 \end{bmatrix} \in \mathbb{R}^{1\times 2}.$$

This is the same example as before, just expressed in vector form. Here, each column corresponds to a person's value.

Definition 3.6. Dot product

For a row vector $\mathbf{a}$ and a column vector $\mathbf{b}$ of the same length, the *dot product* multiplies them entry by entry and sums the results, $\mathbf{ab} = \sum_i a_i b_i$. It turns "multiply each value by a weight and add them up" — an average — into a single product.

Using the *dot product* and the idea of the average as a weighted sum, we introduce a weight vector that assigns proportional and equal weight to each person:

$$\mathbf{w} = \tfrac{1}{2}\begin{bmatrix} 1 \\ 1 \end{bmatrix} = \tfrac{1}{2}\mathbf{1}_2 \in \mathbb{R}^{2\times 1}.$$

Why proportional and equal? Because that is simply the definition of the arithmetic average. The same way we used $\frac{1}{2}$ for the scalar example because of the fact that there were two units, we use the same principle for the vector setting, where our units are columns. Here, $\mathbf{1}_2$ is a column vector of ones:

$$\mathbf{1}_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}.$$

Multiplying by $\frac{1}{2}$ ensures that each entry has the weight $\frac{1}{n}$, just like in the scalar arithmetic mean. The average is then given by the matrix product between the data vector and the weight vector:

$$\mathbf{xw} = \begin{bmatrix} 12 & 18 \end{bmatrix} \left(\frac{1}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \frac{1}{2}(12 + 18) = 15.$$

We get a scalar, in this case 15. We have two people (one in each column) in the vector, and we multiply every value in that column by the weight vector value, add these up, and this is the mean. The way we compute this in Python is:

```
import numpy as np

# --- Example 1: Apples ---
A = np.array([12, 18])
weights_apples = np.full(len(A), len(A)**-1)

# Dot Product
mean_apples = weights_apples @ A

print(f"Mean of Apples (dot product): {mean_apples}")
```

```
Mean of Apples (dot product): 15.0
```

We can represent the same treatment effect example as a row vector:

$$\boldsymbol{\tau} = \begin{bmatrix} 133 & 122 \end{bmatrix} \in \mathbb{R}^{1\times 2}.$$

Here, each column corresponds to a time period, and the value represents the treatment effect for that period. To compute the average, we use the same equal-weight vector as before, $\frac{1}{2}\mathbf{1}_2$. Again, why? We have two post-treatment periods, and per the definition of the arithmetic mean, we need the weight to be equal to the number of elements that we have. So when we do this, we get:

$$\boldsymbol{\tau}^\top \left(\tfrac{1}{2}\mathbf{1}_2 \right) = \begin{bmatrix} 133 & 122 \end{bmatrix} \cdot \left(\frac{1}{2} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 133 \times 0.5 & 122 \times 0.5 \end{bmatrix} = \begin{bmatrix} 66.5 & 61 \end{bmatrix}.$$

Summing the entries gives:

$$\mathbf{1}_2^\top \begin{bmatrix} 66.5 \\ 61 \end{bmatrix} = 66.5 + 61 = 127.5.$$

The average treatment effect across the two periods is 127.5, the same number as before. Thus, arithmetic averaging is simply taking a weighted sum where each weight is the reciprocal of the number of entries.

This can also be done with both examples using Python:

```
import numpy as np

# --- Example 2: ATT ---
y1_treated = np.array([533, 568])
y1_control = np.array([400, 446])
tau = y1_treated - y1_control

weights_tau = np.full(len(tau), len(tau)**-1)
ATT = weights_tau @ tau  # same as np.dot(weights_tau, tau) or np.mean(y1_treated - y1_c

print(f"Treatment effects by period: {tau}")
print(f"Average Treatment Effect on the Treated (dot product): {ATT}")
```

```
Treatment effects by period: [133 122]
Average Treatment Effect on the Treated (dot product): 127.5
```

Of course the above example is pedagogical. The real way we'd do this in code/a project is:

```
import numpy as np

y1_treated = np.array([533, 568])
y1_control = np.array([400, 446])

print(np.mean(y1_treated - y1_control))
```

```
127.5
```

Problem 3.6. Vector Mean of Small Numbers

Represent the sequence $[3, 6, 9]$ as a vector and compute the arithmetic mean using equal weights.

Problem 3.7. Vector Mean of Test Scores

A student scored $[65, 70, 75, 80]$ on four tests. Represent these as a vector and compute the arithmetic mean using equal weights.

Problem 3.8. Vector Mean of Daily Steps

Your daily steps for a week are $[5000, 6000, 5500, 7000, 6500, 6000, 5800]$. Represent this as a vector and compute the arithmetic mean with equal weights.

Problem 3.9. Vector Mean of Monthly Sales

A store's sales for three months are $[1200, 1500, 1300]$. Represent this as a vector and compute the average using equal weights.

Problem 3.10. Vector Mean of Heights

The heights (in cm) of four people are $[160, 165, 170, 175]$. Represent these as a vector and compute the arithmetic mean using equal weights.

3.2.3 Matrices

Definition 3.7. Matrix

A *matrix* is a rectangular grid of scalars arranged in rows and columns, an element of $\mathbb{R}^{m\times n}$ with m rows and n columns. Stacking several units' outcome vectors side by side gives a matrix; throughout this book we take rows to index time periods and columns to index units, so a panel of data is exactly a matrix.

Let's extend the idea to multiple time periods. Suppose today you have 12 apples and tomorrow 30, while your friend has 18 today and 22 tomorrow. We can represent this as a matrix:

$$\mathbf{Y} = \begin{bmatrix} 12 & 18 \\ 30 & 22 \end{bmatrix} \in \mathbb{R}^{2\times 2}.$$

Each row corresponds to a time period, and each column corresponds to a person.

Definition 3.8. Matrix–vector product

The *matrix–vector product* $\mathbf{Yw}$ takes the dot product of each row of $\mathbf{Y}$ with $\mathbf{w}$, one number per row — so weighting the columns by $\mathbf{w}$ averages across units, one time period at a time.

To find the average number of apples per time period, we multiply $\mathbf{Y}$ by the weight vector

$$\mathbf{w} = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}.$$

The result is a column vector, where each entry is the average across people for that time period:

$$\bar{\mathbf{x}} = \mathbf{Yw} = \begin{bmatrix} 12 & 18 \\ 30 & 22 \end{bmatrix} \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix} = \begin{bmatrix} 15 \\ 26 \end{bmatrix}.$$

Thus, the first entry 15 is the average for today, and the second entry 26 is the average for tomorrow. In Python, we'd do:

```python
import numpy as np

# --- Apples Example ---
Y = np.array([[12, 18],
              [30, 22]])

w_units = np.full((Y.shape[1], 1), Y.shape[1]**-1)  # column weights (for people)
avg_by_time = Y @ w_units       # average across people for each time period

print("Average apples per time period:")
print(avg_by_time)
```

```
Average apples per time period:
[[15.]
 [26.]]
```

Suppose we have two treated units over two post-treatment periods. Let the predicted outcomes and observed outcomes be:

$$\hat{\mathbf{Y}} = \begin{bmatrix} 400 & 380 \\ 446 & 420 \end{bmatrix}, \quad \mathbf{Y} = \begin{bmatrix} 533 & 490 \\ 568 & 510 \end{bmatrix}.$$

Here, each row corresponds to a post-treatment period, and each column corresponds to a treated unit. The treatment effects are the differences between the observed outcomes and their predicted outcomes:

$$\boldsymbol{\tau} = \mathbf{Y} - \hat{\mathbf{Y}}.$$

We typically are concerned with the average treatment effect on the treated units, where we average first across units (columns) and then across time periods. To do this, we have weights for each unit (that is, the number of columns) **and** for each particular time point,

$$\mathbf{w}_{\text{units}} = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}, \quad \mathbf{w}_{\text{time}} = \begin{bmatrix} 1/2 \\ 1/2 \end{bmatrix}.$$

This is simply a generalization of th rules we've been developing so far, where units get weight when we average across them, and extending that to the setting where time periods also get weight defined by their length. So, unlike where we have one treated unit and one predicted series, this generalizes very nicely to the setting where we have multiple treated units too:

$$\text{ATT} = \mathbf{w}_{\text{time}}^{\top} \boldsymbol{\tau}\, \mathbf{w}_{\text{units}} = \mathbf{w}_{\text{time}}^{\top} (\mathbf{Y} - \hat{\mathbf{Y}}) \mathbf{w}_{\text{units}}.$$

We start with

$$\text{ATT} = \begin{bmatrix}1/2 & 1/2\end{bmatrix} \begin{bmatrix}133 & 110\\ 122 & 90\end{bmatrix} \begin{bmatrix}1/2\\ 1/2\end{bmatrix}.$$

The first step is to multiply the weight vector on the left (the time vector) by the matrix of treatment effects:

$$\begin{bmatrix}1/2 & 1/2\end{bmatrix} \begin{bmatrix}133 & 110\\ 122 & 90\end{bmatrix} = \begin{bmatrix}\frac{1}{2}\cdot 133 + \frac{1}{2}\cdot 122 & \frac{1}{2}\cdot 110 + \frac{1}{2}\cdot 90\end{bmatrix} = \begin{bmatrix}127.5 & 100\end{bmatrix}.$$

Okay now this one is easy. Now we just multiply the resulting row vector by the column weight vector, or the average of the ATTs between each unit:

$$\begin{bmatrix}127.5 & 100\end{bmatrix} \begin{bmatrix}1/2\\ 1/2\end{bmatrix} = 127.5 \cdot 1/2 + 100 \cdot 1/2 = 63.75 + 50 = 113.75.$$

Again in `NumPy`, we'd do

```
Treatment effect matrix (tau):
[[133 110]
 [122  90]]

Average Treatment Effect: 113.75
```

Averaging is just a form of a dot product that we take across sequences/vectors and matrices. In regression and synthetic control methods, this same idea extends from scalars to matrices. Suppose $\mathbf{y}_1 \in \mathbb{R}^T$ is a target time series and $\mathbf{Y}_0 \in \mathbb{R}^{T\times J}$ is a matrix whose columns contain J potential control series. The goal is to approximate $\mathbf{y}_1$ using a weighted combination of the columns of $\mathbf{Y}_0$. We can express this as minimizing the squared error

$$f(\mathbf{w}) = \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2 = \sum_{t=1}^{T}\left(y_{1t} - \sum_{j=1}^{J} w_j Y_{0,tj}\right)^2.$$

The solution $\mathbf{w}$ gives the weights that produce the best-fitting linear combination of vectors in the least-squares sense. The mean, variance, and mean squared error are therefore all instances of a common idea: each is defined by minimizing the total squared deviation from a reference point. The mean is the minimizer, the variance is the minimized value of the objective, and the mean squared error is the same idea applied to predictions or reconstructions rather than to raw data. Understanding how averaging arises from this optimization perspective provides a unified way to interpret many methods in statistics, including regression and synthetic control, as procedures that compute structured weighted averages.

Problem 3.11. Matrix Representation of Student Scores

Two students take two exams. Their scores are:

	Akiko	Tsumugi
Exam 1	80	70
Exam 2	90	85

Represent this as a matrix and compute the average score per exam. Then, compute the average score per student instead.

Problem 3.12. Matrix of Daily Temperatures

The temperatures (°C) in two cities over three days are:

	City A	City B
Day 1	20	25
Day 2	22	28
Day 3	21	26

Represent this as a matrix and compute: 1. The average temperature per day. 2. The average temperature per city.

Problem 3.13. Matrix of Weekly Sales

A store tracks sales for two products over four weeks:

	Product X	Product Y
Week 1	100	120
Week 2	110	130
Week 3	90	140
Week 4	105	125

Represent this as a matrix. Compute: 1. Average sales per week. 2. Average sales per product.

Problem 3.14. Matrix of Monthly Rainfall

Monthly rainfall (mm) for two cities over three months:

	City X	City Y
Jan	50	60
Feb	45	55
Mar	55	65

Represent this as a matrix. Compute: 1. Average rainfall per month. 2. Average rainfall per city.

Problem 3.15. Matrix of Machine Outputs

Two machines produce units per day for three days:

	Machine 1	Machine 2
Day 1	200	220
Day 2	210	230
Day 3	205	225

Represent this as a matrix. Compute: 1. Average output per day. 2. Average output per machine.

3.3 What Are We Weighting For? Introducing Weighted Averages

Averages allow us compare units (whether people, firms, or countries) across time or groups. This is the reason economists use them so much. In the treatment effect example we just did, we can compare the average treatment effect for one unit to that of another unit. However so far there is a detail I have omitted: we have so far taken it as an article of faith that the weight vector should be $\frac{1}{n}$. But why should this be? In other words, does it have to always be true that n needs to be the length of the columns or rows? Well, no! As it turns out, the weights can even vary across units and time periods! However, the moment we leave the special case where the weights are the reciprocal of the size of the entries, now the choice of weights matters. For comparison purposes, not all units should get the same weight.

Consider a simple example: you earn \$60k/year. Your friend earns \$70k/year. The arithmetic mean is:

$$\mathbf{x} = \begin{bmatrix} 60000 & 70000 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} \frac{1}{2} \\ \frac{1}{2} \end{bmatrix},$$

which evaluates to:

$$\bar{x} = \mathbf{xw} = 0.5 \cdot 60000 + 0.5 \cdot 70000 = 65000 \ / \ \text{year}.$$

Now suppose Mansa Musa joins, earning \$107 billion/year. Mansa Musa's income must be treated equally to you two. So, now the average becomes:

$$\mathbf{x} = \begin{bmatrix} 60000 & 70000 & 1.07 \times 10^{11} \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} \frac{1}{3} \\ \frac{1}{3} \\ \frac{1}{3} \end{bmatrix},$$

$$\bar{x} = 0.33 \cdot 60000 + 0.33 \cdot 70000 + 0.33 \cdot 1.07 \times 10^{11} \approx 35.7 \text{ B / year}.$$

In Python, the way we'd compute this is:

```
import numpy as np

# --- Example 1: Two friends ---
x1 = np.array([60000, 70000])
w1 = np.full((len(x1), 1), 1 / len(x1))

mean_equal = x1 @ w1

print(f"Average income: ${mean_equal.item():,.0f}\n")

# --- Example 2: Adding Mansa Musa ---
x2 = np.array([60000, 70000, 1.07e11])
w2 = np.full((len(x2), 1), 1 / len(x2))

mean_equal_big = x2 @ w2

print(f"Average income: ${mean_equal_big.item():,.2f}")
```

```
Average income: $65,000

Average income: $35,666,710,000.00
```

Clearly, the average is dominated by this extreme outlier, giving a value that is no longer representative of the original group. If the bartender next day bragged to his investors about earning 50k that night and wanted to expand his business, they would say something like: "It is not that you earned 50k organically, a mega-wealthy king was in town and happened to be there that night, that is not a typical night for you. If we looked at your pre-Mansa period, this is almost certainly an outlier relative to your own previous growth as well as the growth of you local competitors."

They would say what we know by simple intuition: on nights where wealthy people are patrons, earnings will very likely be anomalous with respect to other normal nights and cannot be used for decision-making. In other words, if we want a meaningful "bar average," Mansa as an individual should likely get much less weight (if any) than the other patrons. The arithmetic mean assumes equal and proportional contribution from each unit, which may not be appropriate for meaningful comparisons.

This leads us to weighted averages. Recall the idea of the weight that I have been referring to, $\frac{1}{n}$. This is essentially the fraction that we scale the original data by for the weighted sum. In this case, the weight is proportional to the size of the number of units we have at hand. Of course, you may now be asking "Well, are other kinds of averages possible? Are we just left with the arithmetic mean?" As it turns out, yes and no respectively. The weight is a mere convention of arithmetic means; there is no law of nature or script from the Heavens that says the weight for each element must be $1/n$ Practically, we can assign weights however we like. So, instead of treating Mansa as equal to you two, or treating the bar that gets Mansa similar to other bars, we can give each person/bar a weight $w_i \in [0, 1]$.

Definition 3.1 (Weighted Average)**.** For a sequence of numbers $(x_1, x_2, \ldots, x_n) \in \mathbb{R}^n$ and a vector of weights $\mathbf{w} = (w_1, w_2, \ldots, w_n) \in \mathbb{R}^n$, the weighted average is defined as:

$$\mathbf{x}^\top \mathbf{w} = \sum_{i=1}^{n} w_i x_i$$

The classical arithmetic mean is a special case where all weights are equal to $1/n$.

When we think about the weights $\frac{1}{w_i}$, it is best to think about it as a decision variable that decides how much an individual/unit counts in the average. Now reconsider the average with this in mind: with a weighted average, Mansa can get a tiny weight (if any), and you and your friend get larger weights, giving a more reasonable "average bar income." If we assign Mansa weight 0 and you and your friend weight 0.5 each, we obtain:

$$\mathbf{w} = \begin{bmatrix} 0.5 \\ 0.5 \\ 0 \end{bmatrix}, \quad \bar{x} = \mathbf{x}\mathbf{w} = 0.5 \cdot 60000 + 0.5 \cdot 70000 = 65000 \,/\text{year}.$$

Now the average reflects the typical case, making it a useful comparison for "regular" units. In Python, the way we'd do this is:

```
import numpy as np

# Incomes: You, Friend, Mansa Musa
x = np.array([60000, 70000, 1.07e11])

# Assign weights
w = np.array([0.5, 0.5, 0])

# Compute weighted average
weighted_mean = x @ w

print(f"Weighted average: ${weighted_mean:,.0f}.")
```

```
Weighted average: $65,000.
```

Now suppose the weight vector is:

$$\mathbf{w} = \begin{bmatrix} 0.15 \\ 0.85 \\ 0 \end{bmatrix}.$$

The weighted average is computed as:

$$\bar{x}_{\text{weighted}} = 0.15 \cdot 60000 + 0.85 \cdot 70000 + 0 \cdot 1.07 \times 10^{11} = 68500$$

So the weighted average income is \$68,500/year. We can see that this is closer to your friend's income compared to the case where you both shared equal weight, and certainly very different from the setting where Mansa counts at all.

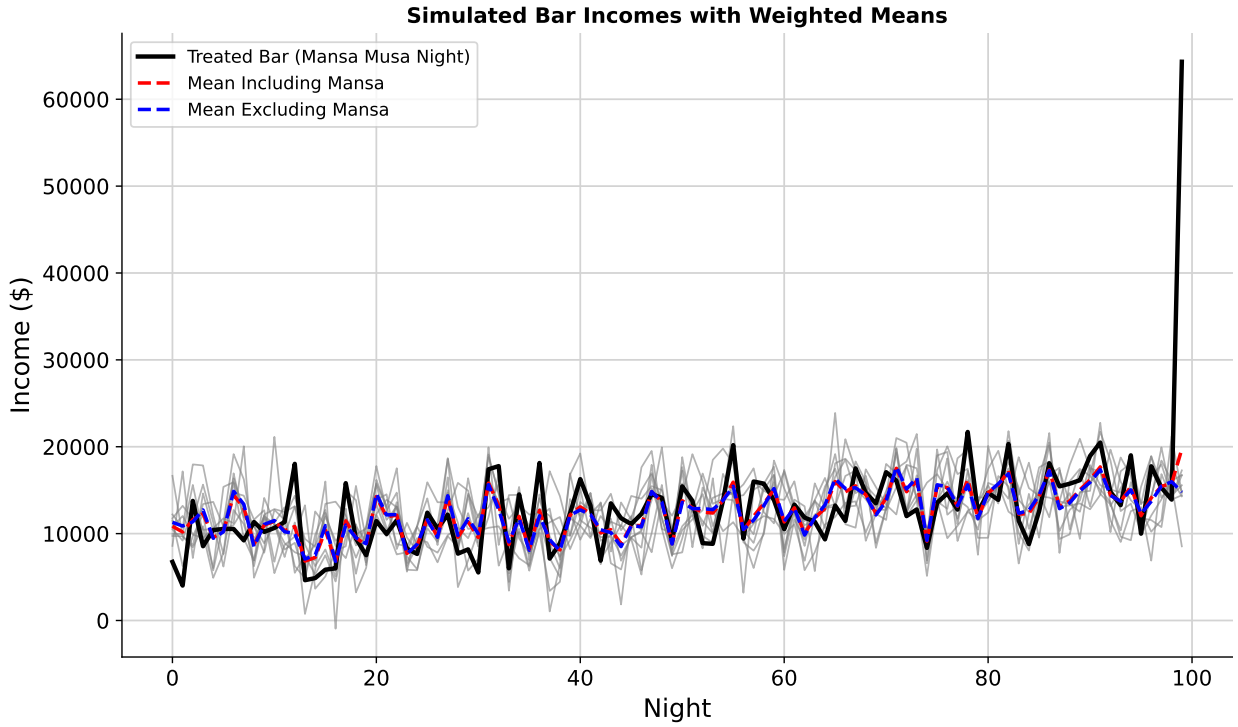
```
import numpy as np

x = np.array([60000, 70000, 1.07e11])
w = np.array([0.15, 0.85, 0]).reshape(-1, 1)

weighted_mean = x @ w
print(f"Weighted average income: ${weighted_mean.item():,.0f}")
```

```
Weighted average income: $68,500
```

Now, consider a panel data version of the above bar example, where now we are interested in comparing one bar to ten other bars:



Here, we can see the the average where the Mansa bar is included pulls the average upwards to roughly 20,000 at the end of the time series. In applied work, group level averages appear all the time, where we compare a single set of units to the mean of a larger group. This idea is also frequent in recent news articles where comparisons are made. Examples like crime rates, healthcare spending, and gasoline prices compare a unit to a larger mean to make inferences from them.

Problem 3.16. Impact of an Outlier on the Mean

One of your friends suddenly receives a lottery prize of $5 million. Compute the new arithmetic mean. Comment on how representative this new mean is of the original group.

Problem 3.17. Weighted Averages

Suppose you want some of the first three students to have more influence than the last one. Assign an arbitrary set of weights and compute the weighted average. Explain the difference with the simple average.

Problem 3.18. Weighted Averages, Applied

From this problem, assign Akiko weight 0.6 and Tsumugi weight 0.4 for the average score for each exam. Comment on how the result changes.

Problem 3.19. Limitations of the Arithmetic Mean

Explain why simply taking the arithmetic mean can be misleading. Give a real-life example.

3.4 Conclusion

In this chapter, we have explored averages in their simplest forms—scalars, vectors, and matrices—and introduced the notion of weighted averages. The key takeaway is that averages are tools for comparison, but their usefulness depends entirely on the weights we assign. In real-world applications, we rarely know the "correct" weights in advance.

You may be thinking this was overkill—why bother with an optimization derivation for something as simple as an average? Because the derivation shows that the average is the solution to an optimization problem, and that single insight connects simple statistics to the machinery of modern econometrics. Variance, standard deviation, mean squared error, regression coefficients, and the synthetic control weights are all built on the same principle: finding a number, or a set of numbers, that minimizes some measure of error. The arithmetic mean is just the simplest, most familiar instance of it.

Sometimes, people have told me in the context of my blog have suggested that much of this content, especially around synthetic controls, would be easier to digest if all technical language were removed. While this might be true for a general audience, it misunderstands who my blog/this book is written for. The primary audience is not business stakeholders, but 1) the employees and practitioners who use data to generate empirical insights for businesses and 2) graduate students who wish to have a good understanding of SCM.

With this in mind, I want to underscore how crucial the idea of averaging is for a data practitioner who seeks to use SCM. At some point, correct explanations cannot be fully conveyed without mathematical formality. Averages are so central to the next chapters that a thorough understanding is essential to grasping and applying the ideas presented. Soon, we will explore how to generate weights in order to construct meaningful comparisons. This, however, is impossible without a firm grasp of averages. Understanding them is not just an academic exercise: averaging forms the foundation for most comparisons in econometrics, and SCM is no exception.

Problem 3.20. Mean as Minimizer of Squared Error

Consider the numbers $x = [4, 7, 10, 6]$.

1. Compute the arithmetic mean $\bar{x}$.

2. Verify that $\bar{x}$ minimizes the sum of squared deviations by computing $\sum_{i=1}^{4}(x_i - \mu)^2$ for $\mu = 5, 6.75, 8$.

Problem 3.21. Variance as Average Squared Deviation

Using the same numbers $x = [4, 7, 10, 6]$:

1. Compute the total squared deviation from the mean.

2. Compute the variance (average squared deviation).

3. Interpret the variance: does it indicate the data are spread out or tightly clustered around the mean?

Problem 3.22. Mean Squared Error for Predictions

A model predicts values $[12, 15, 14, 16]$ while the observed values are $[13, 14, 16, 15]$.

1. Compute the error for each observation.

2. Compute the squared error for each observation.

3. Compute the mean squared error (MSE).

4. Interpret the MSE in terms of prediction accuracy.

Problem 3.23. Weighted Average in Vector Form

Suppose we have a vector of observations $\mathbf{y} = [3, 5, 8]$ and a uniform weight vector $\mathbf{w} = [1/3, 1/3, 1/3]^\top$.

1. Compute the weighted average using the dot product $\mathbf{y} \cdot \mathbf{w}$.

2. Compute the total squared deviation of $\mathbf{y}$ from this weighted average.

Problem 3.24. Vectorized MSE in Regression

Let $\mathbf{y}_1 = [5, 7, 6]$ be observed outcomes, and let $\mathbf{Y}_0 = \begin{bmatrix} 4 & 6 \\ 6 & 8 \\ 5 & 7 \end{bmatrix}$ be two predictor vectors.

If the weight vector is $\mathbf{w} = [0.4, 0.6]^\top$:

1. Compute the predicted values $\mathbf{Y}_0\mathbf{w}$.

2. Compute the residual vector $\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}$.

3. Compute the Mean Squared Error (MSE) using the formula $\frac{1}{n}\|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2$.

Chapter 4

Getting Started with mlsynth

This is not a book about Python. Python is simply the language I use to estimate synthetic control methods, and `mlsynth` is the package I wrote to do it. The book is not about `mlsynth` either, but I lean on it constantly to keep the examples short, so this chapter covers enough of it that the rest of the book reads without friction.

The plan is small. You will install the package, learn the one pattern every estimator follows, learn what an estimator hands back to you when it is finished, and meet the two utilities I reach for before any estimation at all.

4.1 Installing mlsynth

`mlsynth` installs from GitHub:

```
pip install git+https://github.com/jgreathouse9/mlsynth.git
```

It needs Python 3.10 or newer. The install pulls in the usual scientific stack — `numpy`, `pandas`, `scipy`, `matplotlib`, `scikit-learn`, `statsmodels` — plus four you may not have met: `cvxpy` (the convex solver that does the actual weight-finding in the convex synthetic control chapter), `pydantic` (which validates your inputs before any arithmetic happens), `pyarrow`, and `PyYAML`.

A handful of estimators need heavier machinery — a mixed-integer solver, or JAX — and those are optional extras, not part of the base install, so the package stays light for everyone who does not need them:

```
pip install "mlsynth[design] @ git+https://github.com/jgreathouse9/mlsynth.git"
```

Nothing in this book needs an extra. The base install is enough.

4.2 One pattern, every estimator

Every estimator in `mlsynth` works the same way, and once you have seen it once you have seen all of them:

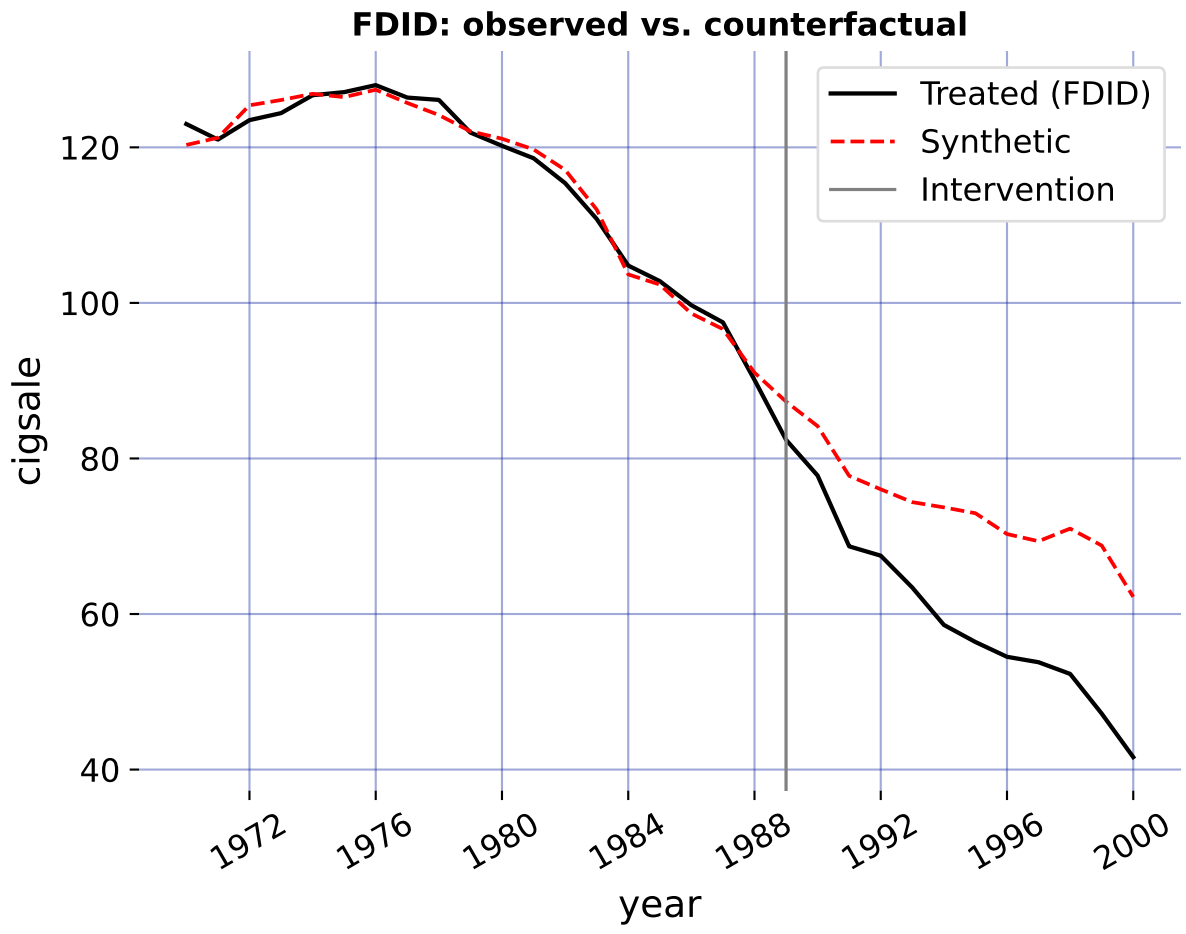
1. Put your panel in a long `DataFrame` — one row per unit per period.
2. Write a configuration describing which column is which.
3. Hand that configuration to an estimator class and call `.fit()`.

That is the whole interface. The configuration is a Python dictionary: a structure that stores key–value pairs, where each key names a piece of information and the value holds it. Five keys are required by every estimator, and together they say what each column of your panel means.

Definition 4.1. The five required configuration keys

- `df` — the long panel `DataFrame`.
- `unitid` — the column naming the cross-sectional units (states, regions, stores).
- `time` — the column naming the time periods.
- `outcome` — the column holding the variable you care about.
- `treat` — the column holding the treatment indicator: `1` when that unit is under treatment in that period, `0` otherwise.

Here is the whole thing on Proposition 99, California's 1988 tobacco tax. I use the Forward DID estimator (`FDID`) because it needs no tuning. The control-group mean chapter is where the method itself gets explained; here it is only a vehicle for showing the interface.



Notice what I did not do. I did not pivot the panel into a wide matrix, separate California from the other states, work out where the pre-treatment period ends, or assemble a donor matrix. The estimator did all of it from those five keys. This matters more than it looks: every one of those steps is a place to introduce a silent off-by-one that produces a number instead of an error.

The configuration is validated before anything is computed. Misspell a column name and you get a clear complaint immediately, not a confusing failure three functions deep. That is `pydantic` doing its job, and it is deliberate — a synthetic control estimate that is wrong without saying so is worse than one that refuses to run.

4.3 What you get back

This is the part that repays a few minutes of attention, because it is where beginners usually stall.

`.fit()` returns a *result object*, not a number and not a dictionary. Every estimator in the package returns the same shape of object, so what you learn here transfers to all of them. The things you most often want are available directly:

```
ATT:            -13.647
95% interval:   (-14.549, -12.745)
Pre-period RMSE: 1.248
```

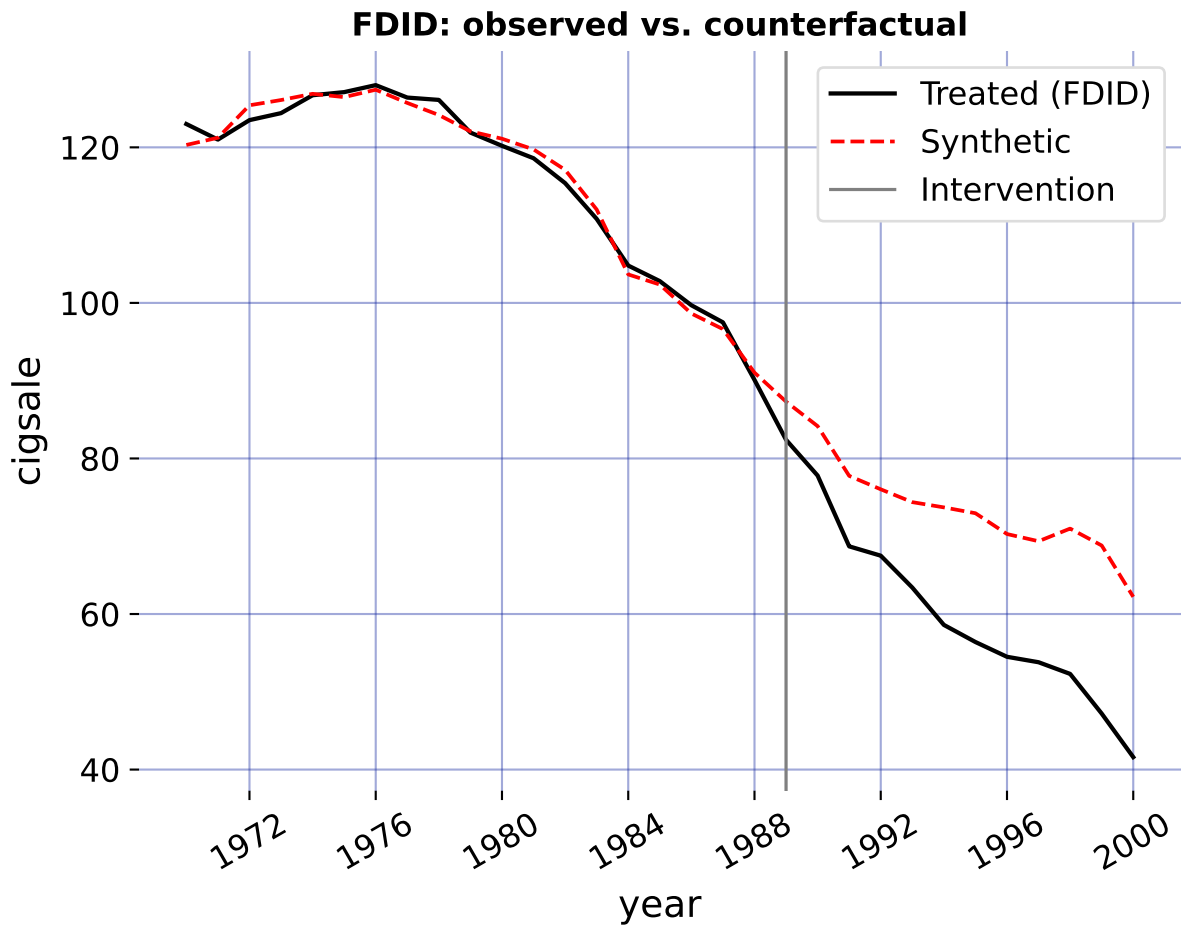
The ATT here is about -13.6 cigarette packs per capita: California smoked roughly fourteen packs per person per year less than its synthetic counterpart after the tax. The pre-period RMSE of about 1.2 packs says the synthetic California tracked the real one closely before 1989, which is what licenses reading the gap afterwards as an effect.

Three more accessors carry the series and the weights:

```
counterfactual: [120.31 121.23 125.41 126.08 126.88] ...
gap:            [ 2.69 -0.23 -1.91 -1.68 -0.18] ...
donors used:    {'Montana': 0.25, 'Colorado': 0.25, 'Nevada': 0.25, 'Connecticut': 0.25}
```

`counterfactual` is the synthetic California — what the model says would have happened without the tax. `gap` is the observed series minus that counterfactual, period by period; it is the treatment effect over time, and it is what gets plotted in every synthetic control paper you have read. `donor_weights` maps each control state to its weight. Forward DID picked four states and split the weight evenly among them, which is a property of that particular estimator, not of synthetic control generally — the convex synthetic control chapter builds estimators that choose unequal weights.

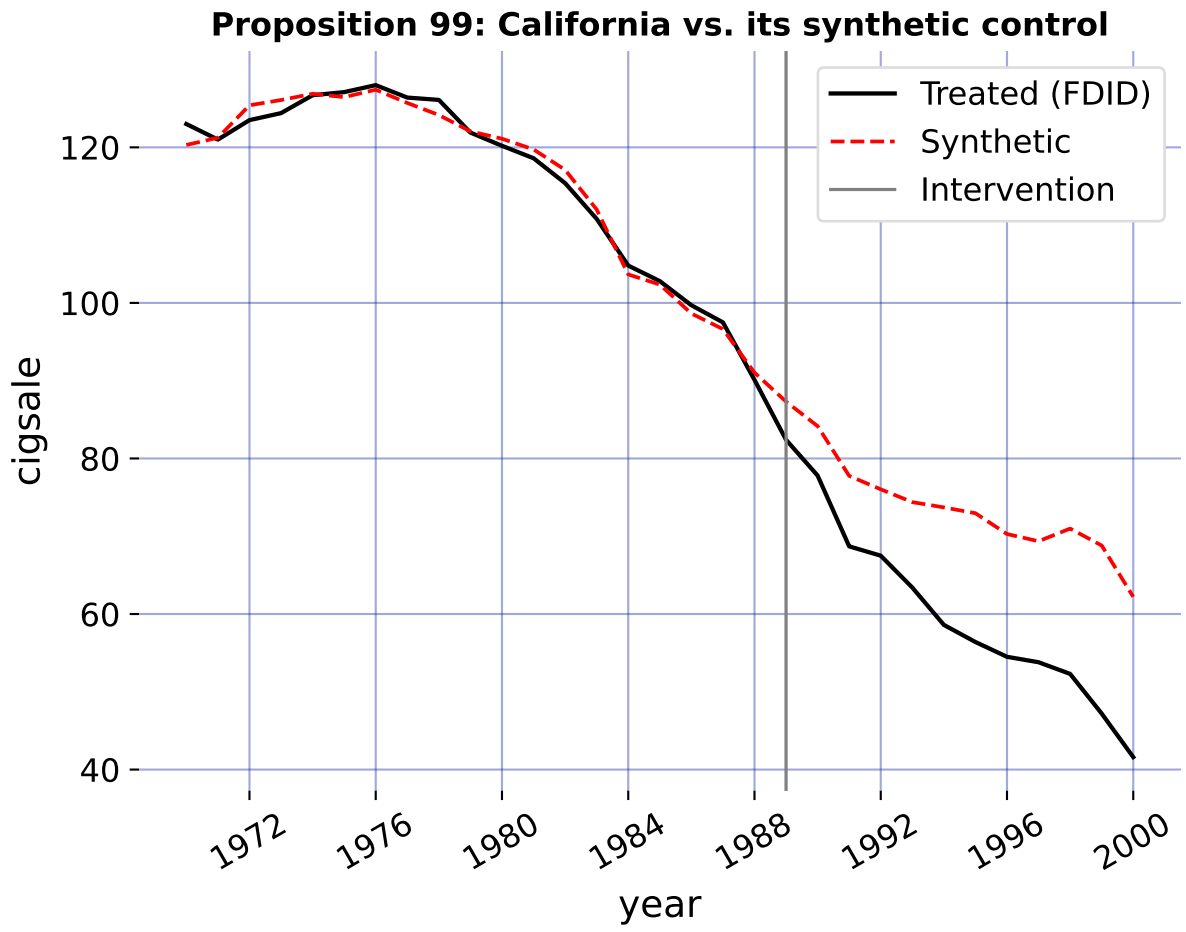
Because the result object knows how to draw itself, you do not have to:



The vertical line marks the intervention. Before it, the two series should sit on top of each other; after it, the distance between them is the estimate.

> **Note**
>
> Older code you may find online sets `"display_graphs": True` in the configuration and lets `.fit()` draw as a side effect. That still works, but `result.plot()` is the better habit: it separates computing from drawing, so you can fit once and plot whenever — or never, if you are running a hundred fits in a loop and want none of them on screen.

If you want to change how the figure looks, the configuration takes a `plot` entry:



4.4 Preparing data with dataprep

`dataprep` is the function underneath all of this. The estimators call it for you, so most of the time you never touch it — but when you want the raw pieces, to build something yourself or to check what the estimator saw, this is how you get them.

Many synthetic control methods want the panel wide: rows are time periods, columns are units, treated and donors separated, pre- and post-treatment periods identified. `dataprep` does that conversion, and it checks the panel is strongly balanced along the way — every unit observed in every period, no duplicated unit–period rows. A panel that fails those checks raises instead of returning something subtly malformed.

```
treated unit:   Basque Country (Pais Vasco)
pre / post:     20 / 23
donor matrix:   (43, 16) (periods x donors)
keys available: ['Ywide', 'donor_matrix', 'donor_names', 'post_periods', 'pre_periods',
```

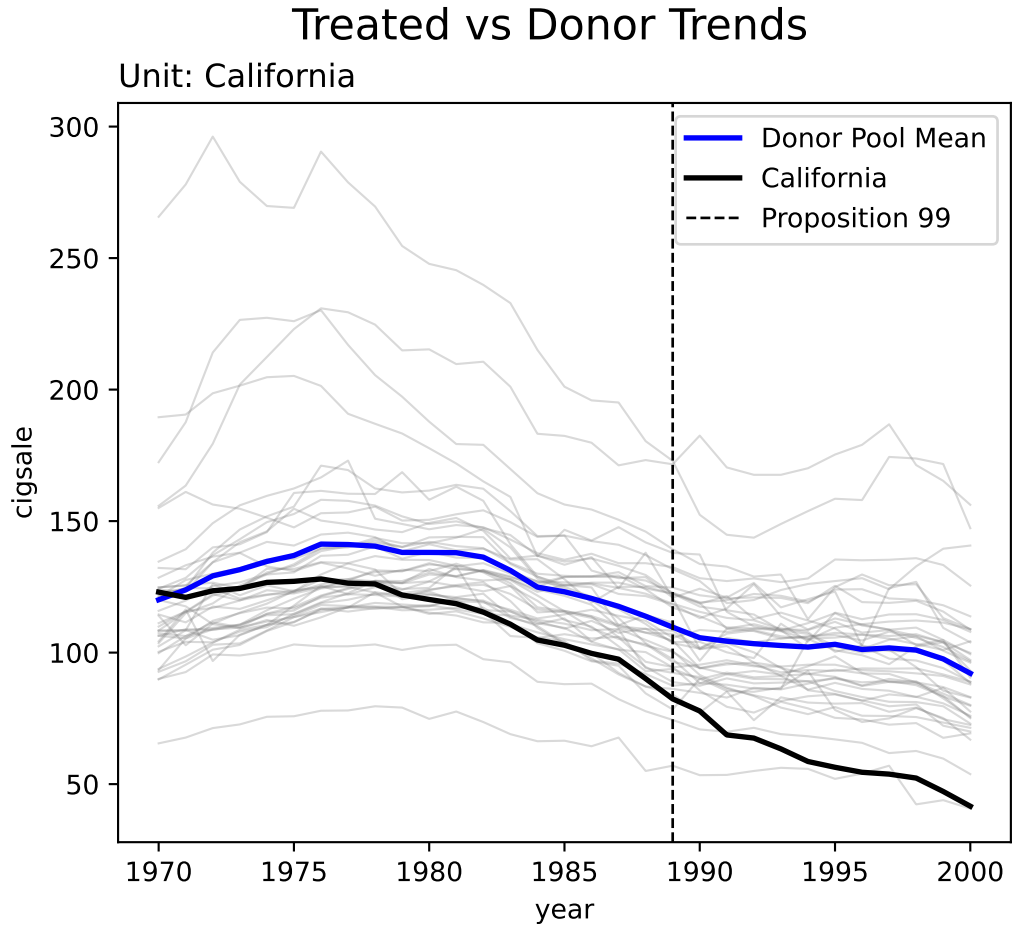
The arguments go in the order *data, unit, time, outcome, treatment*. The return is a dictionary, and the pieces you will use most are `y` (the treated unit's outcome vector), `donor_matrix` (the same outcome for every untreated region, one column each), `pre_periods` (how many

periods precede the intervention), and `time_labels` (the period labels, for plotting).

Those four objects are the whole problem. Every method in this book is a different answer to one question: what combination of the columns of `Y0` best reproduces `y` over the first `T0` periods?

4.5 Looking before you estimate

Before fitting anything, look at the data. `sc_diagplot` draws the treated unit against the donor pool and the donors' mean:



The configuration is the same five keys as before, wrapped in a list because the function accepts several panels at once for side-by-side comparison.

What you are looking for is whether the treated unit is *inside* the donor pool's range before treatment. If California sits in the middle of the cloud, some weighted average of those states can plausibly reproduce it. If it sits above every donor, no convex combination can reach it, and the methods in the convex synthetic control chapter will fit poorly no matter how you tune them — a fact the plot tells you in one second and the estimator tells you much later, if at all. Looking first is cheap.

Problem 4.1. Reading the interface

Load the Proposition 99 data and fit `FDID` as above.

1. What does `result.gap` contain in the periods *before* 1989, and why is it not exactly zero?
2. `result.att` is a single number summarizing `result.gap`. Over which periods is that summary taken?

Problem 4.2. From dataprep to arithmetic

Using the Basque panel and `dataprep` as above:

1. Compute the simple average of the donor regions in each period — that is, the row means of `Y0` — and plot it against `y`.
2. This is the control-group mean of the control-group mean chapter, computed by hand. How well does it track the Basque Country before 1975?
3. What would have to be true about the donor regions for this equally-weighted average to be a credible counterfactual?

Problem 4.3. When the hull is empty

Run `sc_diagplot` on the Proposition 99 panel and find the periods where California lies outside the range of every donor state.

Explain what a weighted average with non-negative weights summing to one can and cannot do in those periods. You are deriving, ahead of time, a limitation we spend all of the convex synthetic control chapter dealing with.

4.6 Conclusion

Everything in `mlsynth` is the same three steps: describe the panel in a dictionary, pass it to an estimator, call `.fit()`. What comes back is always the same kind of object, so `result.att`, `result.gap`, `result.counterfactual`, `result.donor_weights` and `result.plot()` mean the same thing whichever estimator produced them. Underneath, `dataprep` turns your long panel into the treated vector `y` and the donor matrix `Y0`, and `sc_diagplot` shows you whether those two objects stand a chance before you spend any effort on estimation.

The rest of the book is about the interesting question the interface hides: given `y` and `Y0`, how should the weights be chosen? We start, in the next chapter, with the simplest possible answer — give every donor the same weight — and spend the remainder of the book discovering why that is not good enough.

Part II

Estimation and Inference

Chapter 5

Potential Outcomes and Comparative Case Studies

> "If there's another universe, please make some noise. Give me a sign..."
> — Solana Imani Rowe

This chapter introduces the potential outcomes framework—the cornerstone of causal inference—and connects it to the comparative case study framework that underlies SCM (Rubin, 2008). Together, they provide a formal way to think about causality when experiments are infeasible. In principle, causal inference is about constructing another universe: one where what did happen, did not happen. The goal is to use observable data and statistical structure to make that alternate universe as realistic and plausible as possible. Note that this chapter cannot cover all of the basics– there is already a very mature body of work that treats this in more detail. Because I do not wish to reinvent wheels, I refer readers to much more thorough texts such as Cunningham's *Mixtape*, Huntington-Klein's *The Effect*, and Facure's *Causal Inference for the Brave and True* (Cunningham, 2021; Facure Alves, 2022; Huntington-Klein, 2025). The mathematical foundation introduced in the previous chapter now becomes essential. It allows us to express these ideas formally, to see why counterfactual reconstruction is even possible, and to understand the role of similarity between units. We begin by defining the key ideas behind potential outcomes, clarifying why causal effects are unobservable for any individual unit, and introducing the notation and modeling ideas that make these concepts precise.

We then turn to the comparative case study framework, which shows how researchers and practitioners attempt to approximate those missing outcomes in practice. When policymakers, marketers, or data scientists evaluate an intervention—be it a policy reform, a new advertising campaign, or an infrastructure project—they face the same profound challenge: we only ever observe one version of history. Randomized experiments, while ideal, are often impossible, unethical, or prohibitively expensive. The comparative case study approach provides a structured way to reason about "what would have happened otherwise," using other units or regions as stand-ins for the unobserved counterfactual. At the end of this chapter, we will connect this logic to real examples—such as the case of West Germany's reunification.

5.1 The Fundamental Problem of Causal Inference

Academic and business scientists face a fundamental challenge: for any given unit (e.g., a country, region, or individual), we can only observe one outcome at a time (Holland, 1986). If a unit receives a single treatment, we see the version of the outcome we see given that the treatment has happened; if it does not, we see the untreated outcome. Formally, for unit j at time t we have:

$$y_{jt} = d_j y_{jt}(1) + (1 - d_j) y_{jt}(0),$$

where $d_j \in \{0, 1\}$ indicates treatment status. The key algebraic fact here formalizes the intuition above: given that a treatment happens, the untreated outcomes $y_{jt}(0)$ go away. Given that the treatment has not yet been enacted, we do not see $y_{jt}(1)$, or the treated outcomes.

Writing $y_{jt} = y_{jt}(d_j)$ so casually already smuggles in an assumption, and it is worth stating plainly, since the rest of the book leans on it.

Assumption 5.1. Stable Unit Treatment Value Assumption

Two conditions hold. First, *no interference*: the treatment assigned to one unit does not affect any other unit's potential outcomes, so y_{jt} depends only on unit j's own treatment status d_j. Second, *no hidden variation in treatment*: there is a single, well-defined version of the treatment, so the symbols $y_{jt}(1)$ and $y_{jt}(0)$ are unambiguous. Under both, the observed outcome equals the potential outcome for the treatment actually received, $y_{jt} = y_{jt}(d_j)$.

Interference is exactly the failure we return to at the end of this chapter under the heading of spillovers; for now we assume it away. Another way of writing out what we observe, unit by unit and period by period, is

$$y_{jt} = \begin{cases} y_{jt}(0) = f_j(t), & t \in \mathcal{T}_1,\ j \in \mathcal{N}, \\ y_{jt}(0) = f_j(t), & t \in \mathcal{T}_2,\ j \in \mathcal{N}_0, \\ y_{jt}(1) = f_j(t) + \tau_{jt}, & t \in \mathcal{T}_2,\ j \in \mathcal{N}_1, \\ y_{jt}(0) = f_j(t), & t \in \mathcal{T}_2,\ j \in \mathcal{N}_1 \text{ (unobserved)}, \end{cases}$$

where $f_j(t)$ represents the *generic outcome-generating process* in the absence of treatment, and τ_{jt} is the *treatment effect*, potentially varying across time and units. The first two rows are untreated potential outcomes we get to see (everyone before treatment, and the controls throughout); the third row is the treated unit's realized outcome after treatment; the fourth is the one quantity we never observe — the treated unit's untreated path in the post-period, which is precisely the counterfactual to reconstruct.

The first row asserts something we should also name: before treatment, even the units destined for treatment sit at their untreated outcome $f_j(t)$. That is not automatic — a firm might change behavior in anticipation of a policy it knows is coming.

Assumption 5.2. No Anticipation

Treated units do not respond to the intervention before it occurs. For $j \in \mathcal{N}_1$ and $t \in \mathcal{T}_1$, the pre-treatment outcome equals the untreated potential outcome, $y_{jt} = y_{jt}(0) = f_j(t)$; equivalently $\tau_{jt} = 0$ for every $t \in \mathcal{T}_1$.

No anticipation is what lets the pre-treatment period stand as a clean record of the treated unit's untreated process — the object every comparative method uses the donors to reconstruct. The algebra above suggests that we can recover the effect by forming an estimate of $y_{jt}(0)$ for all $t \in \mathcal{T}_2$. Given some estimate of $y_{jt}(0)$, we may compute the treatment effect of the policy or intervention. Since individual causal effects are unobservable, we often estimate average effects across units or time. Define:

Definition 5.1.

Definition 5.1. Average Treatment Effect

The Average Treatment Effect (ATE) measures the expected difference in potential outcomes if all units were treated versus if none were treated:

$$\text{ATE} = \mathbb{E}[\, y_{jt}(1) - y_{jt}(0)\,].$$

Definition 5.2.

Definition 5.2. Average Treatment Effect on the Treated

The Average Treatment Effect on the Treated (ATT) measures the expected treatment effect *for those units that actually received the treatment*:

$$\text{ATT} = \mathbb{E}[\, y_{jt}(1) - y_{jt}(0) \mid d = 1\,].$$

These averages rely on comparing treated and untreated units, assuming we can approximate the counterfactual. Practically, however, these hold only in expectation. We usually work with the in-sample analogs of these:

Definition 5.3.

Definition 5.3. Sample Average Treatment Effect

The sample analog of the ATE is the difference in the average observed outcomes between treated and untreated units:

$$\widehat{\text{ATE}} = \frac{1}{N_1} \sum_{j:d_j=1} y_{jt} - \frac{1}{N_0} \sum_{j:d_j=0} y_{jt},$$

where N_1 and N_0 are the number of treated and untreated units, respectively.

Worked example: If your observed outcomes are $10, 12, 14$ and the outcomes for untreated units are $7, 8, 9$, then the sample $\widehat{\text{ATE}}$ is

$$(10 + 12 + 14)/3 - (7 + 8 + 9)/3 = 12 - 8 = 4.$$

Definition 5.4.

Definition 5.4. Sample Average Treatment Effect on the Treated

The sample analog of the ATT focuses on treated units, using untreated units to approximate their counterfactual outcomes:

$$\widehat{\text{ATT}} = \frac{1}{N_1} \sum_{j:d_j=1} y_{jt}(1) - \frac{1}{N_1} \sum_{j:d_j=1} \widehat{y}_{jt}(0),$$

where $\widehat{y}_{jt}(0)$ denotes the estimated counterfactual outcome for treated unit j in period t.

Worked example: If your observed outcomes are $15, 18, 20$ but absent treatment they would have been $12, 14, 16$, then the sample $\widehat{\text{ATT}}$ is

$$(15 + 18 + 20)/3 - (12 + 14 + 16)/3 = 17.67 - 14 = 3.67.$$

Problem 5.1. Potential Outcomes

Explain in your own words the fundamental problem of causal inference. Why can we never observe both potential outcomes $y_{jt}(1)$ and $y_{jt}(0)$ for the same unit at the same time?

Problem 5.2. ATE vs. ATT

Define the Average Treatment Effect (ATE) and the Average Treatment Effect on the Treated (ATT). How do they differ conceptually and in terms of their estimation?

Problem 5.3. Causal Interpretation

In the potential outcomes framework, how would you explain the difference between saying "the policy caused an increase in y" and saying "y increased after the policy"?

5.2 Counterfactuals and the Role of Controls

Above, I introduced the idea of outcomes y_{jt}, but I did not mention where they came from. It turns out that being able to reason about this is central to understanding causal inference models more generally. A useful place to start is the idea of the linear factor model (Filipovic & Schneider, 2025).

Assumption 5.3. Linear Factor Model

$\forall j \in \mathcal{N}$, $t \in \mathcal{T}_1$, pre-treatment outcomes are generated like

$$y_{jt}(0) = \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \varepsilon_{jt},$$

where $\boldsymbol{\lambda}_t \in \mathbb{R}^F$ is a vector of *common factors* affecting all units, $\boldsymbol{\mu}_j \in \mathbb{R}^F$ are *unit-specific factor loadings*, and ε_{jt} are zero-mean *idiosyncratic errors*. Intuitively, $\boldsymbol{\lambda}_t$ captures time-varying shocks or trends shared across all units, while $\boldsymbol{\mu}_j$ determines how sensitive each unit is to those factors.

If you're anything like me, you're asking what the heck this means practically, so I'll do a small illustration. Suppose there are $F = 2$ factor loadings — one capturing geography and another capturing culture. For a treated unit and three donor units, this might might look like this:

Unit	Geography (μ_{j1})	Culture (μ_{j2})
Treated (1)	1.20	0.50
Donor A	1.05	0.40
Donor B	0.80	0.55
Donor C	0.60	0.70

Each column represents a *time-invariant latent component* that affects all periods. Units differ, however, in how much they are influenced by each factor (or, *load* on each factor). For example, the treated unit loads relatively heavily on Geography (1.20) but moderately on Culture (0.50), while Donor C loads more on Culture (0.70) than on Geography (0.60).

At each time period, the common factors $\boldsymbol{\lambda}_t$ capture *time-varying shocks* that influence all units simultaneously, such as national economic trends, policy changes, or global events. For example, over four periods:

Time t	Seasonal Factor (λ_{t1})	Cyclical Factor (λ_{t2})
t_1	1.0	0.1
t_2	0.5	0.3
t_3	-0.2	0.8
t_4	-0.8	0.4

Here, $\boldsymbol{\lambda}_t$ varies over time but is *common to all units*, which is why these are called "common factors." Multiplying the factor loadings $\boldsymbol{\mu}_j$ by the time-specific factors $\boldsymbol{\lambda}_t$ shows how each unit responds differently to the same underlying shocks, with ε_{jt} capturing unit-specific noise. **?@fig-toylfm** gives a slightly extended version of this toy example, plotted so we can see.

Linear factor models can capture many types of temporal dynamics: seasonal factors repeat periodically within a year, capturing patterns like quarterly sales cycles or flu seasons; cyclical

factors represent longer-term economic or social cycles; and periodic or oscillatory factors capture phenomena that recur with a known period, such as day-night cycles or weekly traffic patterns.

It is also possible to define grouped factor models (Liao et al., 2025), where units are organized into groups that share similar baseline structures. In this setting, the overall model can be thought of as a baseline linear factor model, but the details — such as factor loadings or even the number of active factors — may vary within each group. For example, different regions in a country might have a shared set of seasonal and cyclical factors, but the magnitude of their loadings on each factor differs by region. This flexibility allows linear factor models to accommodate complex multivariate data while remaining interpretable: the common factors capture shared variation, the factor loadings capture heterogeneity in responses, and the idiosyncratic errors capture unit-specific randomness.

Assumption 5.4. Affine Factor Combination

Suppose the treated unit's factor loadings lie in the affine span of the donor loadings:

$$\boldsymbol{\mu}_1 = \alpha \mathbf{1}_F + \mathbf{M}^\top \mathbf{w},$$

for some intercept $\alpha \in \mathbb{R}$ and weights $\mathbf{w} \in \mathbb{R}^{N_0}$, where $\mathbf{1}_F$ is a vector of ones of length F.

Oftentimes, we assume that the treated unit's factor loadings may be approximated by the loadings of the control group. It says that the treated unit's long-run structure — captured by its factor loadings $\boldsymbol{\mu}_1$ — can be expressed as a combination of the donor units' loadings. The scalar α accounts for overall level differences (an intercept or baseline effect), while $\mathbf{M}^\top \mathbf{w}$ represents a weighted blend of donor characteristics.

Much of the synthetic control literature can be interpreted through the lens of factor models and related data-generating processes. While this is not a book on econometric theory, sometimes I will algebraically manipulate the factor model to show how thinking about it carefully allows us to consider the problem of causal impact estimation in a more refined manner. The following result, Lemma 5.1, seeks to convey the utility of the factor model as a basis for making comparisons. This demonstration underlies the rest of the book, as it explains why, in principle, comparative case studies are possible.

Lemma 5.1. *Suppose the untreated outcomes follow the linear factor model. Denote the treated unit's factor loading as well as the matrix of donor factor loadings as*

$$\boldsymbol{\mu}_1^\top \in \mathbb{R}^{1\times F}, \qquad \mathbf{M} = \begin{bmatrix} \boldsymbol{\mu}_2^\top \\ \vdots \\ \boldsymbol{\mu}_{N_0+1}^\top \end{bmatrix} \in \mathbb{R}^{N_0 \times F} \tag{5.1}$$

Further, let the common factors be:

$$\boldsymbol{\lambda}_t \in \mathbb{R}^{F\times 1}, \qquad \boldsymbol{\Lambda} = \begin{bmatrix}\boldsymbol{\lambda}_1 & \boldsymbol{\lambda}_2 & \cdots & \boldsymbol{\lambda}_T\end{bmatrix} \in \mathbb{R}^{F\times T}. \tag{5.2}$$

Now, let the donor pool and target vector be

$$\mathbf{y}_1 = \begin{bmatrix} y_{11}(0) \\ \vdots \\ y_{1T}(0) \end{bmatrix} \in \mathbb{R}^{T\times 1}, \qquad \mathbf{Y}_0 = \begin{bmatrix} y_{21}(0) & \cdots & y_{2T}(0) \\ \vdots & \ddots & \vdots \\ y_{N_0+1,1}(0) & \cdots & y_{N_0+1,T}(0) \end{bmatrix}^\top \in \mathbb{R}^{T\times N_0}, \tag{5.3}$$

Let the affine combination assumption hold. Then the treated unit's outcomes can be expressed as a linear combination of the donor outcomes plus an intercept and a residual:

$$\mathbf{y}_1 = \alpha \mathbf{1}_T + \mathbf{Y}_0 \mathbf{w} + \boldsymbol{\varepsilon}_1^*, \quad \textit{where} \quad \boldsymbol{\varepsilon}_1^* = \boldsymbol{\varepsilon}_1 - \mathbf{E}_0 \mathbf{w},$$

with $\mathbf{y}_1 = (y_{11}(0), \ldots, y_{1T}(0))^\top \in \mathbb{R}^T$, $\boldsymbol{\varepsilon}_1 = (\varepsilon_{11}, \ldots, \varepsilon_{1T})^\top$, *and* $\mathbf{E}_0 \in \mathbb{R}^{T\times N_0}$ *the matrix of donor idiosyncratic errors.*

Corollary 5.1. *Notice that the common factors* $\boldsymbol{\lambda}_t$ *do not explicitly appear in the lemma. There are two reasons for this. First, the common factors are* unobserved*, so we cannot directly use or adjust for them. Second, by definition, common factors affect all units in the same way at each time* t*, so they do not help distinguish the treated unit from the donors. Any linear combination of donor outcomes automatically carries the same influence of the common factors, meaning that these shared shocks cancel out when forming a weighted approximation of the treated unit's outcomes. The variation we can exploit for comparison comes entirely from the unit-specific loadings* $\boldsymbol{\mu}_j$ *and the idiosyncratic errors* ε_{jt}*, which differ across units.*

The linear factor model provides a powerful lens for interpreting comparative case studies. In the potential outcomes framework, the challenge was that the counterfactual outcome $y_{1t}(0)$ is unobserved. The factor model tells us why it might nevertheless be *reconstructable* as an estimate: if units share a common latent structure — that is, if their outcomes depend on a set of unobserved but shared factors $\boldsymbol{\lambda}_t$ — then the treated unit's trajectory before intervention can be expressed as a combination of the donor trajectories.

If the treated unit's factor loadings $\boldsymbol{\mu}_1$ can be approximated by an affine combination of the donors' loadings, the donors already encode information about how the treated unit would have responded to each common factor. Consequently, the untreated potential outcome of the treated unit, $y_{1t}(0)$, can be reconstructed as a weighted combination of donor outcomes. This is the formal justification behind comparative case studies.

There is a subtlety worth dwelling on, because we will meet it again in difference-in-differences. The linear factor model and the affine loading assumption are stated over the pre-treatment periods $\mathcal{T}_1$, where we can see the treated unit's untreated outcomes and check how well a weighted blend of donors reproduces them. But Lemma 5.1 is used to fill in $y_{1t}(0)$ for $t \in \mathcal{T}_2$, the post-treatment periods, where the treated unit's untreated path is never observed. That

step quietly assumes the factor structure — and the treated unit's affine relationship to the donor loadings — carry over unchanged after the intervention. Nothing in the pre-treatment data can confirm this: a close pre-treatment fit is necessary but not sufficient. It is the synthetic control counterpart of the untestable half of the parallel trends assumption we develop in the next chapter — in both designs, the pre-period is where we check the model and the post-period is where we must extrapolate it on faith.

Problem 5.4. Linear Factor Model Interpretation

Consider the linear factor model:

$$y_{jt}(0) = \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \varepsilon_{jt}.$$

a) Explain what the vectors $\boldsymbol{\lambda}_t$ and $\boldsymbol{\mu}_j$ represent.

b) Give an example of a real-world latent factor that could influence outcomes across units.

Problem 5.5. Affine Factor Combination

Describe the affine combination assumption:

$$\boldsymbol{\mu}_1 = \alpha \mathbf{1}_F + \mathbf{M}^\top \mathbf{w}.$$

What does it imply about the relationship between the treated unit and the donor units?

Problem 5.6. Reconstruction of Treated Outcomes

Based on this Lemma, write the treated unit's outcome vector as a linear combination of donor outcomes plus an intercept and residual. Explain intuitively what each component represents.

5.3 What Is a Comparative Case Study?

Case studies in general are used to draw conclusions about a phenomenon of interest (say, economic development) by comparing some unit of interest to a set of others. In the classic text by Tocqueville et al. (2012) (a qualitative case study), the United States of America is compared to England, France, and elsewhere in Europe. Quantitative case studies use empirical data about the world to generate counterfactuals. Quantitative case studies approximate a counterfactual by drawing on the observable outcomes of other units that can stand in for the treated unit under alternative circumstances. The core idea here, as with Alexis de Tocqueville, is to find one or more units that resemble the treated unit as closely as possible prior to the intervention and to use their trajectories as a window into the outcome that might have occurred.

At the heart of the comparative case study framework is the importance of the similarity between the main unit of interest to the units we are comparing it to. Thus, it must begin with a clear identification of the treated unit and the intervention of interest, followed by a careful examination of pre-intervention outcomes and relevant background characteristics. Analysts then search for comparators, either single units or combinations thereof, that replicate the patterns of the treated unit as closely as possible before the intervention. Finally, the comparison of post-intervention trajectories provides an estimate of the intervention's effect, offering a way to reason about what would have happened under alternative circumstances. This framework is not a simple exercise in matching, but a thoughtful integration of domain knowledge, empirical observation, and careful judgment.

SCM builds on this by leveraging panel data (where multiple units (e.g., countries, regions, or markets) are observed over time) to compare a single treated unit against a set of untreated control units. This allows us to construct a more accurate counterfactual trajectory for what might have happened absent the intervention.

5.4 West Germany

To make this concrete, consider the study of German reunification by Abadie et al. (2015) which is one of the most influential applications of the framework. Studying the impact of political and economic reunification on West German GDP per capita, their goal here is to see how GDP would have evolved in the absence of reunification. Of course, they face the challenge that no alternate West Germany exists. To approach this problem, they examined the set of OECD countries whose economic indicators, trade patterns, industrial composition, and growth trends before 1990 closely resembled those of West Germany.

We can begin from the perspective of the linear factor model. Here, each country's trajectory of GDP per capita can be thought of as reflecting a mixture of latent "factors" (maybe economic, geographic, institutional, and cultural ones) that shape its evolution over time. The donor countries (Australia, Austria, Belgium, Denmark, France, Greece, Italy, Japan, Netherlands, New Zealand, Norway, Portugal, Spain, Switzerland, the United Kingdom, and the United States) differ in how they load on these underlying dimensions. For instance, France, Austria, and Belgium may be understood as having high geographic and historical loadings relative to West Germany: they share borders, trade patterns, and deep pre-war economic ties, and Austria in particular was once part of the German Empire itself. These countries likely move closely with Germany on factors such as industrial composition, continental integration, and exposure to European macroeconomic shocks. By contrast, countries like Australia, New Zealand, or the United States, though still members of the OECD, are geographically and institutionally distant. Their loadings on these shared European factors may be weaker or even orthogonal.

Of course, this discussion is purely theoretical. We never observe the factor loadings outside of synthetic study, nor do we know how many underlying loadings exist in reality. The factor model is simply a conceptual device that helps us think clearly about why some countries make better comparisons than others. The guiding idea in comparative case studies is therefore to select units that are as similar as possible to the treated unit along these

unobserved background dimensions — the economic, political, and historical forces that jointly shape their trajectories. When done thoughtfully, such comparisons help approximate the unobservable counterfactual world where the treatment did not occur.

To make this concrete, let's plot the outcome of interest. Consider **?@fig-west-germany-plot-one** which plots the GDP trajectory of West Germany versus the 16 control units. Simply by looking at the plot, which of these units are most likely to be similar to West Germany in the pre-intervention period? The answer unfortunately is not so obvious, as we can see that multiple units are close to the trajectory of West Germany before Reunification took place. The key thing to note here is that the pure empirical mean of OECD nations does not sufficiently resemble West Germany, in level or in trend. This means that the standard difference-in-differences estimator would be biased from parallel pre-intervention trends violations. Abadie et al. (2015) uses the SCM to estimate the impact of the reunification, wherein only some of these OECD countries are assigned weight as a comparison for West Germany.

5.5 Mexican Drug War

Next, we can consider the setting of the Mexican Drug War. In 2008, the Mexican government invaded several states using the military to combat against the drug traffickers. We can view each state's homicide trajectory as arising from a set of latent factors — geographic, economic, and strategic — that influence violence over time. Chihuahua, for instance, exhibits consistently high homicide rates relative to many other states. From the factor model perspective, this pattern can be interpreted as Chihuahua having high loadings on factors related to border proximity, drug trafficking routes, and urban centers such as Ciudad Juárez. These factors capture the persistent structural and strategic drivers of violence, rather than transient shocks or random fluctuations.

Other states, particularly those farther from international borders or major trafficking corridors, load less heavily on these factors, which explains why their quarterly homicide rates are generally lower or less volatile. The empirical mean of other states — plotted in blue in **?@fig-mexhomdata** — can be thought of as a rough, unweighted average of these other latent trajectories. Chihuahua's consistently higher trajectory highlights the importance of underlying structural differences in shaping outcomes.

As with our discussion of West Germany, this is a conceptual exercise. We do not observe these latent factors or the exact loadings, and we do not know how many factors truly govern violence dynamics. The point of the linear factor model is to provide a lens through which we can interpret observed heterogeneity: units that share certain characteristics — geography, strategic importance, socio-economic conditions — are likely to move together on some latent dimensions, while units with different characteristics load differently. Comparative case studies aim to leverage this structure, ideally selecting control units whose latent trajectories resemble those of the treated unit prior to intervention. In Chihuahua's case, this might mean comparing it to other northern border states or urban hubs with similar strategic conditions, rather than to all Mexican states indiscriminately.

5.6 Marketing Measurement

A similar logic underlies how marketers evaluate campaigns in the absence of clean experiments. The challenges faced in marketing contexts mirror those encountered in policy evaluation, though the scale and timing can differ. When a retailer launches a new advertising campaign, managers are immediately confronted with the question of causality: are observed increases in sales truly the result of the campaign, or would sales have risen regardless due to underlying trends in demand, seasonality, or other market forces?

From a theoretical perspective, each region or store can be thought of as having latent factors that influence sales — for instance, local demographics, store size, historical demand patterns, or competitive intensity. Some regions may load heavily on these factors, while others less so. To understand the campaign's effect, marketers often rely on holdout studies, sometimes referred to as geolift experiments. In a geolift study, a subset of regions or customer segments is deliberately excluded from the campaign and treated as a control group, while the remaining regions receive the intervention (for example, an increase in media spend compared to the business-as-usual scenario).

The goal is to see the effect of media spending on a given metric (called "conversions" in marketing jargon). If treated regions show higher conversions than holdout regions, this suggests the campaign caused the lift. Importantly, this comparison relies on the assumption that the latent factors governing sales in the treated regions are reasonably approximated by those in the holdout regions — that is, that the untreated units provide a plausible counterfactual trajectory for what would have happened absent the intervention. Random fluctuations, seasonal patterns, and other external shocks complicate this comparison, just as idiosyncratic noise affects observed outcomes in other contexts.

In short, the principle is the same as in policy evaluation or the study of violence in Chihuahua: differences in outcomes are interpreted through the lens of underlying latent factors. The better the holdout units resemble the treated units on these unobserved dimensions, the more credible the causal inference. Selecting control regions therefore requires careful judgment, informed by both past data and domain knowledge, rather than a purely mechanical choice.

5.7 Beyond Simple Control Selection

Even when the treated unit and a plausible set of comparison units are clearly identified, important design questions remain for researchers conducting quantitative case studies. Selecting which units to include is only part of the challenge; deciding how many units to use, and how to combine them, is equally consequential. Using too few units risks anchoring a counterfactual on idiosyncratic examples that happen to look similar by chance (such as, units that are only incidentally similar where other comparison units would be better). Using too many comparison units may blur distinctions and dilute meaningful resemblance between the treated unit and the comparison units.

A similar tension arises when analysts incorporate additional information beyond the main outcome of interest, such as auxiliary covariates that capture relevant background character-

istics. For example, if we are the data science unit of a hotel chain, we may wish to include temperature in our analyses because it can be a proxy for seasonality. Including covariates can improve comparability, but they also introduce new decisions: which variables matter most (and what would it mean in principle for them to matter most?). Furthermore, how many auxiliary covariates should we use? In policy analysis, for instance, should unemployment, education, and industrial structure all be treated as equally important predictors of economic growth? In marketing, should store size, regional demographics, and past sales volatility all receive comparable emphasis when identifying holdout regions? Each of these choices shapes the resulting counterfactual.

The difficulties do not stop there. A core assumption of the comparative case framework is that the treated and comparison units evolve independently—that there are no spillovers or cross-unit effects. Yet in many real settings, this assumption is fragile. Policy interventions can influence neighboring jurisdictions through migration, trade, or imitation. Marketing campaigns may spill over across geographic boundaries as customers travel, communicate, or respond to national media. When such spillovers occur, the comparison units no longer represent what would have happened without the intervention; they themselves are partly treated. Recognizing, diagnosing, and mitigating these influences is one of the most subtle and important tasks in applied comparative analysis.

5.8 Looking Ahead

Across policy evaluation, historical case studies, and marketing measurement, the central challenge is the same: we can observe only one version of history for any given unit, yet we wish to reason about what would have happened under alternative circumstances. Comparative case studies provide a structured way to approximate these unobserved counterfactuals by leveraging other units as stand-ins, under the assumption that these units share relevant background structure with the treated unit.

The linear factor model offers a conceptual lens for understanding why such approximations can work. In West Germany, latent factors may represent geography, industrial structure, and historical ties; France, Austria, and Belgium load heavily on these factors, making them plausible comparators. In Chihuahua, latent factors capture geographic proximity, trafficking routes, and urban centers like Juárez; northern border states or other strategic hubs are likely to share these underlying influences. In marketing, latent factors might include demographics, store characteristics, or historical demand patterns; holdout regions with similar factor loadings provide credible counterfactuals for the treated regions.

In all three cases, we never observe the factors or their loadings directly. The power of the factor model lies not in seeing the unobservables but in structuring our thinking: it identifies the dimensions along which units must be similar for comparisons to be meaningful, and clarifies why some units are better matches than others. Comparative case studies are, at their heart, an exercise in reasoning about these latent structures, selecting controls, and incorporating statistical adjustments that best approximate the unobserved counterfactual world.

Finally, this perspective underscores a core principle: constructing credible counterfactuals requires judgment, domain knowledge, and careful consideration of context. Whether analyzing historical events, criminal violence, or advertising campaigns, the goal is to approximate what would have happened in another version of the world — a world we cannot see but can approach through thoughtful comparison and careful use of observable data. This is the essence of causal inference, and it is the guiding logic behind every comparative case study we encounter. This idea will become more formalized in later chapters, as we work with more practical examples.

Problem 5.7. Comparative Case Study Concept

What is the core idea of a comparative case study? How does it relate to the potential outcomes framework?

Problem 5.8. Selecting Comparison Units

In designing a comparative case study, what factors make some units more appropriate comparators than others? Give at least two examples.

Problem 5.9. Latent Factors in Real-World Examples

Using the Chihuahua example, explain how latent factors (e.g., geography, drug trafficking routes) influence the homicide trajectory. Why does understanding these latent factors matter for constructing a counterfactual?

Problem 5.10. Marketing Application

Explain how the logic of potential outcomes and latent factors applies in a marketing context. How can holdout regions be used to estimate the effect of a new advertising campaign?

Problem 5.11. Bonus Question: Evaluating a Marketing Campaign

Imagine you are evaluating a marketing campaign across multiple stores, some treated and some held out as controls.

a) Explain how the linear factor model could help you justify which stores to use as controls.

b) Discuss potential challenges if treated stores are geographically clustered and subject to correlated shocks not present in the control stores. How would this affect your ATT estimates?

Proof. We start from the treated unit's factor model in vector form:

$$\mathbf{y}_1 = \mathbf{\Lambda}\boldsymbol{\mu}_1 + \boldsymbol{\varepsilon}_1.$$

Substituting the affine combination $\boldsymbol{\mu}_1 = \alpha \mathbf{1}_F + \mathbf{M}^\top \mathbf{w}$ gives

$$\mathbf{y}_1 = \boldsymbol{\Lambda}(\alpha \mathbf{1}_F + \mathbf{M}^\top \mathbf{w}) + \boldsymbol{\varepsilon}_1.$$

Using the distributive property of matrix multiplication:

$$\mathbf{y}_1 = \alpha \boldsymbol{\Lambda} \mathbf{1}_F + \boldsymbol{\Lambda} \mathbf{M}^\top \mathbf{w} + \boldsymbol{\varepsilon}_1.$$

Define $\mathbf{1}_T := \boldsymbol{\Lambda} \mathbf{1}_F \in \mathbb{R}^T$, so that the first term becomes $\alpha \mathbf{1}_T$. For the second term, note that

$$\boldsymbol{\Lambda} \mathbf{M}^\top \mathbf{w} = (\mathbf{M} \boldsymbol{\Lambda})^\top \mathbf{w}.$$

By definition,

$$\mathbf{Y}_0 = (\mathbf{M} \boldsymbol{\Lambda})^\top + \mathbf{E}_0,$$

which implies $(\mathbf{M} \boldsymbol{\Lambda})^\top = \mathbf{Y}_0 - \mathbf{E}_0$. Substituting gives

$$(\mathbf{M} \boldsymbol{\Lambda})^\top \mathbf{w} = \mathbf{Y}_0 \mathbf{w} - \mathbf{E}_0 \mathbf{w}.$$

Combining all terms:

$$\mathbf{y}_1 = \alpha \mathbf{1}_T + \mathbf{Y}_0 \mathbf{w} + (\boldsymbol{\varepsilon}_1 - \mathbf{E}_0 \mathbf{w}).$$

Defining the combined residual $\boldsymbol{\varepsilon}_1^* := \boldsymbol{\varepsilon}_1 - \mathbf{E}_0 \mathbf{w}$, we have

$$\mathbf{y}_1 = \alpha \mathbf{1}_T + \mathbf{Y}_0 \mathbf{w} + \boldsymbol{\varepsilon}_1^*.$$

This shows that the treated unit's outcomes are a linear combination of the donor outcomes plus an intercept and a modified residual. ■ □

Chapter 6

Using The Control Group Mean

Now that we have spoken a little about averaging as an idea, we can finally formalize this in a panel data setting such that it is useful. Before, I was sort of vague about using averages for comparison; I gave a plot, but I never said how we would more formally do this. This chapter explains how we would do this as a first pass.

6.1 Two Way Fixed Effects

Recall the linear factor model as a data generating process. The original model assumes that some number of factor loadings and common time factors interact (along with covariates, potentially) to generate outcomes. However, this can be simplified by a lot to what we traditionally know as the two-way fixed effects model.

Assumption 6.1. Two-Way Fixed Effects Model

$\forall j \in \mathcal{N}, : t \in \mathcal{T}_1$, pre-treatment outcomes are generated according to

$$y_{jt}(0) = \mu_j + \lambda_t + \varepsilon_{jt},$$

where μ_j are *unit-specific intercepts* capturing time-invariant heterogeneity, λ_t are *time-specific effects* capturing shocks or trends common to all units, and ε_{jt} are zero-mean *idiosyncratic errors*. This model is a special case of the linear factor model, where there is a single factor with identical loadings across all units:

$$\boldsymbol{\lambda}_t = [1, \lambda_t]^\top, \quad \boldsymbol{\mu}_j = [\mu_j, 1]^\top.$$

In this representation, the first component captures unit-level heterogeneity, while the second captures the shared temporal component.

Intuitively, the TWFE model assumes that all units respond *equally* to the same common time trend λ_t, differing only in their fixed intercepts μ_j. This simplicity makes it powerful for

many policy applications but also restrictive — if units react differently to aggregate shocks (i.e. have heterogeneous loadings on λ_t), the model is misspecified, and DID-type estimators may become biased. In that sense, the LFM generalizes TWFE by allowing these sensitivities to vary across units. **?@fig-dag-twfe** represents this DGP graphically.

To make this concrete, consider a toy example with two units over two time periods. Consider two units ($j = 1, 2$) observed over two time periods ($t = 1, 2$). Suppose

$$y_{jt} = \mu_j + \lambda_t + \varepsilon_{jt}$$

holds. Let the fixed effects be

$$\mu = \begin{bmatrix} 2 \\ 5 \end{bmatrix}, \qquad \lambda = \begin{bmatrix} 1 \\ 3 \end{bmatrix}.$$

Ignoring the random errors ε_{jt}, we can represent the predicted outcomes as a matrix where each row corresponds to a time period and each column to a unit:

$$M = \begin{bmatrix} y_{11} & y_{21} \\ y_{12} & y_{22} \end{bmatrix} = \begin{bmatrix} \mu_1 + \lambda_1 & \mu_2 + \lambda_1 \\ \mu_1 + \lambda_2 & \mu_2 + \lambda_2 \end{bmatrix} = \begin{bmatrix} 2+1 & 5+1 \\ 2+3 & 5+3 \end{bmatrix} = \begin{bmatrix} 3 & 6 \\ 5 & 8 \end{bmatrix}.$$

In this example, moving from the first to the second row adds the same amount, +2, to each unit because λ_t changes by the same amount for everyone.

Problem 6.1. Additive Factor Model

Explain in your own words what the additive factor model

$$y_{jt} = \mu_j + \lambda_t + \varepsilon_{jt}$$

represents. What do the terms μ_j and λ_t capture in practical terms?

Problem 6.2. Unit Effects vs. Time Effects

Give an example of a real-world scenario where two units might share λ_t but differ in μ_j. How would this difference affect naive comparisons of levels?

6.2 Demeaning and Difference-in-Differences

A simple way to approximate the counterfactual outcome for a treated unit is to use the mean of the control group. That is, we could take the average outcome of untreated units and use it as an estimate of what the treated unit would have experienced without treatment. However, using the mean alone implicitly assumes that all units start from the same level—that

is, that their μ_j are identical. In practice, this assumption is rarely justified. Units may differ in income, size, resources, or other characteristics long before any intervention occurs. Ignoring these baseline differences can be misleading: post-treatment differences might simply reflect where units began, not the causal effect of the treatment. This is the basic intuition behind DID. To formalize this idea, we can difference out these time-invariant differences via *demeaning*.

Lemma 6.1 (Demeaning)**.** *Let the observed outcome for unit j at time t in the pre-treatment periods $\mathcal{T}_1$ follow the additive two-way fixed effects model*

$$y_{jt} = \mu_j + \lambda_t + \varepsilon_{jt}, \qquad t \in \mathcal{T}_1,$$

where μ_j is a unit-specific baseline, λ_t a common time effect, and ε_{jt} an idiosyncratic error with finite mean. Define the pre-period demeaned outcome

$$r_{jt} := y_{jt} - \frac{1}{|\mathcal{T}_1|} \sum_{s \in \mathcal{T}_1} y_{js}.$$

Then for every j and $t \in \mathcal{T}_1$,

$$r_{jt} = (\lambda_t - \bar{\lambda}) + (\varepsilon_{jt} - \bar{\varepsilon}_j),$$

where $\bar{\lambda} := \frac{1}{|\mathcal{T}_1|} \sum_{s \in \mathcal{T}_1} \lambda_s$ and $\bar{\varepsilon}_j := \frac{1}{|\mathcal{T}_1|} \sum_{s \in \mathcal{T}_1} \varepsilon_{js}$. In particular, the unit baseline μ_j cancels exactly, so r_{jt} depends only on deviations of time effects and idiosyncratic errors from their pre-period averages.

Demeaning is a fundamental idea in econometrics; see, for example, the review by Brabander et al. (2025). Rather than comparing treated and control units at a single moment based on current levels, we examine how their outcomes *evolve* across time. Specifically, we compare the *change* in the treated unit before and after treatment to the *change* observed in untreated units over the same period. By focusing on relative changes, we adjust for baseline differences. The crucial insight is this: what matters is not that two units start at the same level, but that their *pre-treatment trends move similarly*. If trajectories are parallel before treatment, any divergence afterward can be attributed to the treatment itself. This shift from levels to trends is precisely what the Difference-in-Differences estimator formalizes.

Problem 6.3. Baseline Differences

Why is it important to adjust for baseline differences between treated and control units when estimating treatment effects? How does DID address this?

6.3 Counterfactuals Under DID

Suppose we have a setting where a group of units is treated all at once, while others are never treated. The simplest form of the DID estimator can be obtained by minimizing the distance between the treated unit's outcomes and the average of untreated units, up to an additive

constant β that adjusts for baseline differences. The counterfactual for the DID method is simply the mean of the control group plus some constant.

Definition 6.1 (DID Counterfactual)**.** The post-treatment counterfactual for the treated unit under DID can be expressed as

$$\hat{y}_{1t}(0) = \bar{y}_{0t} + \beta^*, \quad t > T_0,$$

where

$$\bar{y}_{0t} \coloneqq \frac{1}{N_0} \sum_{j \in \mathcal{N}_0} y_{jt}.$$

Here, $N_0 = |\mathcal{N}_0|$ is the number of untreated units, so that each control unit receives an equal weight of N_0^{-1}. β is a scalar which is the mean difference between the treated and control group in the pre-treatment period.

A natural extension of this is the idea of DID as a centering estimator.

Lemma 6.2 (Centered DID)**.** *For any fixed donor pool $\mathcal{N}_0$, the DID estimator for a single treated unit has a closed-form solution*

$$\beta^* = \frac{1}{T_0} \sum_{t=1}^{T_0} \Big(y_{1t} - \bar{y}_{\mathcal{N}_0 t} \Big), \qquad \bar{y}_{\mathcal{N}_0 t} = \frac{1}{|\mathcal{N}_0|} \sum_{j \in \mathcal{N}_0} y_{jt}.$$

The residual vector after accounting for this optimal intercept is

$$\mathbf{r} = \mathbf{y}_c - \bar{\mathbf{y}}_{c,\mathcal{N}_0}, \qquad \mathbf{y}_c = \mathbf{y}_1 - \bar{y}_1 \mathbf{1}_{T_0}, \qquad \bar{\mathbf{y}}_{c,\mathcal{N}_0} = \bar{\mathbf{y}}_{\mathcal{N}_0} - \bar{m}_{\mathcal{N}_0} \mathbf{1}_{T_0}.$$

Consequently, the pre-treatment sum of squared residuals is

$$\ell_{DID}(\mathcal{N}_0) = \|\mathbf{y}_c - \bar{\mathbf{y}}_{c,\mathcal{N}_0}\|_2^2.$$

Even if the treated and control units have different levels, the demeaning removes these level differences. DID relies on the Parallel Trends Assumption (PTA), which states that, in the absence of treatment, the expected difference between the treated unit and the control group would remain constant over time — across *every* period, pre-treatment and post-treatment alike:

Definition 6.2.

Definition 6.1. Parallel Trends Assumption

In the absence of treatment,

$$\mathbb{E}[y_{1t}(0) - \bar{y}_{0t}(0)] = \mathbb{E}[y_{1t'}(0) - \bar{y}_{0t'}(0)], \qquad \forall\, t, t' \in \mathcal{T}_1 \cup \mathcal{T}_2.$$

The range of the quantifier is the whole point. The equality must hold for all periods, including the post-treatment periods $\mathcal{T}_2$, where the treated unit's untreated outcome $y_{1t}(0)$ is precisely the counterfactual we are trying to recover. This is what makes DID identify a causal effect: it is the parallel movement of $y_{1t}(0)$ and $\bar{y}_{0t}(0)$ *after* treatment that lets us read post-treatment divergence as the treatment's doing. It is also why the assumption is fundamentally untestable — the post-treatment $y_{1t}(0)$ is never observed, so we can never check the very periods that do the identifying work.

It is worth being clear about how this relates to the additive two-way fixed effects model. If every unit shares the same time effect λ_t, their untreated outcomes move in lockstep and parallel trends holds automatically. But the additive model is stronger than DID actually needs: identification requires only that trends be parallel *on average*, in the sense of the expectation above, not that outcomes follow a literal additive structure with well-behaved idiosyncratic errors. Parallel trends is the primitive assumption; the two-way fixed effects model is merely a convenient sufficient condition for it.

Because the Parallel Trends Assumption cannot be checked, econometricians fall back on the pre-treatment analogue, which we *can* inspect:

Definition 6.3.

Definition 6.2. Parallel Pre-Trends Assumption

$$\mathbb{E}\Big[y_{1t}(0) - N_0^{-1} \sum_{j \in \mathcal{N}_0} y_{jt}(0)\Big] = \beta, \quad \forall\, t \in \mathcal{T}_1.$$

The two assumptions differ only in their range: the Parallel Trends Assumption quantifies over all periods $\mathcal{T}_1 \cup \mathcal{T}_2$, whereas the Parallel Pre-Trends Assumption restricts attention to the pre-treatment periods $\mathcal{T}_1$. Parallel pre-trends is therefore a *testable implication* of parallel trends, not the identifying assumption itself — it is necessary but not sufficient. Li & Van den Bulte (2023) put the split cleanly: parallel trends divides into a testable part, that the assumption at least holds in the pretreatment period, and a nontestable part, that the same relationship between treated and control units carries into the posttreatment period, when the counterfactual is unobserved. Pre-treatment differences can look perfectly constant and yet the trends may still diverge after treatment for reasons that have nothing to do with the intervention. When the pre-trend condition holds, and assuming additionally no anticipation and SUTVA (see Baker et al. (2025) for more), we generally treat the DID estimator as unbiased. But this remains a judgment about the unobserved post-period, extrapolated from the pre-period, rather than something the data can ever confirm.

Problem 6.4. Parallel Trends Assumption

Define the parallel trends assumption formally. How can you assess whether this assumption is plausible in pre-treatment data? You may integrate ideas from the recent staggered DID literature, if you wish.

Problem 6.5. Counterfactual Construction

Explain how the DID estimator, as discussed above, constructs the counterfactual for a treated unit. How does using the mean of control units approximate the unobserved λ_t?

6.4 Parallel Trends in Small N Settings

DID is often *applied* in settings with many units and few time periods (Baker et al., 2025), but the identification argument itself asks only for at least one treated and one control group observed before and after treatment — the "large N" setting is a common use case, not a requirement. In cases of small N, the parallel pre-trends assumption can be less likely to hold. To illustrate, consider the canonical paper on SCM by Abadie et al. (2010), which studies the impact of California's Anti-Tobacco Campaign, Proposition 99, on cigarette sales per capita. Enacted in 1989, Prop 99 attempted to reduce smoking via a statewide public health campaign. In the original dataset, the authors had 38 control states to work with from 1970 to 2000. Using the estimator described above, we will use Python to estimate the DID design for the effect of Prop 99 on tobacco consumption. To follow along, you'll need the `dataprep` helper from my mlsynth code, which may be installed like

```
pip install -U git+https://github.com/jgreathouse9/mlsynth.git
```

In the code below, we load the `dataprep` function alongside `numpy` and `pandas`, as well as the Prop 99 dataset.

We then implement the DID estimator as described above, using `numpy` in this case.

The pre-treatment mean difference ≈ -14.359. So, this is what we subtract the donor mean vector entries by. When we do this, we get **?@fig-did-cali**.

We can see here that the scalar-adjusted control group average does not appear parallel to California. The counterfactual is dissimilar to California for the first five years of the dataset in that it undershoots California's values, and for the rest of the pre-treatment period, it has higher per capita smoking than California. Why might this be? A tempting answer is sampling noise: with only a handful of controls the donor mean $\bar{y}_{0t}$ is a volatile average. That is part of the story, but it is not the part that governs whether parallel trends holds. To see the difference, return to the linear factor model for untreated outcomes and ask what the DID counterfactual actually leaves behind.

Lemma 6.3. *Suppose untreated outcomes follow the linear factor model*

$$y_{jt}(0) = a_j + \mathbf{b}_j^\top \mathbf{f}_t + u_{jt}, \qquad \mathbb{E}[u_{jt}] = 0,$$

with common factors $\mathbf{f}_t$ and unit-specific loadings $\mathbf{b}_j$. Let $\bar{\mathbf{b}}_0 = N_0^{-1} \sum_{j \in \mathcal{N}_0} \mathbf{b}_j$ be the average loading of the donor pool and $\bar{\mathbf{f}} = T_0^{-1} \sum_{t \in \mathcal{T}_1} \mathbf{f}_t$ the pre-period average of the factors. Then the DID counterfactual $\hat{y}_{1t}(0) = \bar{y}_{0t} + \beta^*$ has expected error

$$\mathbb{E}\big[y_{1t}(0) - \hat{y}_{1t}(0)\big] = (\mathbf{b}_1 - \bar{\mathbf{b}}_0)^\top (\mathbf{f}_t - \bar{\mathbf{f}}).$$

Consequently the Parallel Trends Assumption holds — and DID is unbiased for the ATT — if and only if the treated unit's loading equals the donor pool's average loading, $\mathbf{b}_1 = \bar{\mathbf{b}}_0$ *(or the factors do not move over the window).*

The two-way fixed effects model of the previous section is the special case in which every unit shares the same loading, so $\mathbf{b}_1 = \bar{\mathbf{b}}_0$ holds automatically and DID is exact. The lemma says DID needs much less than that — only that the treated unit's exposure to the common factors match the donor pool's *average* exposure — but it does need that, and equal weighting is what pins the requirement to the average.

The expected error isolates the failure mode. The systematic term $(\mathbf{b}_1 - \bar{\mathbf{b}}_0)^\top(\mathbf{f}_t - \bar{\mathbf{f}})$ is a *bias*: it is nonzero exactly when the treated unit loads on the common factors differently from the average donor, and it does not shrink as the panel lengthens. No quantity of pre-treatment data repairs a donor pool whose average factor exposure differs from the treated unit's — this is the violation of parallel trends. Separately, the idiosyncratic term $u_{1t} - \bar{u}_{0t}$ that the expectation averages away is a *variance*: pooling few noisy controls is unstable, and this piece does shrink with the donor count,

$$\mathrm{Var}\Big(\frac{1}{N_0}\sum_{j\in\mathcal{N}_0} u_{jt}\Big) = \frac{1}{N_0^2}\sum_{j\in\mathcal{N}_0}\mathrm{Var}(u_{jt}).$$

The small-N_0 concern, then, is really about this second, variance term; parallel trends itself is a statement about the first.

Li & Van den Bulte (2023) make this bias–variance split precise for exactly this class of estimators. Writing the mean squared error as $\mathrm{MSE}(\hat{\Delta}) = \mathrm{Bias}^2(\hat{\Delta}) + \mathrm{Var}(\hat{\Delta})$, they show through a factor-model study that when treated and control units share the same loadings — the homogeneous case $\mathbf{b}_1 = \bar{\mathbf{b}}_0$ — DID's bias is negligible and it is in fact the *most efficient* of the standard estimators: imposing equal weights that sum to one is not a handicap but a variance-reducing restriction, and superfluous flexibility comes at a cost. When the treated unit's loading instead falls outside, or merely differs from, the donor pool's, DID's error becomes bias-dominated and its MSE the largest among the methods they compare. The Proposition 99 picture is this second regime: California's smoking trajectory does not load on the underlying factors the way the average donor state does, so equal weighting cannot align the two, and the gap in **?@fig-did-cali** is systematic rather than noise.

This becomes even clearer when we experiment with different control units, since changing the donor pool changes its average loading $\bar{\mathbf{b}}_0$ and hence the systematic term in Lemma 6.3. Here, we compare California to Kentucky, NC, and both jointly, using a DID model for each state individually as well as jointly. These results are presented in **?@fig-did-calikent**. Kentucky's cigarette consumption loads on the common factors quite differently from California—its high baseline smoking rates and tobacco-based economy give it a factor exposure far from California's—so $\mathbf{b}_1 - \bar{\mathbf{b}}_0$ is large and the counterfactual tilts systematically. North Carolina differs from California in the same structural way. Demeaning removes the level gap $a_1 - \bar{a}_0$, but it cannot remove a difference in loadings; that mismatch is precisely what survives the intercept adjustment.

Practically, these violations are evident when we compare the ATT estimates. Using all

controls, the naive DID yields an ATT of roughly −27. Using Kentucky alone yields −29, NC alone yields −34, and the Kentucky + NC combination gives −32.36. That the estimate swings this much with the choice of donor is the bias term at work: each pool has a different average loading, so each implies a different systematic gap, and with few controls no single pool sits close to California's factor exposure. This is a concrete illustration of why parallel trends may fail in real-world, small-N applications like Proposition 99 — and, per Li & Van den Bulte (2023), why more flexible weighting schemes than DID's equal weights can be worth the added variance when the donor pool is heterogeneous.

It is important to emphasize that this is not a fault of DID per se. The DID method performs as intended when its key assumptions hold: SUTVA (no interference between units) and parallel trends. In the chapters that follow, we will explore other forms of looking at this problem, which address these issues by constructing a weighted combination of controls to better match the treated unit's pre-treatment trajectory, mitigating the impact of non-parallel trends and small N challenges.

Problem 6.6. Small N and Pre-Treatment Variation

In settings with a small number of control units, the pre-treatment average of the controls can fluctuate due to idiosyncratic shocks in individual units.

a) Explain why this makes the pre-treatment trajectory of the donor mean potentially unstable.

b) How does this instability affect the accuracy of the DID counterfactual for the treated unit?

c) What role does the number of control units (N) play in mitigating or exacerbating this problem?

Problem 6.7. Observed vs. Counterfactual Trajectories

Suppose California's pre-treatment trajectory is steadily decreasing while Kentucky's is flat. Sketch and describe what the DID counterfactual using Kentucky would look like. Why might this estimate be biased?

Problem 6.8. Multiple Donors

Conceptually, why might using multiple control units (e.g., Kentucky + NC) improve the counterfactual estimate? Under what circumstances might it still fail?

Problem 6.9. Interpretation of ATT

If a DID analysis gives an ATT of –32, what does this mean in terms of California's cigarette consumption post–Proposition 99?

Problem 6.10. Limitations of DID

Discuss at least two situations in which DID may give biased estimates, even if implemented correctly. Relate your answer to the additive factor model.

Problem 6.11. Arithmetic Mean as a DID Counterfactual

In previous chapters, we defined the arithmetic average as a weighted sum where each member's weight is proportional to its size in the sequence. In the context of using the arithmetic mean of control units to construct a DID counterfactual:

1. Explain what assumptions are being made implicitly about the control units when we take a simple arithmetic mean. Consider both their contribution to the counterfactual and their relative similarity to the treated unit.
2. Discuss potential limitations of this assumption. Under what circumstances might the arithmetic mean of controls misrepresent the "true" counterfactual trajectory?

Proof. By definition,

$$r_{jt} = y_{jt} - \frac{1}{|\mathcal{T}_1|} \sum_{s \in \mathcal{T}_1} y_{js} = (\mu_j + \lambda_t + \varepsilon_{jt}) - \frac{1}{|\mathcal{T}_1|} \sum_{s \in \mathcal{T}_1} (\mu_j + \lambda_s + \varepsilon_{js}).$$

Because summation is linear,

$$\frac{1}{|\mathcal{T}_1|} \sum_{s \in \mathcal{T}_1} (\mu_j + \lambda_s + \varepsilon_{js}) = \mu_j + \bar{\lambda} + \bar{\varepsilon}_j.$$

Subtracting this average from y_{jt} gives

$$r_{jt} = (\mu_j + \lambda_t + \varepsilon_{jt}) - (\mu_j + \bar{\lambda} + \bar{\varepsilon}_j) = (\lambda_t - \bar{\lambda}) + (\varepsilon_{jt} - \bar{\varepsilon}_j),$$

which proves the claim. □

Proof. Expanding the squared Euclidean norm gives

$$f(\beta) = \sum_{t=1}^{T_0} \Big(y_{1t} - \bar{y}_{0t} - \beta \Big)^2.$$

For clarity, define the inner function

$$u_t(\beta) = y_{1t} - \bar{y}_{0t} - \beta,$$

so that

$$f(\beta) = \sum_{t=1}^{T_0} u_t(\beta)^2.$$

To differentiate $u_t(\beta)^2$ with respect to β, we apply the chain rule. Here, the outer function is $g(u) = u^2$ with derivative $g'(u) = 2u$, and the inner function is $h(\beta) = u_t(\beta) = y_{1t} - \bar{y}_{0t} - \beta$ with

$$\frac{\mathrm{d}h}{\mathrm{d}\beta} = -1,$$

since y_{1t} and $\bar{y}_{0t}$ are constants with respect to β. Applying the chain rule, we find the derivative of each term:

$$\frac{\mathrm{d}}{\mathrm{d}\beta} u_t(\beta)^2 = 2u_t(\beta) \cdot (-1) = -2\Big(y_{1t} - \bar{y}_{0t} - \beta\Big).$$

Summing over all T_0 pre-treatment periods, by linearity of the derivative, gives

$$\frac{\mathrm{d}f}{\mathrm{d}\beta} = -2 \sum_{t=1}^{T_0} \Big(y_{1t} - \bar{y}_{0t} - \beta\Big).$$

Setting this derivative equal to zero, which is our first-order condition (FOC), we have

$$-2 \sum_{t=1}^{T_0} \Big(y_{1t} - \bar{y}_{0t} - \beta\Big) = 0.$$

Dividing both sides by -2 yields

$$\sum_{t=1}^{T_0} \Big(y_{1t} - \bar{y}_{0t} - \beta\Big) = 0.$$

Distributing the summation and noting that β is constant across t:

$$\sum_{t=1}^{T_0} (y_{1t} - \bar{y}_{0t}) - \sum_{t=1}^{T_0} \beta = \sum_{t=1}^{T_0} (y_{1t} - \bar{y}_{0t}) - T_0\beta = 0.$$

Now we solve for beta. To do this we add $T_0\beta$ to the RHS. That gives us this result:

$$\sum_{t=1}^{T_0} (y_{1t} - \bar{y}_{0t}) = T_0\beta$$

Now to isolate beta, we divide both sides by T_0, noting that division by a scalar T_0 is the same as multiplying by the reciprocal of that scalar:

$$\beta^* = \frac{1}{T_0} \sum_{t=1}^{T_0} (y_{1t} - \bar{y}_{0t}).$$

This shows that β^* is simply the average pre-treatment difference between the treated unit and the control group. Under parallel trends, this difference would hold in expectation, even absent treatment. Finally, the post-treatment counterfactual for the treated unit is simply the control group mean adjusted by this optimal β^*:

$$\hat{y}_{1t}(0) = \bar{y}_{0t} + \beta^*, \quad \forall\, t \in \mathcal{T}_2$$

□

Proof. For any fixed donor pool $\mathcal{N}_0$, we know from this proof

$$\beta^* = \frac{1}{T_0}\sum_{t=1}^{T_0}\Big(y_{1t} - \bar{y}_{\mathcal{N}_0,t}\Big) \qquad \text{where} \qquad \bar{y}_{\mathcal{N}_0,t} = \frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt}.$$

Here is the residual:

$$r_t = y_{1t} - \frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt} - \beta^* \qquad \text{where} \qquad \beta^* = \frac{1}{T_0}\sum_{t=1}^{T_0}\left(y_{1t} - \frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt}\right).$$

Substitute β^* directly into the residual:

$$\begin{aligned} r_t &= y_{1t} - \frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt} - \frac{1}{T_0}\sum_{t=1}^{T_0}\left(y_{1t} - \frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt}\right) \\ &= y_{1t} - \frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt} - \frac{1}{T_0}\sum_{t=1}^{T_0} y_{1t} + \frac{1}{T_0}\sum_{t=1}^{T_0}\left(\frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt}\right). \end{aligned}$$

Now recognize the averages:

$$\frac{1}{T_0}\sum_{t=1}^{T_0} y_{1t} = \bar{y}_1$$

and

$$\frac{1}{T_0}\sum_{t=1}^{T_0}\left(\frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt}\right) = \frac{1}{T_0}\sum_{t=1}^{T_0}\bar{y}_{\mathcal{N}_0,t} = \bar{m}_{\mathcal{N}_0}.$$

So

$$r_t = y_{1t} - \bar{y}_1 - \frac{1}{|\mathcal{N}_0|}\sum_{j\in\mathcal{N}_0} y_{jt} + \bar{m}_{\mathcal{N}_0}$$

becomes

$$(y_{1t} - \bar{y}_1) - \left(\frac{1}{|\mathcal{N}_0|} \sum_{j \in \mathcal{N}_0} y_{jt} - \bar{m}_{\mathcal{N}_0} \right),$$

which simplifies to

$$(y_{1t} - \bar{y}_1) - (\bar{y}_{\mathcal{N}_0,t} - \bar{m}_{\mathcal{N}_0}).$$

In vector form, this is:

$$\boxed{\mathbf{r} = \mathbf{y}_c - \bar{\mathbf{y}}_{c,\mathcal{N}_0} \qquad \Rightarrow \qquad \ell_{\text{DID}}(\mathcal{N}_0) = \|\mathbf{y}_c - \bar{\mathbf{y}}_{c,\mathcal{N}_0}\|_2^2}$$

where

$$\mathbf{y}_c = \mathbf{y}_1 - \bar{y}_1 \, \mathbf{1}_{T_0}, \qquad \bar{\mathbf{y}}_{c,\mathcal{N}_0} = \bar{\mathbf{y}}_{\mathcal{N}_0} - \bar{m}_{\mathcal{N}_0} \, \mathbf{1}_{T_0}.$$

□

Proof. Write donor-pool averages as $\bar{a}_0 = N_0^{-1} \sum_{j \in \mathcal{N}_0} a_j$ and $\bar{u}_{0t} = N_0^{-1} \sum_{j \in \mathcal{N}_0} u_{jt}$. Averaging the factor model across the donor pool gives

$$\bar{y}_{0t} = \bar{a}_0 + \bar{\mathbf{b}}_0^\top \mathbf{f}_t + \bar{u}_{0t},$$

so the gap between the treated unit and the donor mean is

$$y_{1t}(0) - \bar{y}_{0t} = (a_1 - \bar{a}_0) + (\mathbf{b}_1 - \bar{\mathbf{b}}_0)^\top \mathbf{f}_t + (u_{1t} - \bar{u}_{0t}).$$

The DID intercept is the pre-period average of this gap over $\mathcal{T}_1$,

$$\beta^* = (a_1 - \bar{a}_0) + (\mathbf{b}_1 - \bar{\mathbf{b}}_0)^\top \bar{\mathbf{f}} + (\bar{u}_1 - \bar{\bar{u}}_0),$$

with $\bar{u}_1 = T_0^{-1} \sum_{t \in \mathcal{T}_1} u_{1t}$ and $\bar{\bar{u}}_0 = T_0^{-1} \sum_{t \in \mathcal{T}_1} \bar{u}_{0t}$. Subtracting the counterfactual $\hat{y}_{1t}(0) = \bar{y}_{0t} + \beta^*$ from $y_{1t}(0)$, the intercept difference $a_1 - \bar{a}_0$ cancels and

$$y_{1t}(0) - \hat{y}_{1t}(0) = (\mathbf{b}_1 - \bar{\mathbf{b}}_0)^\top (\mathbf{f}_t - \bar{\mathbf{f}}) + (u_{1t} - \bar{u}_{0t}) - (\bar{u}_1 - \bar{\bar{u}}_0).$$

Taking expectations and using $\mathbb{E}[u_{jt}] = 0$ leaves the systematic term

$$\mathbb{E}\big[y_{1t}(0) - \hat{y}_{1t}(0)\big] = (\mathbf{b}_1 - \bar{\mathbf{b}}_0)^\top (\mathbf{f}_t - \bar{\mathbf{f}}),$$

which vanishes for every t if and only if $\mathbf{b}_1 = \bar{\mathbf{b}}_0$ or $\mathbf{f}_t = \bar{\mathbf{f}}$ for all t. □

Chapter 7

Using Least Squares

"Hey, baby, I really do, really do j'adore you... But you're so complicated."
— Neo Jessica Joshua

"You're the worst, you know what you've done to me."
— Jhené Aiko Efuru Chilombo

— OLS: beautiful in theory, complicated in interpretation, and sometimes literally the worst choice for generating synthetic controls.

After reviewing DID as a method to generate counterfactuals, we saw that it implicitly assigns equal weight to all control units. In other words, DID assumes that each control unit should contribute identically to the counterfactual outcome of the treated unit, and that the baseline differences are enough to account for differences in trends. While this assumption simplifies estimation, it is often unrealistic. Control units can differ widely in how closely they resemble the treated unit before treatment, so treating them all as equally informative may distort the counterfactual.

To address this limitation, we need a method for weighting the donor pool $j \in \mathcal{N}_0$ in a way that reflects each unit's similarity to the treated unit. One natural idea is to use ordinary least squares (OLS): choose weights that minimize the squared difference between the treated unit's pre-treatment outcomes and a weighted combination of the donors. This formulation is exactly what synthetic control builds upon. In the next chapters, we'll see how SCM modifies this OLS problem by adding constraints to the weights—turning this unconstrained regression into a more interpretable and robust estimator.

7.1 The Model

Formally, if $\mathbf{y}_1$ is the vector of pre-treatment outcomes for the treated unit and $\mathbf{Y}_0$ is the matrix of pre-treatment outcomes for the donor units, we can define the OLS problem as:

$$\mathbf{w}^* = \underset{\mathbf{w} \in \mathbb{R}^{N_0}}{\operatorname{argmin}} \|\mathbf{y}_1 - \mathbf{Y}_0 \mathbf{w}\|_2^2, \qquad t \in \mathcal{T}_1$$

Here, the resulting weights $\mathbf{w}^*$ can then be used to construct a synthetic control, both pre- and post-treatment, by taking a weighted sum of the donor outcomes. In essence, OLS assigns different weights to control units according to their similarity to the treated unit. We can examine the classic example of the Basque Country in Spain, which experienced a surge in terrorism in the 1970s due to the terrorist organization ETA murdering politicians and doing kidnappings. Suppose we want to estimate what Basque GDP per capita would have been in the absence of terrorism. Using OLS, we can easily construct a synthetic Basque by finding the combination of other Spanish regions whose weighted average best reproduces the Basque GDP path in the pre-1975 period. When we solve the OLS problem, each donor region receives a weight that minimizes the squared difference between its weighted combination and the Basque trajectory before the intervention. Intuitively, regions whose economic paths closely track the Basque Country are given higher weights, while less similar regions receive lower weights.

> **i** Programming Note
>
> This is the first chapter in which we will cease doing the math by hand. Although OLS has a closed-form solution, the problem we consider here has about 20 observations and 16 predictors. You may do this by hand if you wish, but that will likely take much longer than you'd want to spend. So, from here on out, `cvxpy` and `numpy` will be your best friends here.

7.2 Counterfactuals Generated by OLS

```
import cvxpy as cp
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from mlsynth.utils.datautils import dataprep
from IPython.display import display, Markdown
from scmbook.style import set_book_style
set_book_style()

url = ("https://raw.githubusercontent.com/"
       "jgreathouse9/mlsynth/"
       "refs/heads/main/basedata/"
       "basque_data.csv")


data = pd.read_csv(url)

prepped_data = dataprep(data,
"regionname",
"year",
```

```
"gdpcap",
"Terrorism")

y, Y0, T0, time = prepped_data["y"], prepped_data["donor_matrix"], prepped_data["pre_per

# Extract Ywide
Ywide = prepped_data["Ywide"]

subset = Ywide.iloc[:10, :4]

# Round to 2 decimal places
subset_rounded = subset.round(2)

# Convert to markdown
md_table = subset_rounded.to_markdown(index=True)

# Display
display(Markdown(md_table))
```

When we use OLS to construct a synthetic control, nothing mysterious is happening. We are literally just running a linear regression of the treated unit on the donor units, using only pre-treatment data. In this setup, the regression coefficients – the betas – are exactly the weights $\mathbf{w}$ that tell us how much each donor contributes to the synthetic control.

Formally, if $\mathbf{y}_1$ is a $T_0 \times 1$ vector of the treated unit's outcomes in the pre-treatment period and $\mathbf{Y}_0$ is a $T_0 \times J$ matrix of donor outcomes, the OLS weights $\mathbf{w}^*$ are the solution to the normal equations $\mathbf{Y}_0^\top \mathbf{Y}_0 \mathbf{w}^* = \mathbf{Y}_0^\top \mathbf{y}_1$. When we solve, we get: $\mathbf{w}^* = (\mathbf{Y}_0^\top \mathbf{Y}_0)^{-1} \mathbf{Y}_0^\top \mathbf{y}_1$. This is just the standard formula for OLS regression coefficients. The solution minimizes the sum of squared differences between the treated unit and the weighted combination of donors over the pre-treatment period. We can compute the OLS weights directly in Python using `numpy` or `cvxpy`:

See? Same weights! No tricks, no fancy adjustments, just plain old least squares. Let's look at who gets selected:

Much of this does not really square very nicely with intuition. Cataluna, which is very close to the Basque Country (both literally in terms of GDP and being a French border region in Northern Spain) gets negative weight from OLS. The units which are much farther away from the Basque Country (on top of having much different political/economic histories) get much more weight. The reason for this is simple: OLS cares about minimizing errors first and foremost, and will assign any kind of weight it needs to pursuant to that end. This is the unconstrained minimizer, subject to no constraints upon the weights other than them being on the real line. Either way, we can still compute a counterfactual. Let's recall our chapter on averaging: all a weighted average is, is where the weight assigned per donor/element may vary. In the DID estimator we do

$$\beta^* = \underset{\beta \in \mathbb{R}}{\operatorname{argmin}} \left\| \underbrace{\mathbf{y}_1 - \mathbf{Y}_0 \mathbf{w}}_{\text{Fit}} - \underbrace{\beta}_{\text{Mean difference}} \right\|_2^2 \quad \text{s.t.} \quad w_j = \frac{1}{N_0}, \quad \forall t \in \mathcal{T}_1, \, \forall j \in \mathcal{N}_0$$

where the arithmetic mean wins the day. Here, by contrast, the weights are assigned entirely through least squares (without an intercept term).

$$\mathbf{w}^* = \underset{\mathbf{w} \in \mathbb{R}^{N_0}}{\operatorname{argmin}} \, \|\mathbf{y}_1 - \mathbf{Y}_0 \mathbf{w}\|_2^2, \qquad t \in \mathcal{T}_1$$

And, as per the definition of a weighted sum, here's what happens next: we take the weight for Andalucia and multiply it by all the values of the Andalucia vector. Then we do the same for Aragon, and the rest of the donors, scaling them by the weight OLS assigns. Finally, we sum these weighted vectors to obtain our synthetic counterfactual trajectory, or $\hat{y}^{\text{LS}} = \mathbf{Y}_0 \mathbf{w}^*$. Let's do that now.

Here is our prediction using OLS. Unfortunately, while OLS fits very well to the Basque Country before terrorism happened, the out-of-sample/post-1975 predicions exhibits a lot more variance than any control unit in the pre-treatment period did. OLS interpolates perfectly within the span of donor trajectories in-sample, but because the coefficients are unconstrained, extrapolation outside the pre-treatment window becomes unstable. The coefficients for the in-sample model swing wildly when applied to new data. We can see that the prediction line for the Basque country suggests that its economy... would have fallen off a cliff... had terrorism not happened? In other words, if the OLS estmate is to be taken seriously, the onset of terrorism saved the Basque economy... This does not seem like a sensible finding historically, or a very reasonable prediction statistically.

Problem 7.1. DID vs OLS

Explain in your own words why DID assigns equal weight to all control units, and why this assumption might be unrealistic when constructing counterfactuals.

Problem 7.2. Understanding OLS Weights

Describe the role of OLS weights in constructing a synthetic control. How does OLS decide which donor units get larger weights?

Problem 7.3. Negative Weights

1. Explain why OLS can produce negative weights.

2. What are the implications of a negative weight when creating a counterfactual?

Problem 7.4. Coding Pre-Treatment Fit

Using the Basque Country dataset:

```
import numpy as np
import pandas as pd
from mlsynth.utils.datautils import dataprep

# Load data
url = "https://raw.githubusercontent.com/jgreathouse9/mlsynth/refs/heads/main/basedata/b
data = pd.read_csv(url)

# Prepare pre-treatment data
prepped_data = dataprep(data, "regionname", "year", "gdpcap", "Terrorism")
y, Y0, T0, time = prepped_data["y"], prepped_data["donor_matrix"], prepped_data["pre_per

# TODO: compute OLS weights and plot pre-treatment fit
```

1. Compute OLS weights using either `numpy` or `cvxpy`.
2. Plot the treated unit against the OLS synthetic control in the pre-treatment period.
3. Comment on which donors dominate the pre-treatment fit. What is the Root Mean Squared Error?

Problem 7.5. Post-Treatment Predictions

1. Extend your synthetic control to the post-treatment period.
2. Plot the post-treatment predictions and the actual outcomes.
3. Discuss any issues you observe with the OLS predictions after the intervention.

Problem 7.6. Overfitting Concept

1. Define overfitting in the context of OLS synthetic controls.
2. Explain why overfitting occurs more readily when there are many donor units relative to pre-treatment periods.

Problem 7.7. Simulated Overfitting

```
# Simulate data
np.random.seed(42)
n, p = 10, 5
X = np.random.randn(n, p)
true_w = np.array([1, 0.5, 0, 0, 2])
y = X @ true_w + np.random.randn(n) * 0.5

# TODO: fit unconstrained OLS and plot predicted vs true values
```

1. Fit an unconstrained OLS model to the data.
2. Plot the predicted values vs the true values.
3. Comment on the in-sample fit and potential issues if you were to predict new outcomes.

7.3 The Problem with OLS

The OLS approach to constructing synthetic controls, while straightforward, often produces unreliable counterfactuals, as seen in the Basque Country example. The erratic post-1975 predictions, which suggest an implausible economic collapse absent terrorism, highlight a fundamental issue: OLS prioritizes in-sample fit at the expense of out-of-sample validity. This section explores why OLS fails in this context and the statistical mechanisms driving these shortcomings. OLS is designed to minimize the sum of squared differences between the treated unit's pre-treatment outcomes ($\mathbf{y}_1$) and the weighted combination of donor outcomes ($\mathbf{Y}_0\mathbf{w}$):

$$\mathbf{w}^* = \underset{\mathbf{w}\in\mathbb{R}^{N_0}}{\operatorname{argmin}} \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2, \qquad t \in \mathcal{T}_1$$

The closed-form solution, $\mathbf{w}^*_{\text{OLS}} = (\mathbf{Y}_0^\top\mathbf{Y}_0)^{-1}\mathbf{Y}_0^\top\mathbf{y}_1$, achieves this by assigning weights that best fit the pre-treatment data. However, this unconstrained optimization leads to overfitting to noise, where OLS captures both systematic patterns and idiosyncratic noise (ε) in the treated unit's pre-treatment outcomes, producing a synthetic control that closely tracks the pre-treatment period but fails to generalize post-treatment. This explains the volatile "zig-zag" counterfactual in the Basque case, which lacks the stability of any individual donor unit.

Without restrictions, OLS can also assign positive, negative, or arbitrarily large weights to donor units, leading to counterintuitive results. In the Basque example, Cataluna—a region economically and geographically similar to the Basque Country—received a negative weight, while distant regions like Castilla y León were assigned large positive weights. These weights prioritize in-sample accuracy over economic or historical plausibility, undermining interpretability. Furthermore, the model $\mathbf{y}_1 = \mathbf{Y}_0\mathbf{w} + \varepsilon$ is only an approximation of the true data-generating process. By fitting the pre-treatment data too closely, OLS produces a synthetic control that lacks predictive power outside the pre-treatment/in-sample period, resulting in absurd post-treatment predictions, such as the Basque economy being helped by terrorism.

In essence the lesson of this chapter is very simple: OLS's flexibility—its ability to select any weights in $\mathbb{R}^{N_0}$—is also its downfall. Without constraints ensuring some form of regularization, OLS sacrifices interpretability and robustness for an overly precise in-sample fit.

7.4 Where OLS Leaves Off

Even though we see that it produces complicated and in this case uninterpretable results, OLS is still the most natural starting point for counterfactual construction in the panel data setting: it learns weights that make the treated unit's pre-treatment outcomes as close as possible to a combination of donors. But as we've seen, this freedom comes at a price. By allowing weights to take any real value, OLS can perfectly interpolate the pre-treatment

data but at the cost of producing implausible, unstable counterfactuals. In this sense, OLS represents the *purest* form of fitting: a method with no guardrails to contain its predictions.

The next chapter introduces *penalized least squares*, where we rein in OLS by attaching a penalty to the size (or shape) of the weight vector. This penalty forces the model to balance two competing goals:

1. **Fit:** stay close to the treated unit's pre-treatment outcomes.

2. **Simplicity:** avoid overly large or erratic weights.

When we add these penalties, new geometries emerge. The ℓ_2 penalty (ridge) keeps weights small but allows them all to stay active; the ℓ_1 penalty (lasso) forces some weights to zero, yielding sparse synthetic controls. And much later on, when we go one step further—replacing soft penalties with hard constraints—we arrive at the synthetic control method. OLS, then, is the first stop in a larger journey: from *fitting everything* to *fitting responsibly*.

Chapter 8

Convex Synthetic Controls

> "People don't need to wear the same shoes; they should find what suit their feet."
> — Wei Yuan

Having discussed the pitfalls of DID in the small-N setting, we saw its validity hinges on the Parallel Trends Assumption: the expected difference between treated and control units must remain constant over time, in the pre-period and the post-period alike. That assumption is untestable, so in practice we lean on its checkable proxy, the Parallel Pre-Trends Assumption — and when the pre-trends look implausible, or fail to hold in a small-N setting, DID can produce biased estimates. One alternative is OLS, which constructs counterfactuals by flexibly matching pre-treatment outcomes. While OLS can achieve near-perfect pre-intervention fit, it allows unconstrained weights that may extrapolate far outside the support of the donor pool, producing counterfactuals with implausibly high variance. Regularization using ℓ_1 and ℓ_2 norms or PCR weights can shrink extreme weights, but we might prefer a structural constraint that ensures the counterfactual remains within the range of the donor units. This insight motivates the SCM introduced by Abadie et al. (2010), which constructs a weighted combination of donor units that closely approximates the treated unit's pre-treatment trajectory while respecting convexity constraints. This chapter presents the canonical SCM, the one taught in graduate school for econ and public policy students.

8.1 Convex Least Squares

For the classic SCM, we return to a familiar setting where we yet again solve another least squares problem. However, this problem is different from all of the other ones that we have seen before:

$$\mathbf{w}^{\text{SCM}} = \underset{\mathbf{w}^{\text{SCM}} \in \mathcal{W}_{\text{conv}}}{\operatorname{argmin}} \left\| \mathbf{y}_1 - \mathbf{Y}_0 \mathbf{w}^{\text{SCM}} \right\|_2^2 \quad \forall\, t \in \mathcal{T}_1, \tag{8.1}$$

$$\mathcal{W}_{\text{conv}} \in \left\{ \mathbf{w}^{\text{SCM}} \in (\{0\} \cup \mathbb{R}_+)^{|\mathcal{N}_0|} : \sum_{j \in \mathcal{N}_0} w_j = 1 \right\}. \tag{8.2}$$

At first glance, this looks like yet another least squares problem — and in a purely algebraic sense, it is. The difference lies not in the objective but in the *constraints*. In OLS and penalized variants, the weights are completely unrestricted: they can theoretically take any real value, positive or negative. This freedom allows the model to "extrapolate" — to combine donor units in ways that generate counterfactuals lying *outside* the convex hull of observed outcomes. In contrast, the convex synthetic control imposes two structural restrictions: $w_j \geq 0$ and $\sum_j w_j = 1$. No donor is allowed to "subtract" from the synthetic control — we can only form additive mixtures of actual units, and the weights must form a convex combination, ensuring that the synthetic unit lies within the convex hull of the donor pool.

These two constraints completely change the character of the problem. Rather than searching the entire vector space of possible weights, SCM confines the solution to a simplex/convex hull — a compact, convex region of feasible combinations. Because $\mathcal{W}_{\text{conv}}$ is a convex set, the solution $\mathbf{w}^{\text{SCM}}$ is a convex combination of the donor units. This implies that the SCM prediction

$$\hat{\mathbf{y}}_1^{\text{SCM}} = \mathbf{Y}_0 \mathbf{w}^{\text{SCM}}$$

must lie within the convex hull of the donor outcomes. Consider the donor outcome vectors $(\mathbf{y}_j)_{j \in \mathcal{N}_0} \in \mathbb{R}^{T_0}$. What this means is that the prediction (or, the weighted average) lies inside the convex hull of the donor outcomes. Geometrically, this means that if we imagine the donor outcomes as points in a space, the synthetic control must live inside the polygon (or polyhedron, in higher dimensions) formed by those points. A simple way to see this is to start with a one-dimensional example: if we have two donors with pre-treatment averages of 3 and 7, then any convex combination of them—say, $0.25 \times 3 + 0.75Ö7 = 6$, will fall somewhere on the line segment connecting 3 and 7.

Consider any points within this line segment: there is no way to get outside that line segment unless we allow negative weights or weights that sum to more than one. We can do this in two dimensions too:

We can see that the space of predictions is a combination of the control units. Here, there are an infinte number of solutions that we can use for the donors to solve for the target weight vector. As we move to higher dimensions, this idea generalizes: the convex hull is simply the set of all mixtures of donors where the weights are nonnegative and sum to one. We can understand the convex hull idea better by projecting them onto a *two-dimensional plane*, per **?@fig-donor-span**.

This plot shows the Basque Country within the convex hull of donor units. If we were to assign any donor a weight of 1 and the rest 0, the predicted synthetic control would just be that control unit. But because one control unit will likely be insufficient to minimize the pre-treatment residual, we oftentimes rely on more donor units. One of the things I struggled with while learning SCM is the reasons why the convex hull constraint mattered. I did not know what it did or why, I just heard it was important for identification. So, to properly illustrate this, below we will derive the SCM weights for a corner solution.

Problem 8.1. Parallel Trends and SCM

Explain why the standard Difference-in-Differences (DID) assumption of parallel trends, assessed through its testable pre-trends proxy, is problematic in small-N settings, and how SCM addresses this issue. In particular, discuss their weighting schemes and what this implies.

Problem 8.2. Convex Hull

Define the convex hull of the donor pool outcomes in your own words. Why might constraining the synthetic control weights to this hull be important?

Problem 8.3. Weight Constraints

Consider the SCM weight set

$$\mathcal{W}_{\text{conv}} \in \left\{ \mathbf{w}^{\text{SCM}} \in (\{0\} \cup \mathbb{R}_+)^{|\mathcal{N}_0|} : \sum_{j \in \mathcal{N}_0} w_j = 1 \right\}.$$

Explain the interpretation of the constraint $\sum_j w_j = 1$ and $w_j \geq 0$. What does the summation to 1 suggest? The nonnegativity?

Problem 8.4. OLS vs SCM

Describe a scenario where unconstrained OLS would assign negative or extreme weights to donor units. How does SCM prevent this?

8.1.1 A Simple Synthetic Control

Above we say how in very simple settings when we have two time periods and one donor that lies within the span of the donor outcomes, we will have an infinite number of solutions. But, what if we could only have one possible solution? Suppose we have a treated unit and two donor units in 2D:

$$\mathbf{y} = \begin{bmatrix} 50 \\ 60 \end{bmatrix}, \qquad \mathbf{Y}_0 = [\mathbf{y}_1 \; \mathbf{y}_2] = \begin{bmatrix} 20 & 48 \\ 22 & 50 \end{bmatrix}.$$

The first of these units is treated, and the latter two are the control units. Graphically, the plot of their trajectories looks like

We aim to find a synthetic control as a convex combination of the donor units that matches the target vector as closely as possible. Again, because of the fact that the feasible weight set is a convex set, this means a tradeoff is at work: where we increase one weight for one

donor, this necessarily reduces the weight the other one gets. Exploiting this fact, this allows us to define one weight *in terms of* the other one. To do this, we need the difference vector:

$$\mathbf{y}_1 - \mathbf{y}_2 = \begin{bmatrix} 20 \\ 22 \end{bmatrix} - \begin{bmatrix} 48 \\ 50 \end{bmatrix}.$$

Perform the subtraction componentwise:

$$(\mathbf{y}_1 - \mathbf{y}_2)_1 = 20 - 48 = -28, \tag{8.3}$$
$$(\mathbf{y}_1 - \mathbf{y}_2)_2 = 22 - 50 = -28. \tag{8.4}$$

Using this, we can write the synthetic prediction as a function of w_1 (the weight on donor 1):

$$\hat{\mathbf{y}}(w_1) = \mathbf{y}_2 + w_1(\mathbf{y}_1 - \mathbf{y}_2) = \begin{bmatrix} 48 \\ 50 \end{bmatrix} + w_1 \begin{bmatrix} -28 \\ -28 \end{bmatrix} = \begin{bmatrix} 48 - 28w_1 \\ 50 - 28w_1 \end{bmatrix}.$$

Finally, the residual vector, or the difference between the treated unit and the in-sample prediction, is

$$\mathbf{r}(w_1) = \mathbf{y} - \hat{\mathbf{y}}(w_1) = \begin{bmatrix} 50 \\ 60 \end{bmatrix} - \begin{bmatrix} 48 - 28w_1 \\ 50 - 28w_1 \end{bmatrix} = \begin{bmatrix} 2 + 28w_1 \\ 10 + 28w_1 \end{bmatrix}.$$

This sets up the next step: minimizing the squared norm of the residual vector to find the optimal weight w_1. Minimizing the ℓ_2 norm gives us the following optimization problem:

$$w_1^* = \underset{w_1 \in \mathbb{R}}{\operatorname{argmin}} \, \|\mathbf{r}(w_1)\|_2^2 \equiv \underset{w_1 \in \mathbb{R}}{\operatorname{argmin}} \, \Big[(2 + 28w_1)^2 + (10 + 28w_1)^2\Big].$$

To find the minimizing weight of this convex function, we differentiate $f(w_1)$ with respect to w_1. Since $f(w_1)$ is a sum of two terms, the sum rule allows us to differentiate each term separately and then sum the results:

$$f(w_1) = (2 + 28w_1)^2 + (10 + 28w_1)^2$$

Let's begin with the first term, $(2 + 28w_1)^2$. This is a composition of two functions: a linear inner function and a quadratic outer function. To differentiate, we apply the chain rule. Let

$$h_1(w_1) = 2 + 28w_1 \quad \text{(inner function)}, \qquad g_1(u) = u^2 \quad \text{(outer function)}.$$

The derivatives are

$$\frac{\mathrm{d}g_1}{\mathrm{d}u} = 2u, \quad \frac{\mathrm{d}h_1}{\mathrm{d}w_1} = 28.$$

Applying the chain rule gives

$$\frac{\mathrm{d}}{\mathrm{d}w_1}(2 + 28w_1)^2 = \left.\frac{\mathrm{d}g_1}{\mathrm{d}u}\right|_{u=h_1(w_1)} \cdot \frac{\mathrm{d}h_1}{\mathrm{d}w_1} = 2(2 + 28w_1) \cdot 28.$$

Next, consider the second term, $(10 + 28w_1)^2$. Let

$$h_2(w_1) = 10 + 28w_1 \quad \text{(inner function)}, \qquad g_2(v) = v^2 \quad \text{(outer function)}.$$

The derivatives are

$$\frac{\mathrm{d}g_2}{\mathrm{d}v} = 2v, \quad \frac{\mathrm{d}h_2}{\mathrm{d}w_1} = 28.$$

Applying the chain rule gives the derivative of the second term:

$$\frac{\mathrm{d}}{\mathrm{d}w_1}(10 + 28w_1)^2 = 2(10 + 28w_1) \cdot 28.$$

By the sum rule we have:

$$\frac{\mathrm{d}f}{\mathrm{d}w_1} = 2(2 + 28w_1) \cdot 28 + 2(10 + 28w_1) \cdot 28.$$

which just simplifies to

$$\frac{\mathrm{d}f}{\mathrm{d}w_1} = 56(2 + 28w_1) + 56(10 + 28w_1).$$

Now we factor out the 56

$$\frac{\mathrm{d}f}{\mathrm{d}w_1} = 56\Big((2 + 28w_1) + (10 + 28w_1)\Big)$$

and then combine like terms inside of the parentheses

$$(2 + 28w_1) + (10 + 28w_1) = 12 + 56w_1,$$

and that gives us this result:

$$\frac{\mathrm{d}f}{\mathrm{d}w_1} = 56(12 + 56w_1).$$

We can now distribute the 56 $56 \cdot 12 + 56 \cdot 56w_1 = 0$, which gives us this result: $672 + 3136w_1 = 0$. Now we isolate the w_1 by subtracting 672 from the LHS

$$3136w_1 = -672$$

and divide by 3136

$$w_1 = \frac{-672}{3136}.$$

Now we find the greatest common divisor. I was too lazy to do this when I was writing this out so I used Python.

Since this value lies outside the feasible interval $[0, 1]$, the SCM solution occurs at the nearest boundary, which is $w_1 = 0$ and $w_2 = 1$. This gives us the synthetic control

$$\hat{\mathbf{y}} = \mathbf{y}_2 = \begin{bmatrix} 48 \\ 50 \end{bmatrix},$$

which is simply the second donor vector copied over. Now, we can verify with `cvxpy`:

Okay, so without question, this is the toughest derivation we've done so far. But I want to emphasize why I chose to do it this way. Even in a corner solution like the one we just saw, there is a powerful lesson to be learned: if your treated unit is extreme with respect to your donor pool, the weight assigned to the donors will increasingly hinge on the nearest neighbors, because that is the best the convex hull condition allows. In other words, the SCM is responding to the geometry of the data. The treated unit lies outside the convex hull/span of the donors, so the optimization goes to the nearest boundary, assigning all weight to the donor closest to the treated unit.

This has a clear practical takeaway: analysts should always check whether the treated unit is extreme relative to the donor set. If it is, the synthetic control may rely heavily on a few donors, and the resulting fit may reflect geometric constraints rather than donor suitability. Of course, there are ways to circumvent this problem, but understanding that it is a problem in the first place is key. Being comfortable reasoning about the boundary behavior of one's is crucial for interpreting SCM results correctly and for making informed decisions about donor selection, as well as diagnosing whether the standard solver needs adjustments.

Problem 8.5. Corner Solutions

What does it mean for a treated unit to lie "outside the convex hull" of the donor pool? How does this affect the synthetic control weights?

Problem 8.6. Simple 2D SCM

Given a treated unit $\mathbf{y} = [50, 60]^\top$ and donor units $\mathbf{y}_1 = [20, 22]^\top$, $\mathbf{y}_2 = [48, 50]^\top$, derive the optimal SCM weight w_1 analytically, and identify the resulting synthetic control.

Problem 8.7. Residual Norm Minimization

Let $\mathbf{r}(w) = \mathbf{y} - \hat{\mathbf{y}}(w)$, where $\hat{\mathbf{y}}(w) = \sum_j w_j \mathbf{y}_j$. Show that minimizing $\|\mathbf{r}(w)\|_2^2$ is equivalent to the standard least squares problem for SCM.

8.2 Returning to the Basque Example

Per our discussion and **?@fig-donor-span**, we can see that the Basque Country is not extreme with respect to its donor set. The Basque Country is an outlier in other ways, such as industrialization and education and other covariates, but it is relatively well within the span of the outcomes of the donor pool. This suggests that we can at least try and use the standard SCM to obtain our weights for the Basque Country. For the purposes of distinguishing these from one another, I will compare the Convex Basque to the OLS Basque as well as the penalized approaches from the previous chapter.

The OLS weights are highly variable, with some donors receiving extremely large positive or negative values, reflecting the unconstrained nature of the estimator and its tendency to overfit the pre-treatment data. Ridge (L2) penalization shrinks these extreme weights toward zero, producing smaller positive and negative values, but still allowing negative contributions. LASSO (L1) introduces sparsity, assigning nonzero weights to only a few donors, notably Cataluna, Madrid, and Principado de Asturias, while setting most others to zero. The SIMPLEX estimator, by contrast, produces a sparse, interpretable set of weights that are strictly non-negative and sum to one, with Cataluna contributing the majority of the weight, followed by moderate contributions from Madrid and a minimal contribution from Principado de Asturias. Compared to LASSO, SIMPLEX is cleaner and more interpretable, as it enforces the convex combination constraint while automatically selecting the most relevant donors for constructing the synthetic Basque counterfactual without needing to tune any lambda parameters. Now we can plot the predictions of both models, depicted in **?@fig-scm-basque**.

We can see here that aside from having more sensible weights, the convex SCM has a more sensible out of sample prediction too, certainly compared to OLS. This is because of the convex hull restriction, meaning that the weights will never allow the post-intervention prediction to extrapolate. In other words, even though SCM does not fit the Basque Country as well as least-squares does, the out-of-sample predictions are more realistic because of the geometric regularization that SCM provides.

The pre-treatment RMSEs indicate that OLS achieves the closest fit to the Basque series before treatment, with a near-zero RMSE of 0.002, reflecting its unconstrained ability to overfit the pre-treatment data. Ridge, LASSO, and SIMPLEX have higher pre-treatment RMSEs (0.065, 0.083, and 0.084, respectively) due to penalization or convexity constraints, which

limit overfitting but produce a slightly less precise fit. In terms of the ATT, OLS predicts a large positive post-treatment effect (1.522), likely exaggerated by its overfitting, while Ridge, LASSO, and SIMPLEX yield negative ATTs (-0.758, -0.671, and -0.692, respectively), suggesting that the Basque outcome was slightly lower than the synthetic counterfactual post-treatment. Overall, these results illustrate a trade-off: OLS achieves perfect pre-treatment fit but unrealistic post-treatment effects, whereas Ridge, LASSO, and SIMPLEX provide more regularized, interpretable estimates at the cost of slightly higher pre-treatment error.

8.3 Finite Sample Error Under The Linear Factor Model

Okay, so this is a nice trick, but analysts should also want to quantify the conditions under which SCM is effective. In other words, we should wish to derive the circumstances where we would expect SCM style methods to perform well. Abadie et al. (2010) motivates SCM by a linear factor model. We are interested in characterizing the post-treatment estimation error for the treated unit under the linear factor model, assuming pre-treatment fit is perfect. When I say estimation error/bias here, I refer to the absolute difference between the estimated treatment effect and the true treatment effect. We do not care at all about the direction of the bias, we just care about the absolute bias. So by definition, for any post-treatment period t, the bias is characterized as

$$\epsilon := \hat{\tau}_{1t} - \tau_{1t} = y_{1t}(0) - \sum_{j \in \mathcal{N}_0} w_j y_{jt}(0).$$

Substituting the linear factor model

$$y_{jt}(0) = \delta_t + \theta_t Z_j + \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \varepsilon_{jt},$$

we obtain

$$\epsilon = (\delta_t + \theta_t Z_1 + \boldsymbol{\mu}_1^\top \boldsymbol{\lambda}_t + \varepsilon_{1t}) - \sum_{j \in \mathcal{N}_0} w_j (\delta_t + \theta_t Z_j + \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \varepsilon_{jt}).$$

Expanding the weighted sum:

$$\epsilon = (\delta_t + \theta_t Z_1 + \boldsymbol{\mu}_1^\top \boldsymbol{\lambda}_t + \varepsilon_{1t}) - \left(\delta_t \sum_j w_j + \theta_t \sum_j w_j Z_j + \sum_j w_j \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \sum_j w_j \varepsilon_{jt} \right).$$

By SCM weight constraints, $\sum_j w_j = 1$, so the intercept cancels:

$$\delta_t - \delta_t \sum_j w_j = 0.$$

If pre-treatment covariates are balanced, $\sum_j w_j Z_j = Z_1$, so the covariate term cancels:

$$\theta_t Z_1 - \theta_t \sum_j w_j Z_j = 0.$$

This leaves

$$\epsilon = \boldsymbol{\mu}_1^\top \boldsymbol{\lambda}_t + \varepsilon_{1t} - \sum_j w_j \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t - \sum_j w_j \varepsilon_{jt}.$$

Define the factor-loading mismatch vector

$$\boldsymbol{\nu} := \boldsymbol{\mu}_1 - \sum_j w_j \boldsymbol{\mu}_j.$$

Then the post-treatment error becomes

$$\epsilon = \boldsymbol{\lambda}_t^\top \boldsymbol{\nu} + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}.$$

Exact pre-treatment fit implies

$$0 = \boldsymbol{\lambda}_t^\top \boldsymbol{\nu} + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}, \quad \forall t \in \mathcal{T}_1.$$

Rearranging:

$$\boldsymbol{\lambda}_t^\top \boldsymbol{\nu} = \sum_j w_j (\varepsilon_{jt} - \varepsilon_{1t}).$$

Stacking over all pre-treatment periods defines

$$\Lambda \boldsymbol{\nu} = b, \quad b := \begin{pmatrix} \sum_j w_j (\varepsilon_{jt_1} - \varepsilon_{1t_1}) \\ \vdots \\ \sum_j w_j (\varepsilon_{jt_{T_0}} - \varepsilon_{1t_{T_0}}) \end{pmatrix},$$

where $\Lambda \in \mathbb{R}^{T_0 \times r}$ has rows $\boldsymbol{\lambda}_t^\top$. If Λ has full column rank r, the least squares solution is

$$\boldsymbol{\nu} = (\Lambda^\top \Lambda)^{-1} \Lambda^\top b.$$

Expanding:

$$\Lambda^\top \Lambda = \sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \boldsymbol{\lambda}_t^\top, \quad \Lambda^\top b = \sum_{t \in \mathcal{T}_t} \boldsymbol{\lambda}_t \sum_j w_j (\varepsilon_{jt} - \varepsilon_{1t}).$$

Hence

$$\boldsymbol{\nu} = \Big(\sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \boldsymbol{\lambda}_t^\top \Big)^{-1} \sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \sum_j w_j (\varepsilon_{jt} - \varepsilon_{1t}).$$

Finally, the post-treatment estimation error can be written as

$$\epsilon = \boldsymbol{\lambda}_t^\top \Big(\sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \boldsymbol{\lambda}_t^\top \Big)^{-1} \sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \sum_j w_j (\varepsilon_{jt} - \varepsilon_{1t}) + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}.$$

This decomposition shows two sources of post-treatment error: (i) pre-treatment idiosyncratic differences projected through the factor space, and (ii) direct post-treatment idiosyncratic mismatch. Given this finite sample bound, we can now get to the main identifying assumption of the SCM:

Lemma B.1 (Li & Shankar (2024))

Consider the linear factor model:

$$y_{jt}(0) = \delta_j + \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \varepsilon_{jt}, \quad j \in \mathcal{N}_0 \cup \{1\},\ t \in \mathcal{T}_1,$$

where δ_j is the intercept, $\boldsymbol{\mu}_j \in \mathbb{R}^r$ is the vector of factor loadings for unit j, and $\boldsymbol{\lambda}_t \in \mathbb{R}^r$ is a vector of common factors at time t. SCM's identification assumption holds if all control units share the same intercept and factor loadings:

$$\delta_j = \delta, \quad \boldsymbol{\mu}_j = \boldsymbol{\mu}, \quad \forall j \in \mathcal{N}_0,$$

and the treated unit's intercept and factor loadings can be expressed as a convex combination of the controls:

$$\delta_1 = \sum_{j \in \mathcal{N}_0} w_j \delta_j, \quad \boldsymbol{\mu}_1 = \sum_{j \in \mathcal{N}_0} w_j \boldsymbol{\mu}_j, \quad \mathcal{W} \in \Big\{ \mathbf{w} \in (\{0\} \cup \mathbb{R}_+)^{|\mathcal{N}_0|} : \sum_{j \in \mathcal{N}_0} w_j = 1 \Big\}.$$

However, it is important to note that the underlying assumption that the treated unit's intercept δ_1 and factor loadings $\boldsymbol{\mu}_1$ can be represented as a convex combination of the controls' parameters is not directly testable, because the latent factors $\boldsymbol{\lambda}_t$ and unit-specific loadings $\boldsymbol{\mu}_j$ are unobserved. This is the finite sample bound; asymptotically, we can show that the error is bounded by a function in Theorem 8.1.

Theorem 8.1 (Asymptotic Bias Bound)**.** *As shown in Abadie et al. (2010), as the number of pre-treatment periods $|\mathcal{T}_1| \to \infty$ and the synthetic control achieves increasingly precise pre-treatment fit, the bound on the post-treatment bias shrinks.*

$$\Big|\mathbb{E}[\epsilon]\Big| \lesssim C(g)^{1/g} \frac{\bar{\lambda}^2\, F\, N^{1/g}}{\xi_{A_P}} \max\Big(\frac{\bar{m}_g^{1/g}}{T_0^{1-1/g}},\ \frac{\bar{\sigma}^{1/2}}{T_0^{1/2}} \Big).$$

Proof

When we construct a synthetic control, the quantity $\epsilon = \hat{\tau}_{1t} - \tau_{1t}$ measures the difference between the estimated and true post-treatment effect. The asymptotic bias bound provides a way to understand how large this expected error can be as the number of pre-treatment periods grows. It shows that as the pre-treatment window T_0 increases and the synthetic control fits the treated unit more precisely, the expected bias shrinks.

The bound depends on several intuitive factors. The term $C(g)^{1/g}$ comes from controlling the contribution of large, rare shocks in the pre-treatment idiosyncratic errors; it scales the bound according to how "heavy-tailed" these shocks are but does not shrink with more data. The factor $\bar{\lambda}^2$ measures the size of the common factors; larger factors amplify any mismatch between the treated unit and the synthetic control. The number of latent factors, F, represents the dimensions along which the treated unit can differ from the controls, so more factors increase potential bias. The term $N^{1/g}$ reflects the influence of having more donor units: even though the synthetic control combines them optimally, more donors introduce additional sources of variation. Finally, the smallest eigenvalue of the factor Gram matrix, ξ_{A_P}, captures the geometry of the pre-treatment factors. A well-conditioned factor space (larger ξ_{A_P}) reduces the potential amplification of any mismatch, while a poorly conditioned factor space increases it.

The last piece of the bound, the maximum between $\bar{m}_g^{1/g}/T_0^{1-1/g}$ and $\bar{\sigma}^{1/2}/T_0^{1/2}$, reflects the stochastic component of the bias coming from the idiosyncratic shocks. The first term captures the effect of occasional large shocks, while the second captures typical fluctuations measured by the variance. As the number of pre-treatment periods grows, both terms shrink, with the variance term typically dominating, leading to the familiar $T_0^{-1/2}$ rate of decay. The expected post-treatment bias decreases as more pre-treatment data are available and as the synthetic control fits better, but it also clarifies how the number of factors, size of the factors, number of donors, and factor space geometry influence the potential size of the bias.

Problem 8.8. Factor-Loading Mismatch

Starting from the linear factor model

$$y_{jt}(0) = \delta_t + \boldsymbol{\mu}_j^\top \boldsymbol{\lambda}_t + \varepsilon_{jt},$$

show that the post-treatment error for the treated unit can be written as

$$\epsilon = \boldsymbol{\lambda}_t^\top \boldsymbol{\nu} + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}.$$

Problem 8.9. Corner Solution Behavior

Suppose the donor pool consists of two units with pre-treatment outcomes

$$\mathbf{y}_1 = [2, 2]^\top, \quad \mathbf{y}_2 = [10, 10]^\top$$

and the treated unit has pre-treatment outcomes

$$\mathbf{y}_{\text{treated}} = [12, 12]^\top.$$

- Let the synthetic control weight vector be $\mathbf{w} = [w_1, w_2]^\top$.
- Analyze what happens to the synthetic control vector $\hat{\mathbf{y}}_{\text{SC}} = w_1\mathbf{y}_1 + w_2\mathbf{y}_2$ as $\lim_{w_2 \to 1^-} \mathbf{w}$.

Problem 8.10. Pre-Treatment Fit and Bias

Using the expression for $\Delta\boldsymbol{\mu}$, explain why increasing the number of pre-treatment periods $|\mathcal{T}_1|$ improves identification of the treated unit's latent factors.

Problem 8.11. Convex SCM Implementation

Using `cvxpy`, write a small script to compute SCM weights for a treated unit with 3 donors in 5 pre-treatment periods. Verify that weights are nonnegative and sum to 1.

Problem 8.12. Comparing OLS and SCM

For the Basque Country data, compute OLS weights and convex SCM weights. Compare the pre-treatment RMSE of both approaches. Which method produces more interpretable weights?

Problem 8.13. Convex Hull Visualization

Using Python or R, create a 2D plot showing the donor vectors and the convex hull. Plot the treated unit and the synthetic control on the same graph.

Problem 8.14. Pre-Treatment MSE and Factor Matching

Compute the pre-treatment mean squared error for SCM with varying subsets of donors. How does the MSE change as you remove the donors closest to the treated unit?

Problem 8.15. Discussion / Critical Thinking

Suppose the treated unit is extreme relative to all donors. What are the implications for SCM estimation and interpretation? How might you address this issue in practice?

Problem 8.16. Negative Weights

In standard SCM, weights are constrained to be nonnegative and sum to one. Discuss the implications of allowing negative weights.

- How might negative weights affect the interpretability of the synthetic control?
- When might allowing negative weights be useful?

Problem 8.17. Adding an Intercept

Suppose we modify the SCM optimization to include an unrestricted intercept term in addition to the convex combination of donors.

- How does this change the optimization problem?
- How might this affect pre-treatment fit and post-treatment extrapolation?
- Discuss potential benefits and drawbacks of including an intercept.

Problem 8.18. High-Dimensional Factor Models

Consider a scenario where the number of latent factors r exceeds the number of pre-treatment periods $|\mathcal{T}_1|$.

- What happens to the factor-loading mismatch solution $\Delta\boldsymbol{\mu} = (\Lambda^\top\Lambda)^{-1}\Lambda^\top b$ in this case?
- How does this affect identification of the SCM weights?

Problem 8.19. Heterogeneous Factor Loadings

Suppose that in the linear factor model, donor units have heterogeneous factor loadings that cannot perfectly replicate the treated unit.

- How does this heterogeneity influence the post-treatment bias?
- How might one diagnose or mitigate this issue in applied settings?

Problem 8.20. Convex Hull Limitations

The SCM ensures that the synthetic control lies within the convex hull of the donors.

- Discuss situations in which this restriction might prevent a good counterfactual fit.
- How does the geometry of the donor pool constrain the SCM solution?
- Are there alternatives or modifications to SCM that can relax the convex hull constraint while retaining interpretability?

8.4 Conclusion

SCM provides a principled way to construct counterfactuals that respect the geometry of the donor pool, avoiding the implausible extrapolations that can occur under unconstrained OLS. By restricting weights to lie within the convex hull of the donors, SCM (ideally) produces interpretable and stable synthetic controls, even in small-N settings. While the key identification assumption—that the treated unit's latent factors can be represented as a convex combination of the controls—is not directly testable, Abadie et al. (2010) shows that as the number of pre-treatment periods increases and pre-treatment fit improves, the bound on post-treatment bias shrinks. In practice, a lower pre-treatment mean squared error indicates that the synthetic control is closely tracking the treated unit's unobserved factors, making it more likely that the SCM weights are effectively matching the relevant unit-specific characteristics. This geometric and factor-based perspective provides both theoretical justification and practical guidance for applying SCM reliably.

8.5 Asymptotic Bias Bound

Proof. The finite sample bound gives us a nice approximation for any given set of factors/loadings in the finite case. However, mathematics is not concerned with secular realities. Econometricians oftentimes wish to make statements about how estimators behave as a sample size grows without bound. For this proof, we are going to algebraically manipulate the finite sample bound into something asymptotic in nature. This way, we can make statements about how the bound of the bias changes given some set of inputs.

Above I mentioned how the finite-sample bound has two sources: (i) pre-treatment idiosyncratic differences projected through the factor space, and (ii) direct post-treatment idiosyncratic mismatch. Thus, this proof proceeds in two acts. In the first act, we focus on the bias coming from pre-treatment idiosyncratic differences projected through the factor space. The tools we use to do this are the triangle inequality, Jensen's inequality, and Cauchy–Schwarz. In the second act, we deal with the bias from direct post-treatment idiosyncratic mismatches. Here, we use Hölder's inequality, the vector-valued Rosenthal inequality, and Minkowski's inequality.

8.5.1 Setup

Recall that in the finite sample bias is expressed as:

$$\epsilon := |\hat{\tau}_{1t} - \tau_{1t}|.$$

The interactive fixed effects model is:

$$y_{jt} = \boldsymbol{\lambda}_t^\top \boldsymbol{\mu}_j + \varepsilon_{jt}, \quad j = 1, \ldots, N+1,$$

where $\boldsymbol{\lambda}_t$ are common factors, $\boldsymbol{\mu}_j$ are factor loadings, and ε_{jt} are idiosyncratic shocks. The post-intervention error for the treated unit is

$$\epsilon = \boldsymbol{\lambda}_t^\top \Big(\sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \boldsymbol{\lambda}_t^\top \Big)^{-1} \sum_{t \in \mathcal{T}_1} \boldsymbol{\lambda}_t \sum_j w_j (\varepsilon_{jt} - \varepsilon_{1t}) + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}$$

or more simply

$$\epsilon = \boldsymbol{\lambda}_t^\top \boldsymbol{\nu} + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt},$$

where

$$\boldsymbol{\nu} := \boldsymbol{\mu}_1 - \sum_j w_j \boldsymbol{\mu}_j$$

is the factor-loading difference between the treated unit and the synthetic control.

8.5.2 Act 1

To start off, we begin by taking expectations and the absolute value of both sides of the equal sign:

$$|\mathbb{E}[\epsilon]| = \Big| \mathbb{E}[\boldsymbol{\lambda}_t^\top \boldsymbol{\nu} + \varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}] \Big|.$$

As we mentioned, we do this because we do not care about the direction of the bias, so the absolute value allows us to make more general statements about the bias in this manner. By the linearity of expectation, we can separate the sum inside the expectation into two parts:

$$|\mathbb{E}[\epsilon]| = |\mathbb{E}[\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}] + \mathbb{E}[\varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}]|.$$

Because the RHS of the equals sign is a sum that evalautes to a scalar, the first inequality from our toolkit that we will apply is the triangle inequality. Recall that this inequality states that the magnitude of a sum cannot exceed the sum of magnitudes — the "length" of two steps taken together can never be longer than taking them one after another. In our context, this means that the total expected bias (which combines factor-driven and idiosyncratic components) cannot exceed the sum of the individual contributions of each source of bias. Formally:

$$|\mathbb{E}[\epsilon]| = |\mathbb{E}[\boldsymbol{\lambda}_t^\top \boldsymbol{\nu} + (\varepsilon_{1t} - \sum_j w_j \varepsilon_{jt})]| \leq |\mathbb{E}[\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}]| + |\mathbb{E}[\varepsilon_{1t} - \sum_j w_j \varepsilon_{jt}]|.$$

We assume that the idiosyncratic shocks are mean-zero and have finite variance, i.e. $\mathbb{E}[\varepsilon_{jt}] = 0$ and $\text{Var}(\varepsilon_{jt}) < \infty$. Hence, the expected post-treatment idiosyncratic mismatch vanishes, leaving only the component driven by the common factors:

$$|\mathbb{E}[\epsilon]| = |\mathbb{E}[\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}]|.$$

Note that even if the random noise cancels out on average, in any finite sample it might not — and we want to know how big that randomness can make our bias look (that is the point of Act 2).

Anyways,this stage, we have the absolute value of an expectation:

$$|\mathbb{E}[\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}]|.$$

Note that the absolute value function $f(x) = |x|$ is convex. Why? Consider two numbers $x = -2$ and $y = 4$, and let $\theta = 0.3$. Then the weighted average is

$$\theta x + (1-\theta)y = 0.3(-2) + 0.7(4) = -0.6 + 2.8 = 2.2.$$

Taking the absolute value gives

$$|\theta x + (1-\theta)y| = |2.2| = 2.2.$$

On the other hand, the weighted sum of absolute values is

$$\theta|x| + (1-\theta)|y| = 0.3(2) + 0.7(4) = 0.6 + 2.8 = 3.4.$$

Indeed, we see that

$$|\theta x + (1-\theta)y| = 2.2 \leq 3.4 = \theta|x| + (1-\theta)|y|,$$

Extending the triangle inequality to weighted sums, the "length" of a weighted average of two numbers cannot exceed the weighted sum of their individual lengths. Because the absolute value function is convex, we can bound this term with Jensen's inequality. Applying Jensen gives

$$|\mathbb{E}[\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}]| \leq \mathbb{E}[|\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}|].$$

Intuitively, this says that the magnitude of the average bias is no larger than the average of the magnitudes of the biases across different realizations.

Now note that $\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}$ on the LHS is a dot product between two vectors: the common-factor vector at time t and the factor-loading difference between the treated unit and the estimated synthetic control. The way we bound dot products between vectors is with Cauchy-Schwarz. When we apply this to the LHS, we get this result:

$$|\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}| \leq \|\boldsymbol{\lambda}_t\|_2 \, \|\boldsymbol{\nu}\|_2.$$

However, this bound may be further simplified by introducing a uniform bound over all pre-treatment periods. To do this, we take the max norm of all of the common factors in the pre-treatment period

$$\bar{\lambda} := \|\boldsymbol{\lambda}\|_\infty = \max_{\in \mathcal{T}_1} \|\boldsymbol{\lambda}_t\|_2.$$

This is the largest value of any common factor across the pre-treatment periods. Since each $\|\boldsymbol{\lambda}_t\|_2 \leq \bar{\lambda}$, we can factor this constant out of the expectation:

$$\mathbb{E}[|\boldsymbol{\lambda}_t^\top \boldsymbol{\nu}|] \leq \mathbb{E}[\bar{\lambda} \, \|\boldsymbol{\nu}\|_2] = \bar{\lambda} \, \mathbb{E}[\|\boldsymbol{\nu}\|_2].$$

Thus, combining with the previous steps, we obtain

$$|\mathbb{E}[\epsilon]| \leq \bar{\lambda} \, \mathbb{E}\|\boldsymbol{\nu}\|_2.$$

At this point, the only remaining random or unknown quantity is the factor-loading difference $\boldsymbol{\nu}$. So far, we've manipulated the norms of these, but we have not considered the factor difference itself. As it happens, we can bound this term too. Recall the definition of $\boldsymbol{\nu}$, or the difference in factor loadings. In the finite sample derivation, we defined it implicitly via the least squares system

$$\Lambda \boldsymbol{\nu} = b, \quad b_t = \sum_j w_j(\varepsilon_{jt} - \varepsilon_{1t}), \quad \Lambda = [\boldsymbol{\lambda}_1^\top; \ldots; \boldsymbol{\lambda}_{T_0}^\top],$$

solving for it analytically. The solution is:

$$\boldsymbol{\nu} = (\Lambda^\top \Lambda)^{-1} \Lambda^\top b.$$

To place a bound on the maximum factor loading mismatch, we can use the operator norm. The point of this here is to essentially answer "Given the pre-treatment mismatches (b) and the geometry of the factor space ($(\Lambda^\top \Lambda)^{-1}\Lambda^\top$), what is the largest possible factor-loading difference that could result?" By the submultiplicativity of the operator norm:

$$\|\boldsymbol{\nu}\|_2 = \|(\Lambda^\top \Lambda)^{-1} \Lambda^\top b\|_2 \leq \|(\Lambda^\top \Lambda)^{-1}\|_{\text{op}} \, \|\Lambda^\top b\|_2.$$

Think of the factor-loading difference as the outcome of shooting a bullet at a bullseye in a dark room. The matrix transformation, which combines the geometry of the factor space and the projection of pre-treatment mismatches, acts like the layout of the room, stretching and rotating the trajectory of the bullet in different directions. The operator norm then

identifies the worst-case scenario: the maximum distance the bullet could land from the bullseye, given your current vantage point. In other words, even without knowing the exact direction of the pre-treatment mismatches, the operator norm provides a bound on how large the factor-loading difference can be, capturing the greatest possible amplification caused by the factor space.

Normalize the Gram matrix $M = \frac{1}{T_0}\Lambda^\top\Lambda$, giving

$$\|(\Lambda^\top\Lambda)^{-1}\|_{\text{op}} = \frac{1}{T_0}\|M^{-1}\|_{\text{op}} \leq \frac{1}{T_0\,\xi_{A_P}}, \quad \xi_{A_P} = \lambda_{\min}(M).$$

In the bullseye analogy, T_0 represents the number of vantage points from which you can take a shot at the bullseye. Each vantage point provides information about the geometry of the room, and averaging across these points (normalizing by T_0) reduces the maximum potential deviation of a shot. The smallest eigenvalue ξ_{A_P} identifies the weakest direction in the room, the direction in which the bullet can be stretched the most. Together, T_0 and ξ_{A_P} give a conservative bound on how far any shot could land from the bullseye, capturing the worst-case amplification of pre-treatment mismatches.

8.5.3 Act 2: Bounding the Stochastic Component

We now address the stochastic component of the estimation error, which arises because the idiosyncratic shocks ε_{jt} of the control units cannot perfectly match those of the treated unit ε_{1t}, even with optimal weights w_j. The goal is to bound the expected factor-loading mismatch $\mathbb{E}\|\boldsymbol{\nu}\|_2$, which captures how random pre-treatment shocks propagate through the factor structure to affect the bias. The approach uses Hölder's inequality to control per-period mismatches, the vector-valued Rosenthal inequality to quantify their accumulation over time, and Minkowski's inequality to simplify the final bound.

The per-period mismatch is defined as $b_t = \sum_j w_j(\varepsilon_{jt} - \varepsilon_{1t})$, where ε_{jt} represents the idiosyncratic shock for control unit j at time t, ε_{1t} is the shock for the treated unit, and w_j are the weights forming the synthetic control. This b_t measures how much the weighted average of control shocks deviates from the treated unit's shock at time t. Imagine trying to match a song's melody with a choir that's slightly out of tune: b_t is the error in that approximation for one note. For clarity, define $\bar{m}_g = \sup_t \mathbb{E}|b_t|^g$ for some $g \geq 2$, which captures the worst-case size of large shocks (higher moments), and $\bar{\sigma} = \sup_t \mathbb{E}[b_t^2]$, which measures typical variability (variance). Assume the shocks ε_{jt} are independent across time t and units j, with mean zero ($\mathbb{E}[\varepsilon_{jt}] = 0$) and finite moments up to order g. The factors $\boldsymbol{\lambda}_t$ are treated as fixed (non-random), a standard simplification in this context.

To control b_t, apply Hölder's inequality, which separates the weights and shocks:

$$|b_t| \leq \left(\sum_j w_j^q\right)^{1/q} \left(\sum_j |\varepsilon_{jt} - \varepsilon_{1t}|^p\right)^{1/p}, \quad \frac{1}{p} + \frac{1}{q} = 1.$$

This is like balancing two forces: the weights w_j (how much each control unit contributes) and the shock differences $\varepsilon_{jt} - \varepsilon_{1t}$ (how far each unit's noise deviates from the treated unit). Hölder's inequality ensures neither dominates excessively, bounding the mismatch by the interplay of weight distribution and shock variability.

Next, define the vector $X_t = b_t \boldsymbol{\lambda}_t$, where $\boldsymbol{\lambda}_t$ scales the mismatch by the factor magnitudes. The total stochastic deviation over the pre-treatment window is $S = \sum_{t=1}^{T_0} X_t$, capturing the cumulative effect of these mismatches. To bound $\mathbb{E}\|S\|_2$, use the vector-valued Rosenthal inequality for independent mean-zero vectors $X_t \in \mathbb{R}^F$ with $g \geq 2$:

$$\mathbb{E}\|S\|_2^g \lesssim C(g) \left[\sum_{t=1}^{T_0} \mathbb{E}\|X_t\|_2^g + \left(\sum_{t=1}^{T_0} \mathbb{E}\|X_t\|_2^2 \right)^{g/2} \right],$$

where $C(g)$ is a constant depending only on g, and $\lesssim$ hides absolute constants irrelevant for the asymptotic rate. This inequality is like estimating the total error in a series of coin tosses over T_0 periods: the first term captures rare large errors (higher moments), while the second reflects typical fluctuations (variance). Since $\|X_t\|_2 = |b_t| \|\boldsymbol{\lambda}_t\|_2 \leq |b_t| \bar{\lambda}$, where $\bar{\lambda} = \max_{t \in \mathcal{T}_1} \|\boldsymbol{\lambda}_t\|_2$, account for F factors (since X_t has F components). Thus, $\mathbb{E}\|X_t\|_2^g \lesssim F \bar{\lambda}^g \mathbb{E}|b_t|^g$ and $\mathbb{E}\|X_t\|_2^2 \lesssim F \bar{\lambda}^2 \mathbb{E}[b_t^2]$.

Using Hölder's inequality and the normalization $\sum_j w_j^q \leq 1$, obtain $\mathbb{E}|b_t|^g \lesssim \bar{m}_g$ and $\mathbb{E}[b_t^2] \leq \bar{\sigma}$. Including N control units (which enters multiplicatively in b_t), this becomes $\mathbb{E}\|X_t\|_2^g \lesssim F N^{1/g} \bar{\lambda}^g \bar{m}_g$ and $\mathbb{E}\|X_t\|_2^2 \lesssim F N^{1/g} \bar{\lambda}^2 \bar{\sigma}$. Summing over $t = 1, \ldots, T_0$ gives $\sum_{t=1}^{T_0} \mathbb{E}\|X_t\|_2^g \lesssim T_0 F N^{1/g} \bar{\lambda}^g \bar{m}_g$ and $\sum_{t=1}^{T_0} \mathbb{E}\|X_t\|_2^2 \lesssim T_0 F N^{1/g} \bar{\lambda}^2 \bar{\sigma}$. Substituting into Rosenthal's inequality yields:

$$\mathbb{E}\|S\|_2^g \lesssim C(g) \left[T_0 F N^{1/g} \bar{\lambda}^g \bar{m}_g + \left(T_0 F N^{1/g} \bar{\lambda}^2 \bar{\sigma} \right)^{g/2} \right].$$

Taking the g-th root and applying Minkowski's inequality (which picks the dominant contribution, like choosing the louder of two competing noises) simplifies to:

$$\mathbb{E}\|S\|_2 \lesssim C(g)^{1/g} F N^{1/g} \max \left(\left(T_0 \bar{\lambda}^g \bar{m}_g \right)^{1/g}, \left(T_0 \bar{\lambda}^2 \bar{\sigma} \right)^{1/2} \right).$$

Simplifying each term, $\left(T_0 \bar{\lambda}^g \bar{m}_g \right)^{1/g} = T_0^{1/g} \bar{\lambda} \bar{m}_g^{1/g}$ and $\left(T_0 \bar{\lambda}^2 \bar{\sigma} \right)^{1/2} = \sqrt{T_0} \bar{\lambda} \bar{\sigma}^{1/2}$, so:

$$\mathbb{E}\|S\|_2 \lesssim C(g)^{1/g} F N^{1/g} \bar{\lambda} \max \left(T_0^{1/g} \bar{m}_g^{1/g}, \sqrt{T_0} \bar{\sigma}^{1/2} \right).$$

This bound shows that the stochastic mismatch grows with the number of factors F and control units $N^{1/g}$, but is moderated by the pre-treatment window T_0. To connect to the factor-loading mismatch, recall $\boldsymbol{\nu} = (\Lambda^\top \Lambda)^{-1} \Lambda^\top b$, so $\mathbb{E}\|\boldsymbol{\nu}\|_2 \leq \|(\Lambda^\top \Lambda)^{-1}\|_{\text{op}} \mathbb{E}\|S\|_2$. Since $\|(\Lambda^\top \Lambda)^{-1}\|_{\text{op}} = 1/(T_0 \xi_{A_P})$, where $\xi_{A_P} = \lambda_{\min}(M)$ and $M = \frac{1}{T_0} \Lambda^\top \Lambda$, combine to get:

$$\mathbb{E}\|\boldsymbol{\nu}\|_2 \lesssim C(g)^{1/g} \frac{FN^{1/g}\bar{\lambda}}{T_0 \xi_{A_P}} \max\left(T_0^{1/g}\bar{m}_g^{1/g}, \sqrt{T_0}\bar{\sigma}^{1/2}\right).$$

Simplify the powers of T_0: $T_0^{1/g}/T_0 = T_0^{-(1-1/g)}$ and $\sqrt{T_0}/T_0 = T_0^{-1/2}$, yielding:

$$\mathbb{E}\|\boldsymbol{\nu}\|_2 \lesssim C(g)^{1/g} \frac{FN^{1/g}\bar{\lambda}}{\xi_{A_P}} \max\left(\bar{m}_g^{1/g} T_0^{-(1-1/g)}, \bar{\sigma}^{1/2} T_0^{-1/2}\right).$$

Since the post-treatment bias satisfies $|\mathbb{E}[\epsilon]| \leq \bar{\lambda}\mathbb{E}\|\boldsymbol{\nu}\|_2$, the final bound is:

$$|\mathbb{E}[\epsilon]| \lesssim C(g)^{1/g} \frac{FN^{1/g}\bar{\lambda}^2}{\xi_{A_P}} \max\left(\bar{m}_g^{1/g} T_0^{-(1-1/g)}, \bar{\sigma}^{1/2} T_0^{-1/2}\right).$$

This bound reveals the bias's behavior as T_0 grows. The term $\max\left(\bar{m}_g^{1/g} T_0^{-(1-1/g)}, \bar{\sigma}^{1/2} T_0^{-1/2}\right)$ determines the decay rate: the variance term $\bar{\sigma}^{1/2} T_0^{-1/2}$ typically dominates, giving a decay rate of $O(T_0^{-1/2})$, meaning the bias shrinks as more pre-treatment data is available. For example, if $T_0 = 100$ and the variance term dominates, $T_0^{-1/2} = 0.1$, significantly reducing the bias. However, more factors F or control units N can increase the bound, while a well-conditioned factor space (larger ξ_{A_P}) reduces it. Hölder's inequality controlled per-period errors, Rosenthal's inequality quantified their accumulation, and Minkowski's inequality isolated the dominant term, providing a complete picture of how stochastic noise propagates through the factor structure and decays with T_0. □

:::

Part III

Further Topics

Chapter 9

Penalized Synthetic Controls

> "I'll keep it simple, baby... I'ma keep it simple with you, baby. You know I don't ever play no games. You know I don't ever complicate it."
> — *Jhené Aiko Efuru Chilombo*

In the last chapter we saw how OLS can fit the pre-treatment data almost perfectly, yet fail catastrophically out of sample. This behavior is a textbook example of *overfitting*. Overfitting occurs when a model captures not only the systematic patterns in the data but also the random noise specific to the sample at hand. An overfit model performs tends to perform extremely well on the data it was trained on but generalizes poorly to new or unseen data. In our case, the OLS synthetic control overfits the pre-treatment outcomes of the Basque Country by adjusting the donor weights so precisely that it reproduces every wiggle in the historical GDP series, including idiosyncratic shocks that have nothing to do with long-run economic structure. Formally, overfitting is a manifestation of the bias–variance tradeoff. A highly flexible model has low bias—it can approximate the training data closely—but high variance, meaning that small perturbations in the data can cause large swings in the estimated parameters. Bias, by extension, is about the difference between the model prediction and the true values; the OLS synthetic control has very low bias in the pre-treatment period, but has extremely high variance in the post-intervention period. That is, the close fit OLS obtains comes at the cost of having high post-intervention variance in the out of sample predictions.

To make OLS behave more responsibly, we need to give it boundaries — some way to tell the model that a perfect fit is not always desirable if it means using bizarre or extreme weights. Statistically, this is called **regularization**. In optimization terms, it means we trade off fidelity for discipline: we still want the in-sample (pre-treatment) prediction to fit well, but not so well that it memorizes noise or violates interpretability. Regularization is a method for tempering this variance by constraining or penalizing the size of the coefficient vector. In this setting, we accept a small amount of bias in exchange for substantial reductions in variance and improved stability of the predictions. Regularization modifies the least squares objective by adding a penalty term that discourages overly complex or extreme coefficients. When we say penalty term, we usually talk about adding a norm to the regression problem. In geometric terms, regularization shrinks the solution space from an open field to a smaller

playground—the model can still move, but not run off into absurd directions. This trades off fit (the first term) against simplicity (the second). The tuning parameter λ controls how much we're willing to sacrifice accuracy to gain stability. The general form of a regularized least squares problem is

$$\mathbf{w}^* = \underset{\mathbf{w}\in\mathbb{R}^{N_0}}{\operatorname{argmin}} \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_p^p,$$

where $\lambda \geq 0$ controls the strength of the penalty and p determines its type.

To develop intuition, consider an analogy from the music industry—one that reflects how access, selectivity, and diversity shape outcomes.

9.0.1 The Sparse World: Motown and the ℓ_1 Norm

Back in the 1960s and 70s, the music world was dominated by a handful of major record labels and A&R gatekeepers. Recording equipment was expensive, and access to distribution channels was limited. Only a select few artists ever made it to the airwaves. This was the era of Motown, Prince, and the Jackson 5—a small, elite set of artists who carried enormous weight in defining popular music in the United States and globally.

This is the world of the ℓ_1 norm. Under ℓ_1 regularization, each coefficient pays a *flat fee* simply for existing—just as every artist must pass through an expensive and highly selective gatekeeping process. As a result, only a few coefficients "make it big." The rest are driven exactly to zero, left unsigned and unheard. The resulting model is *sparse*: only the most essential features or donor units remain active. Mathematically, if our unpenalized mean squared error (MSE) is 12 for weights $(0.03, 0.9, 12)$ and we apply an ℓ_1 penalty with $\lambda = 1$, the total loss becomes

$$\text{Loss}_1(\mathbf{w}) = \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_1,$$

where $\|\mathbf{w}\|_1 = \sum |w_i|$ penalizes the sum of the absolute values of the coefficients. For example, with weights $\mathbf{w} = (0.03, 0.9, 12)$, an unpenalized MSE of 12, and $\lambda = 1$, the total loss is:

$$12 + (|0.03| + |0.9| + |12|) = 12 + 0.03 + 0.9 + 12 = 24.93.$$

The large coefficient (12) incurs a hefty penalty, so the optimizer may shrink it or set it to zero unless it significantly improves the fit. Unless it drastically improves the fit, the optimizer will prefer to zero it out entirely—just as only a handful of artists survive the Motown audition process. The ℓ_1 norm thus rewards exclusivity in variable selection: few stars, high impact.

Consider weights $\mathbf{w} = (0.03, 0.9, 12)$ with an unpenalized MSE of 12. Let's explore two scenarios with different λ values. A small λ is like a lenient A&R manager, allowing more artists to get a deal. The ℓ_1 penalty is:

$$\|\mathbf{w}\|_1 = |0.03| + |0.9| + |12| = 0.03 + 0.9 + 12 = 12.93,$$

and the total loss is:

$$\text{Loss}_1(\mathbf{w}) = 12 + 0.1 \cdot 12.93 = 12 + 1.293 = 13.293.$$

The penalty is small, so the optimizer may keep all coefficients, as the cost of including even the large weight (12) is modest. The model remains close to the unpenalized OLS solution, prioritizing fit over sparsity.

A large λ is like a ruthless gatekeeper, signing only the biggest stars. The penalty becomes:

$$\|\mathbf{w}\|_1 = 12.93,$$

and the total loss is:

$$\text{Loss}_1(\mathbf{w}) = 12 + 10 \cdot 12.93 = 12 + 129.3 = 141.3.$$

The large penalty, especially for the weight 12, pressures the optimizer to shrink or eliminate coefficients. The optimizer is likely to set smaller weights (e.g., 0.03 and 0.9) to zero and reduce the large weight, favoring a sparse model with only the most essential donor units.

As λ becomes very large, the ℓ_1 penalty term $\lambda\|\mathbf{w}\|_1$ dominates the objective function. To keep the total loss finite, the optimizer must minimize $\|\mathbf{w}\|_1$, driving all coefficients to zero ($\mathbf{w} = \mathbf{0}$). In this limit, the loss becomes:

$$\text{Loss}_1(\mathbf{w}) \approx \|\mathbf{y}_1\|_2^2,$$

since $\mathbf{Y}_0\mathbf{w} = \mathbf{0}$, and the penalty term vanishes ($\|\mathbf{w}\|_1 = 0$). The model predicts $\mathbf{y}_1 \approx \mathbf{0}$, ignoring the data entirely, resulting in high bias but zero variance. In the Motown analogy, this is like an impossibly high entry cost—no artists get signed, and the label produces no music. Such a model is useless for prediction but illustrates how ℓ_1 enforces extreme sparsity as λ grows. The ℓ_1 norm thus rewards exclusivity: only a few coefficients—our "star artists"—remain non-zero, creating a simpler, more interpretable model that avoids overfitting by focusing on the most essential donor units.

Problem 9.1. Computing the l1 Norm

Compute the ℓ_1 norm of the weight vector $\mathbf{w} = (0.5, -1.2, 3.4, 0)$.

Problem 9.2. Total Loss with Small lambda

Suppose the unpenalized MSE is 5 and $\mathbf{w} = (0.5, -1.2, 3.4, 0)$. Compute the total ℓ_1-penalized loss for $\lambda = 0.1$.

Problem 9.3. Total Loss with Large lambda

Using the same weights and unpenalized MSE as in Problem 2, compute the total loss for $\lambda = 10$.

Problem 9.4. Effect of Increasing lambda

Explain in one sentence what happens to small coefficients when λ increases in ℓ_1 penalization. Then compute the total loss if $\mathbf{w}$ is shrunk to $(0, 0, 2, 0)$ with $\lambda = 5$ and unpenalized MSE 5.

Problem 9.5. Extreme Sparsity Limit

If λ becomes extremely large, the optimizer drives all weights to zero. Compute the approximate total loss if $\mathbf{y}_1$ has norm $\|\mathbf{y}_1\|_2^2 = 20$ and $\mathbf{w} = \mathbf{0}$.

9.0.2 The Dense World: TikTok and the ℓ_2 Norm

Now, fast forward to today. In today's music industry, digital production, social media, and streaming platforms have democratized access. Artists from Lagos, Medellín, or London (think Burna Boy and Tems, Karol G, and Ego Ella May) can all reach a global audience in ways that would be impossible even in the 1990s. Distribution of music is *relatively* cheap. Beyond, methods of discovery (like Instagram, SoundCloud, or Spotify) are close to free, open to countless creators. The practical result of this is that we have a dense, inclusive ecosystem where many artists share the spotlight/different sugments of the spotlight. This is the world of the ℓ_2 norm in regularization. The ℓ_2 penalty adds a cost to the loss function based on the squared magnitudes of the coefficients:

$$\text{Loss}_2(\mathbf{w}) = \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2 + \lambda\|\mathbf{w}\|_2^2,$$

where $\|\mathbf{w}\|_2^2 = \sum w_i^2$. For example, with weights $\mathbf{w} = (0.03, 0.9, 12)$, an unpenalized MSE of 12, and $\lambda = 1$, the total loss is:

$$12 + (0.03^2 + 0.9^2 + 12^2) = 12 + (0.0009 + 0.81 + 144) = 156.8109.$$

The quadratic penalty scales sharply: small coefficients, like 0.03, contribute almost nothing (0.0009), while large ones, like 12, are heavily penalized (144). This encourages the optimizer to shrink all coefficients rather than eliminate them, keeping every donor unit in the model but reducing their individual impact. The result is a smooth, balanced ensemble—no single unit dominates, mirroring the diverse, inclusive world of modern music streaming.

As above, we can think about what happens as the penality term λ increases or decreases. Consider weights $\mathbf{w} = (0.03, 0.9, 12)$ with an unpenalized MSE of 12. Let's examine two scenarios. A small λ is like a low streaming platform fee, allowing artists to thrive with minimal restriction. The ℓ_2 penalty is:

$$\|\mathbf{w}\|_2^2 = 0.03^2 + 0.9^2 + 12^2 = 0.0009 + 0.81 + 144 = 144.8109,$$

and the total loss is:

$$\text{Loss}_2(\mathbf{w}) = 12 + 0.1 \cdot 144.8109 = 12 + 14.48109 = 26.48109.$$

The penalty is moderate, so the optimizer slightly shrinks the coefficients, especially the large one (12), but keeps all non-zero. The model remains close to the OLS solution, prioritizing fit over aggressive shrinkage.

A large λ is like a steep tax on popularity, heavily penalizing dominant artists. The penalty remains:

$$\|\mathbf{w}\|_2^2 = 144.8109,$$

but the total loss is:

$$\text{Loss}_2(\mathbf{w}) = 12 + 10 \cdot 144.8109 = 12 + 1448.109 = 1460.109.$$

The large penalty, driven by the $12^2 = 144$ term, pushes the optimizer to significantly shrink all coefficients, especially the largest. The model becomes smoother, with no single donor unit dominating, resembling a diverse playlist where many artists contribute modestly. As λ grows very large, the ℓ_2 penalty term $\lambda\|\mathbf{w}\|_2^2$ dominates the objective. To keep the loss finite, the optimizer minimizes $\|\mathbf{w}\|_2^2$, driving all coefficients toward zero ($\mathbf{w} \to \mathbf{0}$). The loss approaches:

$$\text{Loss}_2(\mathbf{w}) \approx \|\mathbf{y}_1\|_2^2,$$

since $\mathbf{Y}_0\mathbf{w} \to \mathbf{0}$ and the penalty term vanishes ($\|\mathbf{w}\|_2^2 \to 0$). The model predicts $\mathbf{y}_1 \approx \mathbf{0}$, sacrificing fit for extreme simplicity, resulting in high bias but low variance. In the streaming analogy, this is like a platform charging such high fees that no artist can afford to stand out—everyone's presence shrinks to nearly nothing. Unlike ℓ_1, ℓ_2 doesn't produce sparsity; coefficients approach zero smoothly, maintaining a balanced but minimal model. The ℓ_2 norm thus fosters inclusivity: all coefficients—our "global artists"—remain in the model, but their contributions are shrunk, creating a smooth, balanced ensemble that avoids overfitting by distributing influence evenly.

9.0.3 The Tradeoff

The contrast between these two regimes mirrors the evolution of music. The ℓ_1 norm enforces scarcity, pushing only the most valuable features into the spotlight. The ℓ_2 norm promotes inclusivity, blending many small contributions into a harmonious whole. Both approaches combat overfitting by introducing discipline—one through *selectivity*, the other through

shrinkage. In econometric terms, regularization constrains the solution space of $\mathbf{w}$ and prevents the unconstrained OLS estimator from taking extreme values just to eliminate small residuals. The difference lies in how each penalty shapes this constraint: ℓ_2 encourages smooth shrinkage across all donors, while ℓ_1 encourages sparsity by zeroing out many coefficients entirely. Both serve the same purpose—to teach our model not to memorize noise, but to focus on structure that generalizes.

Problem 9.6. Computing the ℓ_2 Norm Squared

Compute the squared ℓ_2 norm of the weight vector $\mathbf{w} = (0.5, -1.2, 3.4, 0)$.

Problem 9.7. Total Loss with Small λ

Suppose the unpenalized MSE is 5 and $\mathbf{w} = (0.5, -1.2, 3.4, 0)$. Compute the total ℓ_2-penalized loss for $\lambda = 0.1$.

Problem 9.8. Total Loss with Large λ

Using the same weights and unpenalized MSE as in Problem 2, compute the total loss for $\lambda = 10$.

Problem 9.9. Effect of Increasing λ

Explain in one sentence how increasing λ affects large coefficients in ℓ_2 penalization. Then compute the total loss if $\mathbf{w}$ is shrunk to $(0.1, -0.2, 0.5, 0)$ with $\lambda = 5$ and unpenalized MSE 5.

Problem 9.10. Extreme Shrinkage Limit

If λ becomes extremely large, the optimizer drives all weights toward zero. Compute the approximate total loss if $\|\mathbf{y}_1\|_2^2 = 20$ and $\mathbf{w} \approx \mathbf{0}$.

9.1 Cross Validation

Whether we live in the sparse Motown world of ℓ_1 or the crowded TikTok ecosystem of ℓ_2, we still face one final act of judgment: how much selectivity is too much? If the label signs too few artists, we miss out on talent; if it signs everyone, we drown in noise. In statistical terms, this question is about choosing the right value of λ. Recall that the basic penalized model was

$$\mathbf{w}^* = \underset{\mathbf{w} \in \mathbb{R}^{N_0}}{\operatorname{argmin}} \|\mathbf{y}_1 - \mathbf{Y}_0 \mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_p^p,$$

where we have a lambda term which chooses the strength of the norm that we select. Thus, a crucial aspect of penalized synthetic control methods is the choice of the regularization

parameter, lambda. For the ℓ_1 norm, if lambda is too small, the method behaves much like OLS, producing many non-zero weights and potentially overfitting the pre-treatment period. If lambda is too large, most weights are shrunk toward zero, resulting in an overly sparse synthetic control that may underfit the treated unit. For the ℓ_2, a small lambda allows weights to take extreme values, again risking overfitting, while a large lambda shrinks all weights toward zero, distributing influence more evenly but potentially introducing bias in the post-treatment counterfactual. To do this, we employ cross validation. Cross validation is used to select hyperparameters for machine-learning models.

Definition 9.1 (Cross Validation)**.** Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ be a dataset of n labeled observations and let $\mathcal{A}$ be a learning algorithm producing a model $\hat{f}$ given training data. Cross validation is a procedure to estimate the out-of-sample prediction error of $\hat{f}$ by partitioning $\mathcal{D}$ into K disjoint folds $\mathcal{D}_1, \ldots, \mathcal{D}_K$. For each fold $k = 1, \ldots, K$, we train the model on the complement $\mathcal{D} \setminus \mathcal{D}_k$ and evaluate it on the holdout fold $\mathcal{D}_k$, producing a validation error $\mathrm{Err}^{(k)}$. The cross-validated error is the arithmetic average over the folds:

$$\mathrm{CV}(\mathcal{A}, \mathcal{D}) = \frac{1}{K} \sum_{k=1}^{K} \mathrm{Err}^{(k)}.$$

In our case we wish to select λ. The idea is simple: for a given section of the pre-treatment period and a candidate lambda, we fit the penalized least-squares model and compute the prediction error on held-out pre-treatment data. By repeating this process across multiple folds or rolling windows and averaging the validation errors, we identify the lambda that minimizes out-of-sample pre-treatment error, balancing the bias-variance tradeoff for more reliable post-treatment counterfactual predictions. Let $\mathbf{y}_1 \in \mathbb{R}^{T_0}$ be the vector of pre-treatment outcomes for the treated unit, and $\mathbf{Y}_0 \in \mathbb{R}^{T_0 \times N_0}$ the corresponding donor matrix. Define the rolling-origin splits of the pre-treatment period. For each window $k = 1, \ldots, K$, let

$$\mathbf{y}_{1,\mathrm{train}}^{(k)} \in \mathbb{R}^{|\mathcal{T}_{\mathrm{train}}^{(k)}|}, \quad \mathbf{Y}_{0,\mathrm{train}}^{(k)} \in \mathbb{R}^{|\mathcal{T}_{\mathrm{train}}^{(k)}| \times N_0}$$

$$\mathbf{y}_{1,\mathrm{val}}^{(k)} \in \mathbb{R}^{|\mathcal{T}_{\mathrm{val}}^{(k)}|}, \quad \mathbf{Y}_{0,\mathrm{val}}^{(k)} \in \mathbb{R}^{|\mathcal{T}_{\mathrm{val}}^{(k)}| \times N_0}$$

be the training and validation vectors/matrices for that window. For a given norm $p \in \{1, 2\}$ and regularization λ, the penalized weights for the k-th training window are

$$\mathbf{w}^{(k)}(\lambda) = \underset{\mathbf{w} \in \mathbb{R}^{N_0}}{\operatorname{argmin}} \|\mathbf{y}_{1,\mathrm{train}}^{(k)} - \mathbf{Y}_{0,\mathrm{train}}^{(k)} \mathbf{w}\|_2^2 + \lambda \|\mathbf{w}\|_p^q$$

where $q = 1$ for LASSO and $q = 2$ for Ridge. The validation MSE for this window is

$$\mathrm{MSE}^{(k)}(\lambda) = \frac{1}{|\mathcal{T}_{\mathrm{val}}^{(k)}|} \|\mathbf{y}_{1,\mathrm{val}}^{(k)} - \mathbf{Y}_{0,\mathrm{val}}^{(k)} \mathbf{w}^{(k)}(\lambda)\|_2^2$$

The cross-validated MSE for each λ is the average across all rolling windows:

$$\overline{\text{MSE}}(\lambda) = \frac{1}{K}\sum_{k=1}^{K} \text{MSE}^{(k)}(\lambda)$$

The optimal regularization parameter is

$$\lambda^* = \underset{\lambda \in \{\lambda_1,\ldots,\lambda_L\}}{\text{argmin}} \quad \overline{\text{MSE}}(\lambda)$$

Finally, the weights for the full pre-treatment period using λ^* are

$$\mathbf{w}^* = \underset{\mathbf{w}\in\mathbb{R}^{N_0}}{\text{argmin}} \|\mathbf{y}_1 - \mathbf{Y}_0\mathbf{w}\|_2^2 + \lambda^* \|\mathbf{w}\|_p^q$$

We can even visualize this. For example, suppose we had pre-treatement data from 1900 to 1975. **?@fig-rocvex** plots the folds of "training" and validation.

Many forms of cross validation exist, and users can choose the ones they feel are most important for their purposes.

Problem 9.11. Computing Pre-Treatment RMSE

Given `y_pre = [2, 3, 5]` and `y_hat_pre = [2.1, 1.9, 4.8]`, compute the pre-treatment RMSE.

Problem 9.12. Comparing LASSO and Ridge Effects

Explain in one sentence why LASSO produces sparse weights while Ridge produces more evenly distributed weights. Then, for weights $\mathbf{w} = [0.5, 0.5, 0.5, 0.5]$ and $\lambda = 0.2$, compute the ℓ_1 and ℓ_2 penalties.

Problem 9.13. Conceptual Understanding of Lambda

Explain in your own words why choosing a very small λ or a very large λ can lead to poor out-of-sample performance for LASSO and Ridge. Use the bias-variance tradeoff in your explanation.

Problem 9.14. Rolling-Origin Window Creation

For pre-treatment years 1900–1910, create three rolling-origin windows with `initial_train = 5` and `step = 2`. List the training and validation indices for each fold.

Problem 9.15. Custom Cross Validation Routine

Write a short Python function that takes a donor matrix Y_0, treated vector y, and a list of candidate λ values, and performs K-fold cross validation to select the best λ for LASSO. You may use `cvxpy` for optimization.

Problem 9.16. Interpreting CV Results

After cross validation, suppose $\lambda^* = 0.3$ for LASSO and $\lambda^* = 0.1$ for Ridge. Sketch or describe how you expect the post-treatment counterfactual predictions to differ between these two models. Which would be sparser and why?

9.2 Implementing Penalized OLS Models

Okay, now that we have a good sense of what these models do conceptually, let's work on implementing this in code. At this point, we will develop more structured forms of script to accomplish our tasks. Consider the function below:

```
l2results = Opt.SCopt(
    num_control_units=Y0.shape[1],
    target_outcomes_pre_treatment=y[:T0],
    num_pre_treatment_periods=T0,
    donor_outcomes_pre_treatment=Y0,
    scm_model_type="OLS",
    donor_names=donor_names,
    lambda_penalty=0.2,
    p=2,
    q=2,
)
```

This is the `Opt.SCopt` backend. It is buried within the lightless depths of the `mlsynth` API. This class provides a unified way to estimate penalized regression models using unconstrained OLS and penalized least-squared models such as the Lasso and Ridge penalties. producing the donor weights, the predicted counterfactual trajectory for the treated unit, and additional information such as pre-treatment RMSE and a mapping of donor names to weights. The key inputs include the donor matrix `Y0` (columns correspond to control units, rows to time periods), the treated series `y` to construct a counterfactual for, and `T0`, the number of pre-treatment periods. Optional inputs such as `donor_names` make the resulting weights easier to interpret, while `lambda_penalty` sets the strength of the penalty, and `p` and `q` determine the type of regularization: `p=1` for Lasso, `p=2` for Ridge. The function outputs a structured object containing the estimated numeric weights, predicted synthetic series, a dictionary mapping donor names to weights, and the pre-treatment RMSE. Using this approach, we can directly compare OLS and penalized synthetic control estimates in a consistent framework, while ensuring that each donor's contribution is appropriately weighted. Given that we have the ability to fit penalized synthetic controls with `Opt.SCopt`, we can choose the optimal penalty parameter λ using cross-validation on the pre-treatment period. The `fit_scopt_cv` function automates this process.

```
results = fit_scopt_cv(Y0, y, T0, donor_names=donor_names, p_list=[1,2])
```

The function takes the full donor matrix `Y0`, the treated series `y`, and `T0`, the number of pre-treatment periods. Optional inputs include `donor_names` for easier interpretation, and `p_list` to indicate which penalization norms to fit: 1 for Lasso and 2 for Ridge. For each penalized model, the function tests a grid of λ values and evaluates performance on held-out pre-treatment periods. The λ that minimizes the average root mean squared error (RMSE) is selected as optimal. After choosing the best λ, the function fits the full synthetic control series for the treated unit, producing the numeric weights, the counterfactual predictions, a mapping of donor names to weights, and the pre-treatment RMSE for each method. OLS can also be included alongside penalized models for comparison.

The pre-treatment RMSE for the LASSO with $\lambda = 0.2$ is 0.083, for Ridge with $\lambda = 0.2$ is 0.065, and for OLS is 0.002. For the LASSO, the weights are very sparse. Cataluna contributes the most with 0.757, followed by Madrid at 0.190 and Principado de Asturias at 0.066. All other donors have weights effectively zero, reflecting the L1 penalty's tendency to select only a few contributors. For Ridge, the weights are more evenly distributed across donors. The largest contributions come from Madrid at 0.299, Cataluna at 0.239, Principado de Asturias at 0.247, Rioja at 0.202, and Cantabria at 0.172. Other donors have moderate positive or negative weights, showing the L2 penalty spreads influence across many donors rather than selecting only a few. OLS produces more extreme weights, with Castilla Y Leon at 6.093, Andalucia at 4.177, and Castilla-La Mancha at -2.655, while most other donors have smaller positive or negative contributions. From these results, we can clearly see that OLS happens to fit the best out of all of these, but the weights are much different when we look across them. This is most obvious when we plot the counterfactual as we do in **?@fig-penlsest**.

The out of sample estimates are much smoother than the OLS predictions which have much higher variance.

Unlike the positive effect suggested by OLS, the penalized estimators tell a very different story. Both LASSO and Ridge produce negative ATT estimates: LASSO at -671 and Ridge at -758. This indicates that once we shrink or regularize the donor weights, the synthetic control tracks the treated unit more cautiously, and the apparent "boost" disappears, even turning negative. In machine learning terms, this reflects the bias-variance tradeoff: we accept slightly more bias in the pre-treatment fit if it reduces the variance of out-of-sample predictions, leading to more reliable post-treatment estimates.

9.3 Principal Components Regression

In the previous section, we used penalized regression to shrink or select donor weights. An alternative approach to control overfitting is Principal Component Regression (PCR) introduced by Amjad et al. (2018) and Agarwal et al. (2021). Principal Component Analysis is based on the idea that large datasets may be well approximated by only a few key dominant patterns. Now, imagine the outcome matrix of controls as a literal cloud of points whose

shape we wish to explain using as few dimensions as possible, capturing the essence of the data without reproducing every detail.

We already know the idea of fitting a line of best fit in two dimensions: it's just a line that summarizes the linear relationship among the points. PCA is the natural extension of that idea to higher dimensions. Instead of one line, we now look for a few directions (principal components) through this cloud of points that capture the most variance. We have as many principal components as we have donor units. Each principal component is like a stiff line through the data that explains as much of the spread as possible. Once we've identified the top few directions, we can project the donor points onto these lines. This gives us a low-dimensional summary that preserves most of the original structure. Reconstructing the donor series from these projections is just like asking: "If I only knew the points' positions along these main lines, what would the original points look like?" The reconstruction is never perfect, but with enough top components, it usually captures the bulk of the pattern, filtering out minor fluctuations or noise.

9.3.1 Singular Value Decomposition

We can formalize the idea of projecting onto dominant patterns using Singular Value Decomposition (SVD). SVD decomposes the centered matrix $\mathbf{Y}_0 - \bar{\mathbf{Y}}_0$ as

$$\mathbf{Y}_0 - \bar{\mathbf{Y}}_0 = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top,$$

where $\mathbf{U} \in \mathbb{R}^{T_0 \times T_0}$ and $\mathbf{V} \in \mathbb{R}^{N_0 \times N_0}$ are orthogonal matrices, and $\mathbf{\Sigma} \in \mathbb{R}^{T_0 \times N_0}$ is diagonal with non-negative singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. Centering ensures PCA focuses on the variation around the mean, not the absolute values. The columns of $\mathbf{V}$ define the principal components, i.e., directions through the "cloud of donor points" that explain maximal variance. Projecting the donor series onto the top K components,

$$\mathbf{Z}_K = (\mathbf{Y}_0 - \bar{\mathbf{Y}}_0)\mathbf{V}_K, \quad \mathbf{V}_K \in \mathbb{R}^{N_0 \times K},$$

provides a low-dimensional summary of the donor behavior. Each row of $\mathbf{Z}_K$ gives the coordinates of a donor in this K-dimensional subspace—analogous to the projected coordinates along a line of best fit in 2D. The original donor matrix can then be approximately reconstructed as

$$\hat{\mathbf{Y}}_0^{(K)} = \mathbf{Z}_K \mathbf{V}_K^\top + \bar{\mathbf{Y}}_0,$$

where $\hat{\mathbf{Y}}_0^{(K)}$ captures the bulk of the variation in the donors while filtering out minor fluctuations or noise. This is entirely analogous to fitting a line of best fit in two dimensions: here, instead of one line, we have a few stiff directions through the high-dimensional cloud of points.

To visualize this in 2D, we load the Basque data and slice the donor matrix off by its pre-intervention period (before 1975).

Next, we fit the principal components. I do this both in SVD and PCA, but either way is correct:

We can see that while we have 16 donor units at hand, we only need 4 or 5 components to explain 99.23 percent of the variation in the donors. The colorful lines are the values taken by the reconstructed donor matrix, and this is what we use to fit the target series that is treated.

9.3.2 Singular Value Thresholding

To select K, we use procedure called Universal Singular Value Thresholding developed by Chatterjee (2015). USVT works by analyzing the singular values from the SVD of the centered donor matrix. Recall that our singular values measure the importance of each principal component, with larger values indicating directions that capture more variance. USVT selects K by setting a threshold to distinguish significant singular values (signal) from smaller ones likely representing noise. To choose K with USVT, we first calculate the matrix's aspect ratio, defined as the ratio of its smaller dimension to its larger dimension, i.e.,

$$r = \frac{\min(T_0, N_0)}{\max(T_0, N_0)},$$

where T_0 is the number of pre-treatment periods and N_0 is the number of donor units. For the Basque data, the pre-treatment donor matrix has shape 20×16 (20 time periods, 16 donors), so the aspect ratio is $16/20 = 0.8$. Next, USVT computes a multiplier, called the omega factor, based on the aspect ratio using the formula:

$$\omega = 0.56 \cdot r^3 - 0.95 \cdot r^2 + 1.82 \cdot r + 1.43$$

Essentially, we adjust the threshold to account for how the matrix's shape affects the distribution of singular values. For our Basque data with an aspect ratio of 0.8, we calculate:

$$\omega = 0.56 \cdot (0.8)^3 - 0.95 \cdot (0.8)^2 + 1.82 \cdot 0.8 + 1.43 \approx 2.584$$

The omega factor ensures the threshold is appropriately scaled for matrices of different shapes (e.g., tall and thin vs. wide and short). The threshold for retaining singular values is then set as:

$$\text{threshold} = \omega \cdot \text{median}(\sigma_1, \sigma_2, \ldots, \sigma_{\min(T_0, N_0)})$$

The median singular value is used because it provides a robust measure of the typical scale of singular values, less sensitive to outliers than the mean. Singular values above this threshold are considered significant, capturing the main patterns in the data, while those below are treated as noise. The number of principal components, K, is the count of singular values greater than the threshold:

$$K = \text{number of } \sigma_i \text{ where } \sigma_i > \text{threshold}$$

Once the rank K has been selected, we rebuild the donor matrix using only the top K singular components. Recall that the centered donor matrix can be decomposed by Singular Value Decomposition (SVD) as

$$\mathbf{Y}_0 - \bar{\mathbf{Y}}_0 = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^\top,$$

where $\mathbf{U} \in \mathbb{R}^{T_0 \times T_0}$ and $\mathbf{V} \in \mathbb{R}^{N_0 \times N_0}$ are orthogonal matrices, and $\mathbf{\Sigma} \in \mathbb{R}^{T_0 \times N_0}$ is diagonal with singular values $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$. To obtain a low-rank approximation that captures the main structure of $\mathbf{Y}_0$, we retain only the first K singular values and their corresponding singular vectors:

$$\mathbf{U}_K = [\mathbf{u}_1, \ldots, \mathbf{u}_K], \quad \mathbf{\Sigma}_K = \text{diag}(\sigma_1, \ldots, \sigma_K), \quad \mathbf{V}_K = [\mathbf{v}_1, \ldots, \mathbf{v}_K].$$

The low-rank reconstruction of the donor matrix is then given by

$$\mathbf{Y}_0^{(K)} = \mathbf{U}_K \mathbf{\Sigma}_K \mathbf{V}_K^\top + \bar{\mathbf{Y}}_0.$$

This reconstruction, $\mathbf{Y}_0^{(K)}$, preserves the dominant co-movements across donors while filtering out minor, idiosyncratic variations. In practice, this means that each donor's trajectory is projected into a K-dimensional subspace that captures the most important temporal and cross-sectional patterns.

Now, we can perform linear regression. Like the above, our target vector is the pre-treatment outcome of the treated unit. However, instead of the raw donor matrix as predictors, we use the denoised version of the donor matrix:

$$\mathbf{w}^* = \underset{\mathbf{w} \in \mathbb{R}^{N_0}}{\text{argmin}} \, \|\mathbf{y}_1 - \mathbf{Y}_0^{(K)}\mathbf{w}\|_2^2 + \lambda \, \|\mathbf{w}\|_p^q.$$

Here the target vector is the outcomes of the treated unit and $\mathbf{Y}_0^{(K)}$ is the denoised donor matrix. Because this is just a form of OLS, we can also impose the standard penalties we have discussed above to the PCR setting. To implement this, we use the `CLUSTERSC` class from `mlsynth`. This method calls on the `pcr` method, which itself calls on the `Opt.SCopt` class. Below, I run unregularized PCR (which I just refer to as OLS) and PCR with an ℓ_1 and ℓ_2 penalty on the weights. I use a lambda of 0.05 for both. Below, here are the weights.

```
from mlsynth import CLUSTERSC
import pandas as pd

# Load Basque dataset
url = (
```

```
    "https://raw.githubusercontent.com/"
    "jgreathouse9/mlsynth/"
    "refs/heads/main/basedata/"
    "basque_data.csv"
)
data = pd.read_csv(url)

# Base configuration
base_config = {
    "df": data,
    "outcome": data.columns[2],
    "treat": data.columns[-1],
    "unitid": data.columns[0],
    "time": data.columns[1],
    "display_graphs": False,
    "save": False,
    "counterfactual_color": ["red"],
    "cluster": False,
    "method": "PCR"
}

# Define model setups
model_setups = {
    "OLS": {"lambda_penalty": 0, "p": 1, "q": 1},
    "Lasso": {"lambda_penalty": 0.05, "p": 1, "q": 1},
    "Ridge": {"lambda_penalty": 0.05, "p": 2, "q": 2}
}

# Containers for results
weight_dict = {}
counterfactual_dict = {}
att_dict = {}
rmse_dict = {}

# Fit each model
for name, params in model_setups.items():
    config = base_config.copy()
    config.update(params)

    model = CLUSTERSC(config)
    arco = model.fit()

    weights = arco.sub_method_results["PCR"].weights.donor_weights
    counterfactual = arco.sub_method_results["PCR"].time_series.counterfactual_outcome
```

```
    weight_dict[name] = weights
    counterfactual_dict[name] = counterfactual

    att_dict[name] = arco.sub_method_results["PCR"].effects.additional_effects.get("ATT"
    rmse_dict[name] = arco.sub_method_results["PCR"].fit_diagnostics.additional_metrics.


# Convert each donor_weights dict to a Series and round to 3 decimals
weights_df = pd.DataFrame({
    name: pd.Series({k: round(v, 3) for k, v in weights.items()})
    for name, weights in weight_dict.items()
})


display(Markdown(weights_df.to_markdown()))
```

These weights return these predictions:

along with these ATTs and RMSEs for each penalized SC method.

We can see that the PCR approach effectively regularizes the OLS fit compared to the non-PCR OLS fit. Although OLS in theory allows weights to freely adapt to noise, PCR implicitly regularizes the solution by expressing $\mathbf{Y}_0$ only in terms of its dominant components. This effectively smooths the fit, as high-variance, low-signal directions are penalized. This has the effect of distributing weight across donors that explain the largest share of GDP variation in the pre-treatment period, while downweighting minor, noisy fluctuations. As a result, the synthetic Basque GDP is smoother, more stable, and driven by the main patterns in the donor pool rather than overfitting to idiosyncratic deviations.

Looking at the actual donor weights, we see that OLS assigns weight to multiple regions, including Cataluna (0.264) and Madrid (0.277), but also some negative weights such as Canarias (-0.218). Ridge regularization keeps a similar distribution but slightly shrinks extreme values toward zero, producing a smoother set of weights. Lasso, on the other hand, sparsifies the solution dramatically, assigning nearly all the weight to Cataluna (0.867) and Madrid (0.134) while zeroing out contributions from most other donors.

These weight patterns are reflected in the treatment effect estimates and pre-treatment RMSE. OLS produces an ATT of -0.849 with an RMSE of 0.077, indicating tight pre-treatment fit. The regularized solutions give very similar results, with slight differences in fit and ATT. Ridge gives a slightly smaller ATT compared to OLS (-0.826) with effectively the same RMSE, maintaining stability while smoothing extreme weights. Lasso, by concentrating weight on a few donors, produces a smaller ATT in magnitude (-0.732) and slightly higher pre-treatment RMSE (0.083), reflecting a trade-off between sparsity and fit. Overall, regularization allows the PCR approach to balance fidelity to the pre-treatment data with stability and interpretability of the donor weights.

9.4 Downsides of Penalized Least Squares

Implementing penalized least squares requires several practical tuning decisions. We must choose the size of the candidate λ grid. A coarse grid may miss the optimal λ, while an overly fine grid increases computational cost without guaranteed improvement. The grid spacing is also critical. Analysts can linearly, logarithmically, or geometrically space lambda values. This choice also matters. A logarithmic scale can capture behavior when small changes in λ have large effects, whereas a linear scale may suffice when sensitivity is low.

Beyond the choices, another factor is how these methods influence the interpretation of the decisions. LASSO's sparsity can simplify interpretation by selecting a few donors, but the choice of which donors remain can be sensitive to small changes in the data. Ridge distributes weights more evenly, making the model stable, but at the cost of obscuring which units are most influential. In practice, both penalties can shift the relative importance of donors in ways that might not align with substantive knowledge. Penalized models also rely on pre-treatment fit to guide the penalty. If the pre-treatment period is short, noisy, or unrepresentative, the regularization can either overcorrect or undercorrect, affecting post-treatment predictions.

These challenges do not go away for PCR either. PCR requires choosing how many top singular values to retain. USVT is one way of doing this, but other methods have been proposed in the literature as well. For example, Gavish & Donoho (2014) find the optimal threshold to be $\frac{4}{\sqrt{3}}$ (also see Donoho et al., 2023). The practical consequences of this are simple: keeping too few components may miss important variation in the donor pool, underfitting the treated unit. Keeping too many may reintroduce noise, overfitting minor idiosyncrasies. There is no universally correct rule, and analysts may experiment according to different rules as per their context.

9.5 Onward to Convex SC

Regularization teaches OLS restraint. By adding a penalty term, we give the model a sense of proportion: fit matters, but so does humility. The ℓ_1 and ℓ_2 norms are two ways of communicating that discipline—one selective, the other inclusive. Both make OLS behave less like an overzealous mimic and more like a thoughtful imitator. Methods such as PCR allow us to exploit data-compression techniques in order to do counterfactual prediction. Yet, penalized methods still leave the model free to misbehave, for the reasons I mentioned above. To address this, we may wish to move beyond soft constraints upon the weights to harsher constraints with stricter rules to follow. This is the subject for the next chapter. Geometrically, we go from a soft, round penalty landscape to a sharp, polygonal constraint space. Conceptually, we shift from telling OLS to "behave" to explicitly defining what "behavior" is allowed.

Problem 9.17. Analogy

Come up with your own analogy to explain the ℓ_1 and ℓ_2 norms, to accompany my music analogy.

Problem 9.18. Understanding Overfitting

1. Define overfitting in the context of synthetic control.

2. Explain why OLS synthetic control tends to overfit the pre-treatment period in the Basque Country example.

3. How does overfitting affect out-of-sample (post-treatment) predictions?

Problem 9.19. L1 vs L2 Regularization

1. For the same dataset, fit both LASSO (L1) and Ridge (L2) penalized regressions.

2. Compare the donor weights obtained from each method. Which donors are selected or shrunk more under each norm?

3. Explain how the choice of λ affects the sparsity of the LASSO weights and the smoothness of Ridge weights.

Problem 9.20. Bias-Variance Tradeoff

1. Explain the bias-variance tradeoff in your own words.

2. Using your OLS, LASSO, and Ridge results, describe which models have higher bias or higher variance.

3. How does regularization help balance this tradeoff in synthetic control?

Problem 9.21. Implementing Penalized OLS

1. Using the `fit_penalized` function, compute penalized weights for the Basque dataset with $\lambda = 0.1, 0.5, 1.0$.

2. Plot the predicted counterfactuals for each λ.

3. Discuss how increasing λ affects the pre-treatment RMSE and the donor weight distribution.

Problem 9.22. Cross-Validation Setup

1. Explain the purpose of rolling-origin cross-validation in the context of synthetic control.

2. Define training and validation periods for a rolling window with initial train size 10 and step 5.

3. Why might a logarithmic grid of λ values be preferable to a linear grid in CV?

Problem 9.23. Selecting Optimal λ

1. Perform rolling-origin cross-validation to select the optimal λ for LASSO and Ridge.

2. Plot the cross-validated MSE against λ for each norm.

3. Report the λ that minimizes CV error and explain why it represents a balance between bias and variance.

Problem 9.24. ATT Estimation

1. Compute the average post-treatment treatment effect (ATT) for OLS, LASSO, and Ridge synthetic controls.

2. Compare the magnitude and sign of the ATT estimates across models.

3. Interpret the differences in the context of penalization and overfitting.

Problem 9.25. Hyperparameter Sensitivity

1. Explore how changing the size of the λ grid affects the CV-selected λ.

2. Try linear, logarithmic, and geometric grids.
3. Discuss how grid choice could influence your synthetic control results.

Abadie, A., Diamond, A., & Hainmueller, J. (2010). Synthetic control methods for comparative case studies: Estimating the effect of california's tobacco control program. *Journal of the American Statistical Association*, *105*(490), 493–505. https://doi.org/10.1198/jasa.2009.ap08746

Abadie, A., Diamond, A., & Hainmueller, J. (2015). Comparative politics and the synthetic control method. *American Journal of Political Science*, *59*(2), 495–510. https://doi.org/https://doi.org/10.1111/ajps.12116

Agarwal, A., Shah, D., Shen, D., & Song, D. (2021). On robustness of principal component regression. *Journal of the American Statistical Association*, *116*(536), 1731–1745. https:

//doi.org/10.1080/01621459.2021.1928513

Amjad, M., Shah, D., & Shen, D. (2018). Robust synthetic control. *Journal of Machine Learning Research*, *19*(22), 1–51. http://jmlr.org/papers/v19/17-777.html

Baker, A., Callaway, B., Cunningham, S., Goodman-Bacon, A., & Sant'Anna, P. H. C. (2025). *Difference-in-differences designs: A practitioner's guide*. https://arxiv.org/abs/2503.13323

Brabander, E. de, Juodis, A., & Szini, G. M. (2025). On the use of synthetic difference-in-differences approach with (-out) covariates: The case study of brexit referendum. *Econometric Reviews*, *44*(10), 1617–1646. https://doi.org/10.1080/07474938.2025.2530649

Chatterjee, S. (2015). Matrix estimation by Universal Singular Value Thresholding. *The Annals of Statistics*, *43*(1), 177–214. https://doi.org/10.1214/14-AOS1272

Cunningham, S. (2021). *Causal inference: The mixtape*. Yale University Press. https://mixtape.scunning.com/

Donoho, D., Gavish, M., & Romanov, E. (2023). ScreeNOT: Exact MSE-optimal singular value thresholding in correlated noise. *The Annals of Statistics*, *51*(1), 122–148. https://doi.org/10.1214/22-AOS2232

Facure Alves, M. (2022). *Causal inference for the brave and true*. https://matheusfacure.github.io/python-causality-handbook

Filipovic, D., & Schneider, P. (2025). *Fundamental properties of linear factor models*. https://arxiv.org/abs/2409.02521

Gavish, M., & Donoho, D. L. (2014). The optimal hard threshold for singular values is $4/\sqrt{3}$. *IEEE Transactions on Information Theory*, *60*(8), 5040–5053. https://doi.org/10.1109/TIT.2014.2323359

Holland, P. W. (1986). Statistics and causal inference. *Journal of the American Statistical Association*, *81*(396), 945–960. https://doi.org/10.1080/01621459.1986.10478354

Huntington-Klein, N. (2025). *The effect: An introduction to research design and causality* (2nd ed.). Chapman; Hall/CRC. https://theeffectbook.net/

Li, K. T., & Shankar, V. (2024). A two-step synthetic control approach for estimating causal effects of marketing events. *Management Science*, *70*(6), 3734–3747. https://doi.org/10.1287/mnsc.2023.4878

Li, K. T., & Van den Bulte, C. (2023). Augmented difference-in-differences. *Marketing Science*, *42*(4), 746–767. https://doi.org/10.1287/mksc.2022.1406

Liao, C., Shi, Z., & Zheng, Y. (2025). *A relaxation approach to synthetic control*. https://arxiv.org/abs/2508.01793

Nguyen, M. (2025). Synthetic control. In *A guide on data analysis*. https://bookdown.org/mike/data_analysis/sec-synthetic-control.html.

Rubin, D. B. (2008). For objective causal inference, design trumps analysis. *The Annals of Applied Statistics*, *2*(3), 808–840. https://doi.org/10.1214/08-AOAS187

Shi, Z. (n.d.). *Lecture notes on econometrics*. https://zhentaoshi.github.io/Econ5121A/.

Stöffelbauer, A. (2023). *Causal inference using synthetic controls*. https://medium.com/data-science-at-microsoft/causal-inference-using-synthetic-controls-d96a890c83a7.

Tocqueville, A. de, Mansfield, H. C., & Winthrop, D. (2012). *Democracy in america*. University of Chicago Press. https://books.google.com/books?id=obyzxRmY7nEC